Natural Language Processing

Shawon Sir's Lectures

CSE 4107 ? Natural Language Processing

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

Table of Contents

1	Lecture 01: Introduction to NLP	Page 3
2	Lecture 02: Part-of-Speech (POS) Tagging	Page 17
3	Lecture 03: Hidden Markov & Maximum Entropy Models	Page 53
4	Lecture 04: Recurrent Neural Networks (RNN) & LSTM	Page 115
5	Lecture 05: Transformers & Attention Mechanisms	Page 174
6	Lecture 06: Masked Language Models (BERT)	Page 242
7	Lecture 07: Information Retrieval & RAG	Page 277
8	Lecture 08: Speech Synthesis (TTS)	Page 379

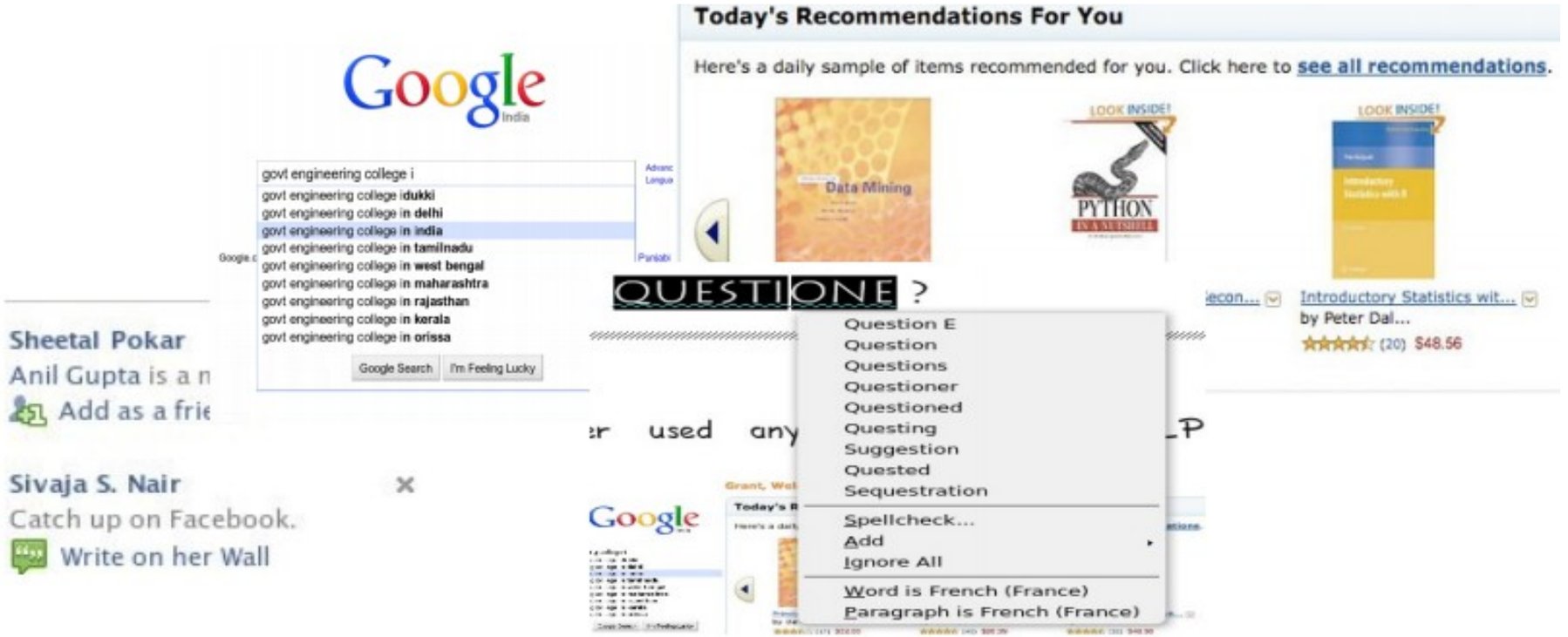
Natural Language Processing

Outline

- **Use of NLP**
- **Introduction**
- **Why NLP ?**
- **Areas in NLP**
- **References**

Use of NLP

➢ **Have you ever used any NLP products/ NLP Powered tools ?**



Natural Language Processing

- **A sub-field of Artificial Intelligence (AI)**
- **An inter disciplinary subject.**
- **Aim:**
- **To build intelligent computers that can interact with human being like a human being !!**

Definition

Natural language processing (NLP) is a field of computer science, artificial intelligence (also called machine learning), and linguistics concerned with the interactions between computers and human (natural) languages. Specifically, the process of a computer extracting meaningful information from natural language input and/or producing natural language output.

Why NLP

- **Huge amounts of data**
  - **Internet => at least 20 billions pages**
  - **Text data => web sites, blog, tweets .....**
  - **Audio data => speech .....**
- **Impossible to read all data and then extract information**
- **Applications for processing large amounts of texts require NLP expertise.**

Example

➢ **News: AN EARTHQUAKE struck Indonesia today - a strapping 7.7 magnitude earthquake that struck early today off the northern coast of the island of Sumatra. It caused minor damage and there are no reports of any deaths, although electricity was interrupted in several places.**

➢ **Information From the News:**

Location : Indonesia
Magnitude: 7.7
Region: Sumatra (Northern Cost)
Deaths: Nil
Damage: Minor

Major Areas of Research & Development

➢ **Text Processing**
- processing of raw text.

➢ **Morphological Analysis – study of word formation (how words are built up from smaller pieces)**
- Washing = Wash + ing
- Running = Run + ing

➢ **POS Tagging**

```
>> text = word_tokenize(“today is a beautiful day")
>> nltk.pos_tag(text)
```

Output: [('today', 'NN'), ('is', 'PRP'), ('a', ‘DT'), ('beautiful', 'JJ'), ('day', 'NN')]

Major Areas of Research & Development

- **Machine Translation**

  Translating content in one natural language to another natural language. Example :

  আমার পড়াশোনা ভালো লাগে না **=> I don't like studying.**

- **Parsing**

  Identifying sentence structure
  S → NP + VP

- **Text to speech**

  Converting electronic text to digital speech.

Major Areas of Research & Development

- **Automatic Speech Recognition**

  Automatic transcription of spoken content to electronic text

- **Speech to speech translation**

  Translating spoken content from one language to another in real time or offline.

industrial Applications

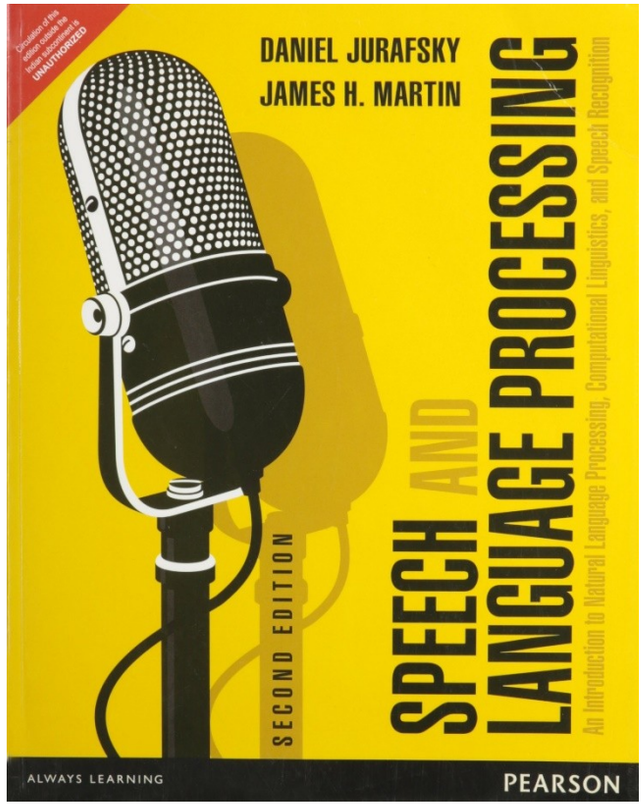
- **Search Engines**
- **Fraud Detection**
- **Information Extraction**
- **Collaborative filtering**
- **Computational Advertising**
- **Advanced Text Editors**
- **Sentiment Analysis**
- **Opinion Mining**

NLP in other Domains

- **Bio-Medical**
- **Forensic Science**
- **Advertisement**
- **Politics**
- **E-governance**
- **Business Development**
- **Marketing**
- **and where ever we use language !!!**

References

- “Speech and Language Processing”
  by Daniel Jurafsky & James H. Martin



Thank You ☺

Chapter 5. Part-of-Speech Tagging

From: Chapter 5 of *An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*, by Daniel Jurafsky and James H. Martin

Background

- **Part of speech**:
  - Noun, verb, pronoun, preposition, adverb, conjunction, particle, and article
- Recent lists of **POS** (also know as **word classes**, **morphological class**, or **lexical tags**) have much larger numbers of word classes.
  - 45 for Penn Treebank
  - 87 for the Brown corpus, and
  - 146 for the C7 tagset
- **The significance of the POS for language processing is that it gives a significant amount of information about the word and its neighbors**.
- **POS can be used in stemming for information retrieval(IR)**, since
  - Knowing a word's POS can help tell us which morphological affixes it can take.
  - They can help an IR application by helping select out nouns or other important words from a document.

8.1 English Word Classes

- Give a more complete definition of the classes of POS.
  - Traditionally, the definition of POS has been based on morphological and syntactic function.
  - While, it has tendencies toward semantic coherence (e.g., nouns describe “people, places, or things and adjectives describe properties), this is not necessarily the case.
- Two broad subcategories of POS:
  1. **Closed class**
  2. **Open class**

8.1 English Word Classes

1. **Closed class**
   - Having relatively fixed membership, e.g., prepositions (because new prepositions are rarely coined)
   - **Function words:**
     - Grammatical words like *of*, *and*, or *you*, which tend to be very short, occur frequently, and play an important role in grammar.
2. **Open class**
   - Four major open classes occurring in the languages of the world: **nouns**, **verbs**, **adjectives**, and **adverbs.**
     - Many languages have no adjectives, e.g., the native American language Lakhota, and Chinese

8.1 English Word Classes
Open Class: Noun

- **Noun**
  - The name given to the lexical class in which the words for most people, places, or things occur
  - Since lexical classes like noun are defined functionally (morphological and syntactically) rather than semantically,
    - some words for people, places, or things may not be nouns, and conversely
    - some nouns may not be words for people, places, or things.
  - Thus, nouns include
    - Concrete terms, like *ship*, and *chair*,
    - Abstractions like *bandwidth* and *relationship*, and
    - Verb-like terms like *pacing*
  - Noun in English
    - Things to occur with determiners (*a goat*, *its bandwidth*, *Plato's Republic*),
    - To take possessives (*IBM's annual revenue*), and
    - To occur in the plural form (*goats*, *abaci*)

8.1 English Word Classes

Open Class: Noun

- Nouns are traditionally grouped into **proper nouns** and **common nouns.**
  - **Proper nouns:**
    - Names of specific persons or entities *(e.g. Rahim, Dhaka,* and *IBM)*
    - Generally not preceded by articles.
  - **Common nouns**
    - **Count nouns:**
      - Allow grammatical enumeration, i.e., both singular and plural (*goal/goals*), and can be counted (*one goal/ two goals*)
    - **Mass nouns:**
      - Something is conceptualized as a homogeneous group. (e.g. *snow, salt)*
      - Appear without articles where singular nouns cannot (*Snow is white* but not **Goal is white*)

8.1 English Word Classes

Open Class: Verb

- **Verbs**
  - Most of the words referring to actions and processes including main verbs like *draw*, *provide*, *differ*, and *go*.
  - A number of morphological forms: non-3rd-person-sg (*eat*), 3rd-person-sg(*eats*), progressive (*eating*), past participle (*eaten*)
  - A subclass: **auxiliaries** (discussed in closed class)

8.1 English Word Classes

Open Class: Adjectives

- **Adjectives**
  - Terms describing properties or qualities
  - Most languages have adjectives for the concepts of color (*white*, *black*), age (*old*, *young*), and value (*good*, *bad*), but
  - There are languages without adjectives, e.g., Chinese.

8.1 English Word Classes

Open Class: Adverbs

- **Adverbs**
  - Words viewed as modifying something (often verbs)
    - **Directional (or locative) adverbs:** specify the direction or location of some action, *hoe, here, downhill*
    - **Degree adverbs:** specify the extent of some action, process, or property, *extremely, very, somewhat*
    - **Manner adverb:** describe the manner of some action or process, *slowly, slinkily, delicately*
    - **Temporal adverbs:** describe the time that some action or event took place, *yesterday, Monday*

8.1 English Word Classes
Open Classes

- Some important closed classes in English
  - **Prepositions:** on, under, over, near, by, at, from, to, with
  - **Determiners:** a, an, the
  - **Pronouns:** she, who, I, others
  - **Conjunctions:** and, but, or, as, if, when
  - **Auxiliary verbs:** can, may, should, are
  - **Particles:** up, down, on, off, in, out, at, by
  - **Numerals:** one, two, three, first, second, third

8.1 English Word Classes
Open Classes: Prepositions

- **Prepositions** occur before noun phrases, semantically they are relational
  - Indicating spatial or temporal relations, whether literal (*on it, before then, by the house*) or metaphorical (*on time, with gusto, beside herself*)
  - Other relations as well

of	540,085	through	14,964	worth	1,563	pace	12
in	331,235	after	13,670	toward	1,390	nigh	9
for	142,421	between	13,275	plus	750	re	4
to	125,691	under	9,525	till	686	mid	3
with	124,965	per	6,515	amongst	525	o'er	2
on	109,129	among	5,090	via	351	but	0
at	100,169	within	5,030	amid	222	ere	0
by	77,794	towards	4,700	underneath	164	less	0
from	74,843	above	3,056	versus	113	midst	0
about	38,428	near	2,026	amidst	67	o'	0
than	20,210	off	1,695	sans	20	thru	0
over	18,071	past	1,575	circa	14	vice	0

- *Preposition (and particles) of English from CELEX online dictionary.*
- *Frequency counts are from the COBUILD 16-million-word corpus.*

8.1 English Word Classes
Open Classes: Particles

- A **particle** is a word that resembles a preposition or an adverb, and that often combines with a verb to form a larger unit call a **phrasal verb**

  So I *went on* for some days cutting and hewing timber …

  Moral reform is the effort to *throw off* sleep …

aboard	aside	besides	forward(s)	opposite	through
about	astray	between	home	out	throughout
above	away	beyond	in	outside	together
across	back	by	inside	over	under
ahead	before	close	instead	overhead	underneath
alongside	behind	down	near	past	up
apart	below	east, etc.	off	round	within
around	beneath	eastward(s),etc.	on	since	without

English single-word particles from Quirk, et al (1985)

8.1 English Word Classes
Open Classes: Articles and Conjunctions

- English has three articles: *a*, *an*, and *the*
  - Articles begin a noun phrase.
  - Articles are frequent in English.

- **Conjunctions**
  - used to join two phrases, clauses, or sentences.
  - *and*, *or,* or, *but*
  - Subordinating conjunctions are used when one of the elements is of some sort of embedded status. *I thought* ***that*** *you might like some milk.*

and	514,946	yet	5,040	considering	174	forasmuch as	0
that	134,773	since	4,843	lest	131	however	0
but	96,889	where	3,952	albeit	104	immediately	0
or	76,563	nor	3,078	providing	96	in as far as	0
as	54,608	once	2,826	whereupon	85	in so far as	0
if	53,917	unless	2,205	seeing	63	inasmuch as	0
when	37,975	why	1,333	directly	26	insomuch as	0
because	23,626	now	1,290	ere	12	insomuch that	0
so	12,933	neither	1,120	notwithstanding	3	like	0
before	10,720	whenever	913	according as	0	neither nor	0
though	10,329	whereas	867	as if	0	now that	0
than	9,511	except	864	as long as	0	only	0
while	8,144	till	686	as though	0	provided that	0
after	7,042	provided	594	both and	0	providing that	0
whether	5,978	whilst	351	but that	0	seeing as	0
for	5,935	suppose	281	but then	0	seeing as how	0
although	5,424	cos	188	but then again	0	seeing that	0
until	5,072	supposing	185	either or	0	without	0

Coordinating and subordinating conjunctions of English From the CELEX. Frequency counts are from COBUILD (16 million words).

8.1 English Word Classes
Open Classes: Pronouns

- **Pronouns** act as a kind of shorthand for referring to some noun phrase or entity or event.
  - **Personal pronouns:** persons or entities (*you*, *she*, *I*, *it*, *me*, *etc*)
  - **Possessive pronouns:** forms of personal pronouns indicating actual possession or just an abstract relation between the person and some objects.
  - **Wh-pronouns:** used in certain question forms, or may act as complementizer.

it	199,920	how	13,137	yourself	2,437	no one	106
I	198,139	another	12,551	why	2,220	wherein	58
he	158,366	where	11,857	little	2,089	double	39
you	128,688	same	11,841	none	1,992	thine	30
his	99,820	something	11,754	nobody	1,684	summat	22
they	88,416	each	11,320	further	1,666	suchlike	18
this	84,927	both	10,930	everybody	1,474	fewest	15
that	82,603	last	10,816	ourselves	1,428	thyself	14
she	73,966	every	9,788	mine	1,426	whomever	11
her	69,004	himself	9,113	somebody	1,322	whosoever	10
we	64,846	nothing	9,026	former	1,177	whomsoever	8
all	61,767	when	8,336	past	984	wherefore	6
which	61,399	one	7,423	plenty	940	whereat	5
their	51,922	much	7,237	either	848	whatsoever	4
what	50,116	anything	6,937	yours	826	whereon	2
my	46,791	next	6,047	neither	618	whoso	2
him	45,024	themselves	5,990	fewer	536	aught	1
me	43,071	most	5,115	hers	482	howsoever	1
who	42,881	itself	5,032	ours	458	thrice	1
them	42,099	myself	4,819	whoever	391	wheresoever	1
no	33,458	everything	4,662	least	386	you-all	1
some	32,863	several	4,306	twice	382	additional	0
other	29,391	less	4,278	theirs	303	anybody	0
your	28,923	herself	4,016	wherever	289	each other	0
its	27,783	whose	4,005	oneself	239	once	0
our	23,029	someone	3,755	thou	229	one another	0
these	22,697	certain	3,345	'un	227	overmuch	0
any	22,666	anyone	3,318	ye	192	such and such	0
more	21,873	whom	3,229	thy	191	whate'er	0
many	17,343	enough	3,197	whereby	176	whenever	0
such	16,880	half	3,065	thee	166	whereof	0
those	15,819	few	2,933	yourselves	148	whereto	0
own	15,741	everyone	2,812	latter	142	whereunto	0
us	15,724	whatever	2,571	whichever	121	whichsoever	0

Pronouns of English from the CELEX on-line dictionary.

8.1 English Word Classes
Open Classes: Auxiliary Verbs

- **Auxiliary verbs:** mark certain semantic feature of a main verb, including
  - whether an action takes place in the present, past or future (tense),
  - whether it is completed (aspect),
  - whether it is negated (polarity), and
  - whether an action is necessary, possible, suggested, desired, etc (mood).
  - Including **copula** verb *be*, the two verbs *do* and *have* along with their inflection forms, as well as a class of **modal verbs.**

can	70,930	might	5,580	shouldn't	858
will	69,206	couldn't	4,265	mustn't	332
may	25,802	shall	4,118	'll	175
would	18,448	wouldn't	3,548	needn't	148
should	17,760	won't	3,100	mightn't	68
must	16,520	'd	2,299	oughtn't	44
need	9,955	ought	1,845	mayn't	3
can't	6,375	will	862	dare	??
have	???				

English modal verbs from the CELEX on-line dictionary.

8.1 English Word Classes
Open Classes: Others

- **Interjections:** *oh, hey, alas*
- **Negatives:** *no, not*
- **Politeness markers:** *please, thank you*
- **Greetings:** *hello, goodbye*
- Existential **there:** *there are two on the table*

8.2 Tagsets for English

- There are a small number of popular tagsets for English, many of which evolved from the 87-tag tagset used for the Brown corpus.
  - Three commonly used
    - **The small 45-tag Penn Treebank tagset**
    - The medium-sized 61 tag C5 tageset used by the Lancaster UCREL project's CLAWS tagger to tag the British National Corpus, and
    - The larger 146-tag C7 tagset

Tag	Description	Example	Tag	Description	Example
CC	Coordin. Conjunction	*and, but, or*	SYM	Symbol	+,%, &
CD	Cardinal number	*one, two, three*	TO	"to"	*to*
DT	Determiner	*a, the*	UH	Interjection	*ah, oops*
EX	Existential 'there'	*there*	VB	Verb, base form	*eat*
FW	Foreign word	*mea culpa*	VBD	Verb, past tense	*ate*
IN	Preposition/sub-conj	*of, in, by*	VBG	Verb, gerund	*eating*
JJ	Adjective	*yellow*	VBN	Verb, past participle	*eaten*
JJR	Adj., comparative	*bigger*	VBP	Verb, non-3sg pres	*eat*
JJS	Adj., superlative	*wildest*	VBZ	Verb, 3sg pres	*eats*
LS	List item marker	*1, 2, One*	WDT	Wh-determiner	*which, that*
MD	Modal	*can, should*	WP	Wh-pronoun	*what, who*
NN	Noun, sing. or mass	*llama*	WP$	Possessive wh-	*whose*
NNS	Noun, plural	*llamas*	WRB	Wh-adverb	*how, where*
NNP	Proper noun, singular	*IBM*	$	Dollar sign	$
NNPS	Proper noun, plural	*Carolinas*	#	Pound sign	#
PDT	Predeterminer	*all, both*	“	Left quote	(‘ or “)
POS	Possessive ending	*'s*	”	Right quote	(’ or ”)
PP	Personal pronoun	*I, you, he*	(	Left parenthesis	([, (, {, <)
PP$	Possessive pronoun	*your, one's*	)	Right parenthesis	( ],), }, >)
RB	Adverb	*quickly, never*	,	Comma	,
RBR	Adverb, comparative	*faster*	.	Sentence-final punc	(. ! ?)
RBS	Adverb, superlative	*fastest*	:	Mid-sentence punc	(: ; ... – -)
RP	Particle	*up, off*			

Penn Treebank POS tags

8.2 Tagsets for English

The(DT) grand(JJ) jury(NN) commented(VBD) on(IN) a(DT) number(NN) Of(IN) other(JJ) topics(NNS) .

- Certain syntactic distinctions were not marked in the Penn Treebank tagset because
  - Treebank sentences were parsed, not merely tagged, and
  - So some syntactic information is represented in the phrase structure.
- For example, prepositions and subordinating conjunctions were combined into the single tag *IN*, since the tree-structure of the sentence disambiguated them.

8.3 Part-of-Speech Tagging

- POS tagging
  - The process of assigning a POS or other lexical marker to each word in a corpus.
  - Also applied to punctuation marks
  - Thus, tagging for NL is the same process as **tokenization** for computer language, although tags for NL are much more ambiguous.
  - Taggers play an increasingly important role in speech recognition, NL parsing and Information Retrieval.

8.3 Part-of-Speech Tagging

- The input to a tagging algorithm is a string of words and a specified **tagset** of the kind described previously.

```
VB    DT   NN  .
Book that flight .

VBZ  DT  NN   VB     NN   ?
Does that flight serve dinner ?
```

- Automatically assigning a tag to a word is not trivial
  - For example, *book* is ambiguous: it can be a verb or a noun
  - Similarly, *that* can be a determiner, or a complementizer
- The problem of POS-tagging is to resolve the ambiguities, choosing the proper tag for the context.

8.3 Part-of-Speech Tagging

- How hard is the tagging problem?

Unambiguous (1 tag)	**35,340**	
Ambiguous (2–7 tags)	**4,100**	
2 tags	3,760	
3 tags	264	
4 tags	61	
5 tags	12	
6 tags	2	
7 tags	1	(“still”)

The number of word types in Brown corpus by degree of ambiguity.

- Many of the 40% ambiguous tokens are easy to disambiguate, because
  - The various tags associated with a word are not equally likely.

8.3 Part-of-Speech Tagging

- Many tagging algorithms fall into two classes:
  - **Rule-based** taggers
    - Involve a large database of hand-written disambiguation rule specifying, for example, that ***an ambiguous word is a noun rather than a verb if it follows a determiner***.
  - **Stochastic** taggers
    - Resolve tagging ambiguities by using a training corpus to *count the probability of a given word having a given tag in a given context*.
- The **Brill tagger**, called the **transformation-based tagger,** shares features of both tagging architecture.

8.4 Rule-Based Part-of-Speech Tagging

- The earliest algorithms for automatically assigning POS were based on a two-stage architecture
  - First, use a dictionary to assign each word a list of potential POS.
  - Second, use large lists of hand-written disambiguation rules to winnow down this list to a single POS for each word
- The **ENGTWOL** tagger (1995) is based on the same two stage architecture, with much more sophisticated lexicon and disambiguation rules than before.
  - Lexicon:
    - 56000 entries
    - A word with multiple POS is counted as separate entries

Word	POS	Additional POS features
smaller	ADJ	COMPARATIVE
entire	ADJ	ABSOLUTE ATTRIBUTIVE
fast	ADV	SUPERLATIVE
that	DET	CENTRAL DEMONSTRATIVE SG
all	DET	PREDETERMINER SG/PL QUANTIFIER
dog's	N	GENITIVE SG
furniture	N	NOMINATIVE SG NOINDEFDETERMINER
one-third	NUM	SG
she	PRON	PERSONAL FEMININE NOMINATIVE SG3
show	V	IMPERATIVE VFIN
show	V	PRESENT -SG3 VFIN
show	N	NOMINATIVE SG
shown	PCP2	SVOO SVO SV
occurred	PCP2	SV
occurred	V	PAST VFIN SV

Sample lexical entries from the ENGTWOL lexicon.

8.4 Rule-Based Part-of-Speech Tagging

- In the first stage of tagger,
  - each word is run through the two-level lexicon transducer and
  - the entries for all possible POS are returned.
- A set of about 1,100 constraints are then applied to the input sentences to rule out incorrect POS.

```
Pavlov     PALOV N NOM SG PROPER
had        HAVE V PAST VFIN SVO
           HAVE PCP2 SVO
shown      SHOW PCP2 SVOO SVO SV
that       ADV
           PRON DEM SG
           DET CENTRAL DEM SG
           CS
salivation  N NOM SG
…
```

8.4 Rule-Based Part-of-Speech Tagging

- A simplified version of the constraint:

ADVERBIAL-THAT RULE
Given input: “that”
if
(+1 A/ADV/QUANT); /* *if next word is adj, adverb, or quantifier* */
(+2 SENT-LIM); /* *and following which is a sentence boundary,* */
(NOT -1 SVOC/A); /* *and the previous word is not a verb like* */
/* *‘consider’ which allows adj as object complements* */
then eliminate non-ADV tags
else eliminate ADV tags

Example:
I can’t wait that long. – ‘that’ used as adverb
I consider that odd. – ‘that’ used as determiner

8.5 HMM Part-of-Speech Tagging

- We are given a sentence, for example, like

  Secretariat is expected to **race** tomorrow.

  - What is the best sequence of tags which corresponds to this sequence of words?
- We want: out of all sequences of *n* tags t_1^n the single tag sequence such that $P(t_1^n \mid w_1^n)$ is highest.

$$\hat{t}_1^n = \arg\max_{t_1^n} P(t_1^n \mid w_1^n)$$

  ˆ means “our estimate of the correct tag sequence”.

- It is not clear how to make the equation operational
  - that is, for a given tag sequence t_1^n and word sequence w_1^n, we don’t know how to directly compute $P(t_1^n \mid w_1^n)$.

8.5 HMM Part-of-Speech Tagging

$$P(x \mid y) = \frac{P(y \mid x)P(x)}{P(y)}$$

$$\hat{t}_1^n = \arg\max_{t_1^n} \frac{P(w_1^n \mid t_1^n)P(t_1^n)}{P(w_1^n)}$$

But $P(w_1^n)$ doesn't change for each tag sequence.

$$\hat{t}_1^n = \arg\max_{t_1^n} P(w_1^n \mid t_1^n)P(t_1^n)$$

likelihood *prior probability*

Still too hard to compute directly

8.5 HMM Part-of-Speech Tagging

Assumption 1: the probability of a word appearing is dependent only on its own part of speech tag:

$$P(w_1^n \mid t_1^n) \approx \prod_{i=1}^{n} P(w_i^n \mid t_i^n)$$

Assumption 2: the probability of a tag appearing is dependent only on the previous tag:

$$P(t_1^n) \approx \prod_{i=1}^{n} P(t_i \mid t_{i-1})$$

8.5 HMM Part-of-Speech Tagging

$$\hat{t}_1^n = \arg\max_{t_1^n} P(t_1^n \mid w_1^n) \approx \arg\max_{t_1^n} P(w_i \mid t_i) P(t_i \mid t_{i-1})$$

- This equation contains two kinds of probabilities,
  - tag transition probabilities and
  - word likelihoods.
- The tag transition probabilities: $P(t_i \mid t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$

$$P(NN \mid DT) = \frac{C(DT, NN)}{C(DT)} = \frac{56,509}{116,454} = .49$$

- The word likelihood probabilities: $P(w_i \mid t_i) = \frac{C(t_i, w_i)}{C(t_i)}$

$$P(is \mid VBZ) = \frac{C(VBZ, is)}{C(VBZ)} = \frac{10,073}{21,627} = .47$$

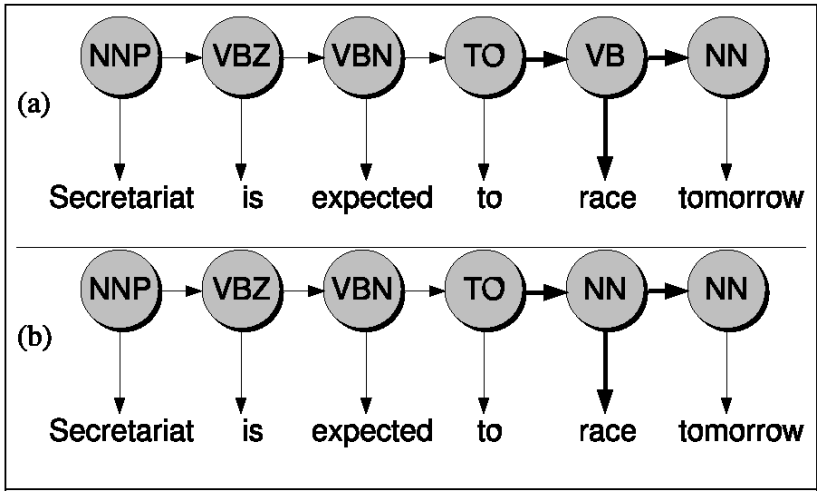
8.5 HMM Part-of-Speech Tagging

Computing the most-likely tag sequence: A motivating example

(8.36)**Secretariat**/NNP **is**/BEZ **expected**/VBN **to**/TO **race**/VB **tomorrow**/NR

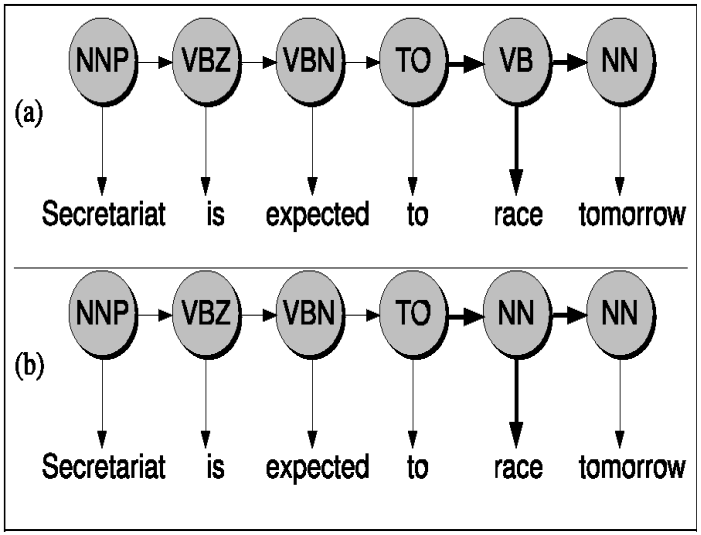
(8.37)**People**/NNS **continue**/VB **to**/TO **inquire**/VB **the**/AT **reason**/NN **for**/IN **the**/AT **race**/NN **for**/IN **outer**/JJ **space**/NN

- Let's look at how race can be correctly tagged as a VB instead of an NN in (8.36).



8.5 HMM Part-of-Speech Tagging

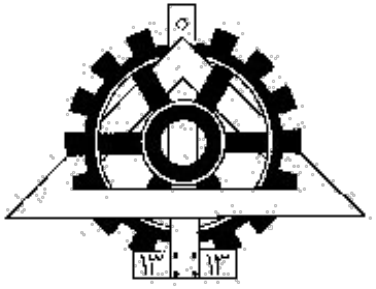
Computing the most-likely tag sequence: A motivating example



$$P(NN \mid TO) = .00047$$
$$P(VB \mid TO) = .83$$
$$P(race \mid NN) = .00057$$
$$P(race \mid VB) = .00012$$
$$P(NR \mid VB) = .0027$$
$$P(NR \mid NN) = .0012$$
$$P(VB \mid TO)P(NR \mid VB)P(race \mid VB) = .00000027$$
$$P(NN \mid TO)P(NR \mid NN)P(race \mid NN) = .00000000032$$

8.5 HMM Part-of-Speech Tagging

- Extending the HMM Algorithm to Trigrams (self)





Chapter 6

HIDDEN MARKOV AND MAXIMUM ENTROPY MODELS

Introduction

Statistical Models for processing text and speech:

- Hidden Markov Model (HMM)
- Maximum Entropy Model
- Maximum Entropy Markov Model (MEMM)

These are machine learning methods

- A **sequence classifier** or **sequence labeler** is a model whose job is to assign some label or class to each unit in a sequence
  - finite-state transducer is a non-probabilistic sequence classifier for transducing from sequences of words to sequences of morphemes
  - HMM and MEMM extend this notion by being probabilistic sequence classifiers

*

Markov chain

- Sometimes called **Observed Markov model**
- **Weighted finite-state automaton**

  - arc is associated with a probability

  - probability on all the arcs leaving a node must sum to 1.

- **Markov Chain**: a weighted automaton in which the input sequence uniquely determines which states the automaton will go through
  - Can't represent inherently ambiguous problems
  - Only useful for assigning probabilities to unambiguous sequences

*

Markov Chain

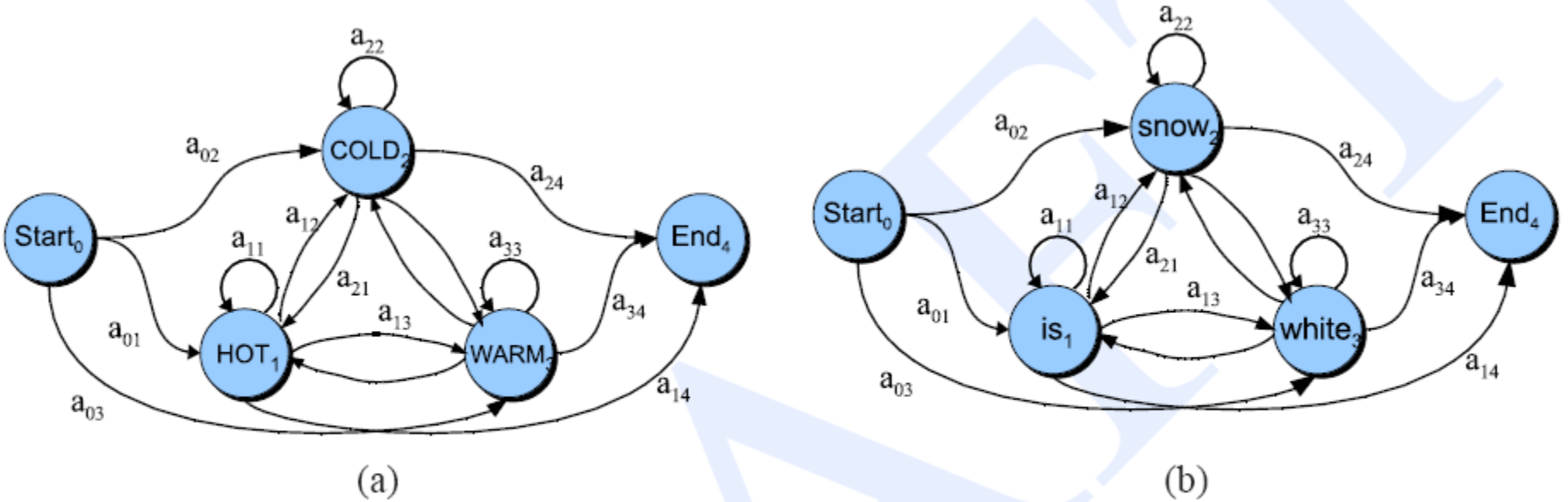


Figure 6.1 A Markov chain for weather (a) and one for words (b). A Markov chain is specified by the structure, the transition between states, and the start and end states.

*

Formal Description

- A Markov chain is specified by the following components:

$Q = q_1 q_2 \ldots q_N$	a set of N **states**
$A = a_{01} a_{02} \ldots a_{n1} \ldots a_{nn}$	a **transition probability matrix** A, each a_{ij} representing the probability of moving from state i to state j, s.t. $\sum_{j=1}^{n} a_{ij} = 1 \quad \forall i$
q_0, q_F	a special **start state** and **end (final) state** which are not associated with observations.

*

Formal Description

- **First-order Markov Chain**: the probability of a particular state is dependent only on the previous state
- **Markov Assumption:** $P(q_i | q_1 \ldots q_{i-1}) = P(q_i | q_{i-1})$

$\pi = \pi_1, \pi_2, \ldots, \pi_N$ an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^{n} \pi_i = 1$

$QA = \{q_x, q_y \ldots\}$ a set $QA \subset Q$ of legal **accepting states**

*

Markov Chain example

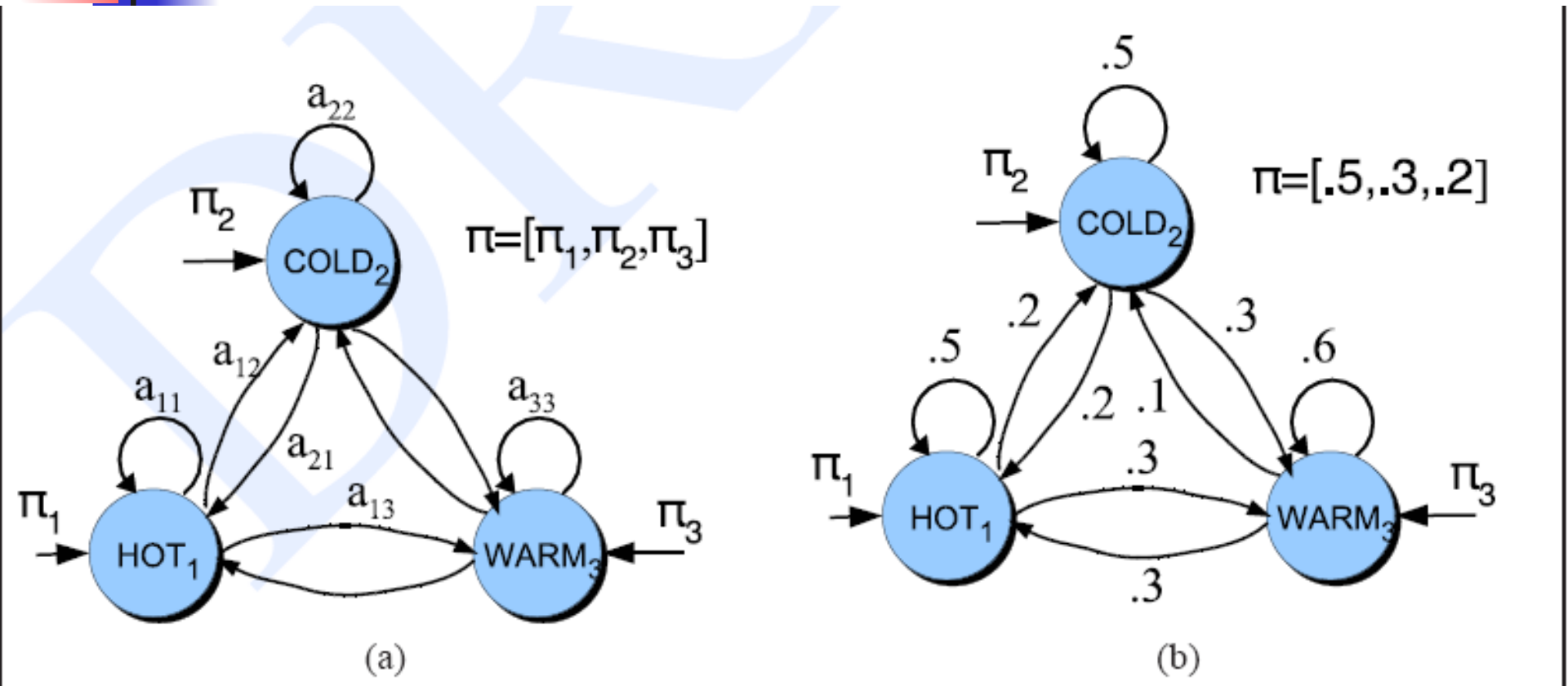


Figure 6.2 Another representation of the same Markov chain for weather shown in Fig. 6.1. Instead of using a special start state with a_{01} transition probabilities, we use the π vector, which represents the distribution over starting state probabilities. The figure in (b) shows sample probabilities.

Hidden Markov Model

- in POS tagging we didn't observe POS tags in the word; we saw words, and had to infer the correct tags from the word sequence. We call the POS tags **hidden** because they are not observed
- **HMM** allows us to talk HIDDEN MARKOV about both *observed* events (like words that we see in input) and *hidden* events (like POS tags) that we think of as causal factors in our probabilistic model

*

Jason Eisner (2002) example

- Imagine that you are a climatologist in the year 2150 studying the history of global warming. You cannot find any records of the weather in Dhaka, for the summer of 2007, but you do find Jason Eisner's diary, which lists how many ice creams Jason ate every day that summer.
- Our goal is to use these observations to estimate the temperature every day
  - Given a sequence of observations *O*, each observation an integer corresponding to the number of ice creams eaten on a given day, figure out the correct 'hidden' sequence *Q* of weather states (H or C) which caused Jason to eat the ice cream

*

Formal Description

$Q = q_1 q_2 \ldots q_N$	a set of N **states**
$A = a_{11} a_{12} \ldots a_{n1} \ldots a_{nn}$	a **transition probability matrix** A, each a_{ij} representing the probability of moving from state i to state j, s.t. $\sum_{j=1}^{n} a_{ij} = 1 \quad \forall i$
$O = o_1 o_2 \ldots o_T$	a sequence of T **observations**, each one drawn from a vocabulary $V = v_1, v_2, \ldots, v_V$.
$B = b_i(o_t)$	a sequence of **observation likelihoods:**, also called **emission probabilities**, each expressing the probability of an observation o_t being generated from a state i.
q_0, q_F	a special **start state** and **end (final) state** which are not associated with observations, together with transition probabilities $a_{01} a_{02} .. a_{0n}$ out of the start state and $a_{1F} a_{2F} \ldots a_{nF}$ into the end state.

Formal Description

$\pi = \pi_1, \pi_2, \ldots, \pi_N$ an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^{n} \pi_i = 1$

$QA = \{q_x, q_y \ldots\}$ a set $QA \subset Q$ of legal **accepting states**

Markov Assumption: $P(q_i | q_1 \ldots q_{i-1}) = P(q_i | q_{i-1})$

Output Independence Assumption: $P(o_i | q_1 \ldots q_i, \ldots, q_T, o_1, \ldots, o_i, \ldots, o_T) = P(o_i | q_i)$

*

HMM Example

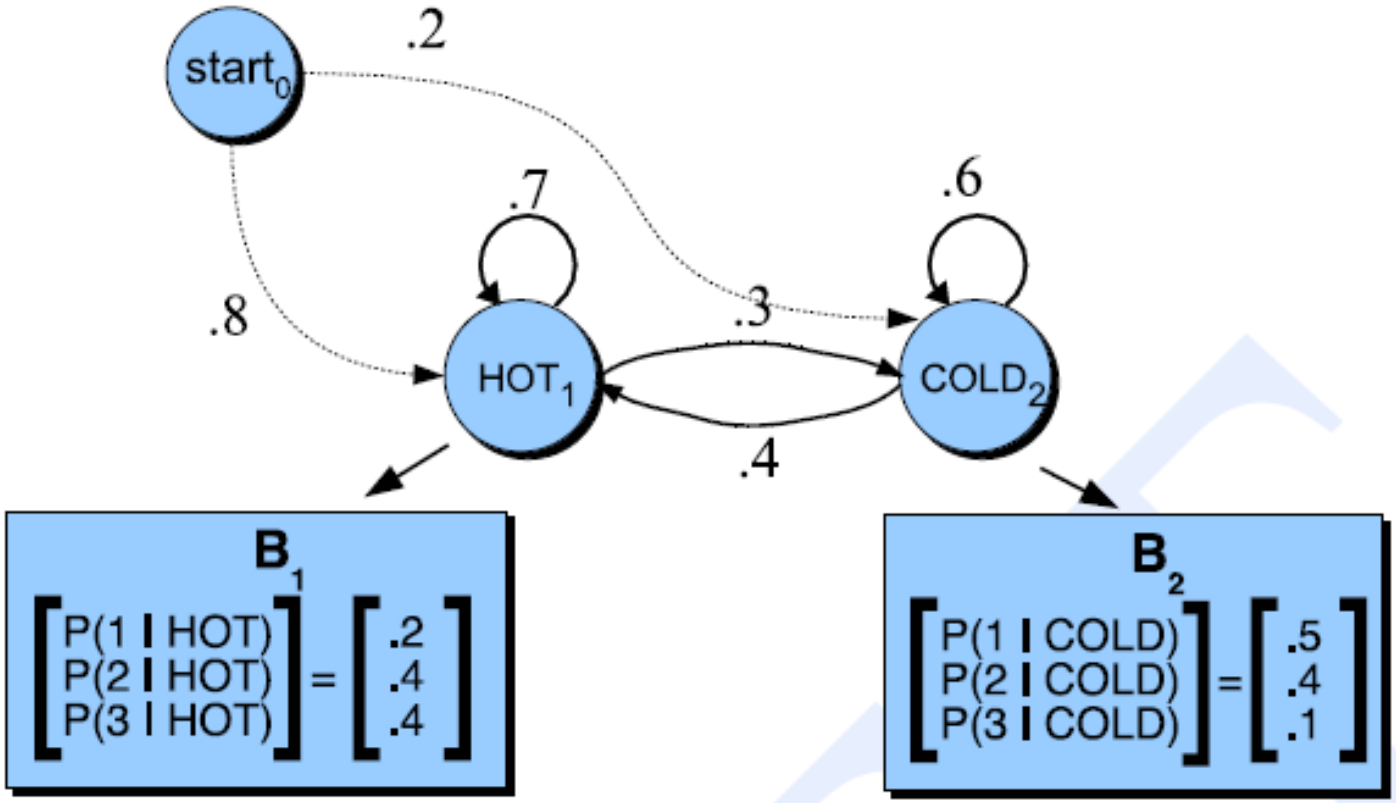


Figure 6.3 A Hidden Markov Model for relating numbers of ice creams eaten by Jason (the observations) to the weather (H or C, the hidden variables). For this example we are not using an end-state, instead allowing both states 1 and 2 to be a final (accepting) state.

Fully-connected (Ergodic) & **Left-to-right** (Bakis) HMM

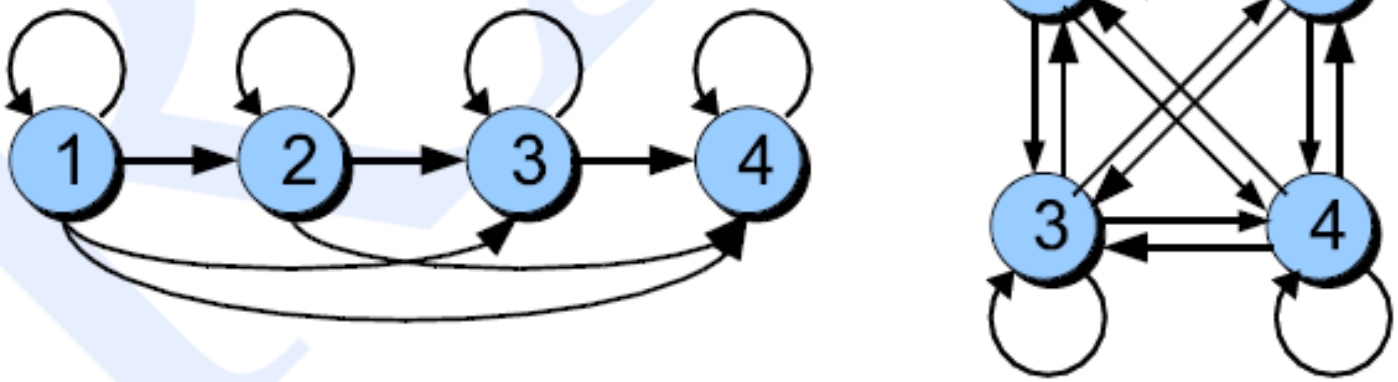


Figure 6.4 Two 4-state Hidden Markov Models; a left-to-right (Bakis) HMM on the left, and a fully-connected (ergodic) HMM on the right. In the Bakis model, all transitions not shown have zero probability.

*

HMM characterized by three fundamental problems

- **Problem 1 (Computing Likelihood):** Given an HMM $\lambda = (A,B)$ and an observation sequence O, determine the likelihood $P(O \mid \lambda)$

- **Problem 2 (Decoding):** Given an observation sequence O and an HMM $\lambda = (A,B)$, discover the best hidden state sequence Q

- **Problem 3 (Learning):** Given an observation sequence *O* and the set of states in the HMM, learn the HMM parameters *A* and *B*

*

COMPUTING LIKELIHOOD: THE FORWARD ALGORITHM

- Given an HMM $\lambda = (A,B)$ and an observation sequence O, determine the likelihood $P(O \mid \lambda)$
- For a Markov chain: we could compute the probability of *3 1 3* just by following the states labeled *3 1 3* and multiplying the probabilities along the arcs
- We want to determine the probability of an ice-cream observation sequence like *3 1 3*, but we don't know what the hidden state sequence is!
- Markov chain: Suppose we already knew the weather, and wanted to predict how much ice cream Jason would eat
  - For a given hidden state sequence (e.g. *hot hot cold*) we can easily compute the output likelihood of *3 1 3*.

*

THE FORWARD ALGORITHM

$$P(O) = \sum_{Q} P(O,Q) = \sum_{Q} P(O|Q)P(Q)$$

For our particular case, we would sum over the 8 three-event sequences *cold cold cold*, *cold cold hot*, i.e.:

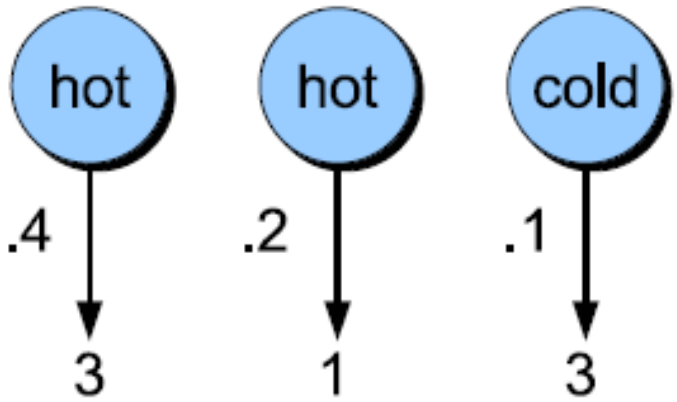
$$P(3\ 1\ 3) = P(3\ 1\ 3, \text{cold cold cold}) + P(3\ 1\ 3, \text{cold cold hot}) + P(3\ 1\ 3, \text{hot hot cold}) + \ldots$$

*

THE FORWARD ALGORITHM

$$P(O|Q) = \prod_{i=1}^{T} P(o_i|q_i)$$

$$P(3\ 1\ 3|\text{hot hot cold}) = P(3|\text{hot}) \times P(1|\text{hot}) \times P(3|\text{cold})$$

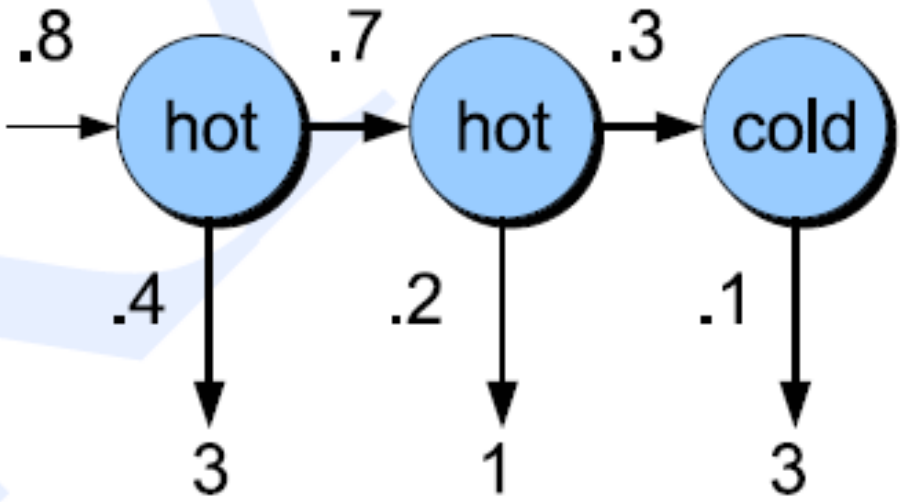


*

THE FORWARD ALGORITHM

$$P(O,Q) = P(O|Q) \times P(Q) = \prod_{i=1}^{n} P(o_i|q_i) \times \prod_{i=1}^{n} P(q_i|q_{i-1})$$

$$P(3\ 1\ 3, \text{hot hot cold}) = P(\text{hot}|\text{start}) \times P(\text{hot}|\text{hot}) \times P(\text{cold}|\text{hot}) \times P(3|\text{hot}) \times P(1|\text{hot}) \times P(3|\text{cold})$$



*

THE FORWARD ALGORITHM

- For and HMM with N hidden states and an observation sequence of T observations, there are N^T Possible hidden sequences.
- For real tasks, N and T are both large.
- Instead of using such an extremely exponential algorithm, we use an efficient $O(N^2T)$ algorithm called the forward algorithm.
- The forward algorithm is a kind of dynamic programming algorithm.

*

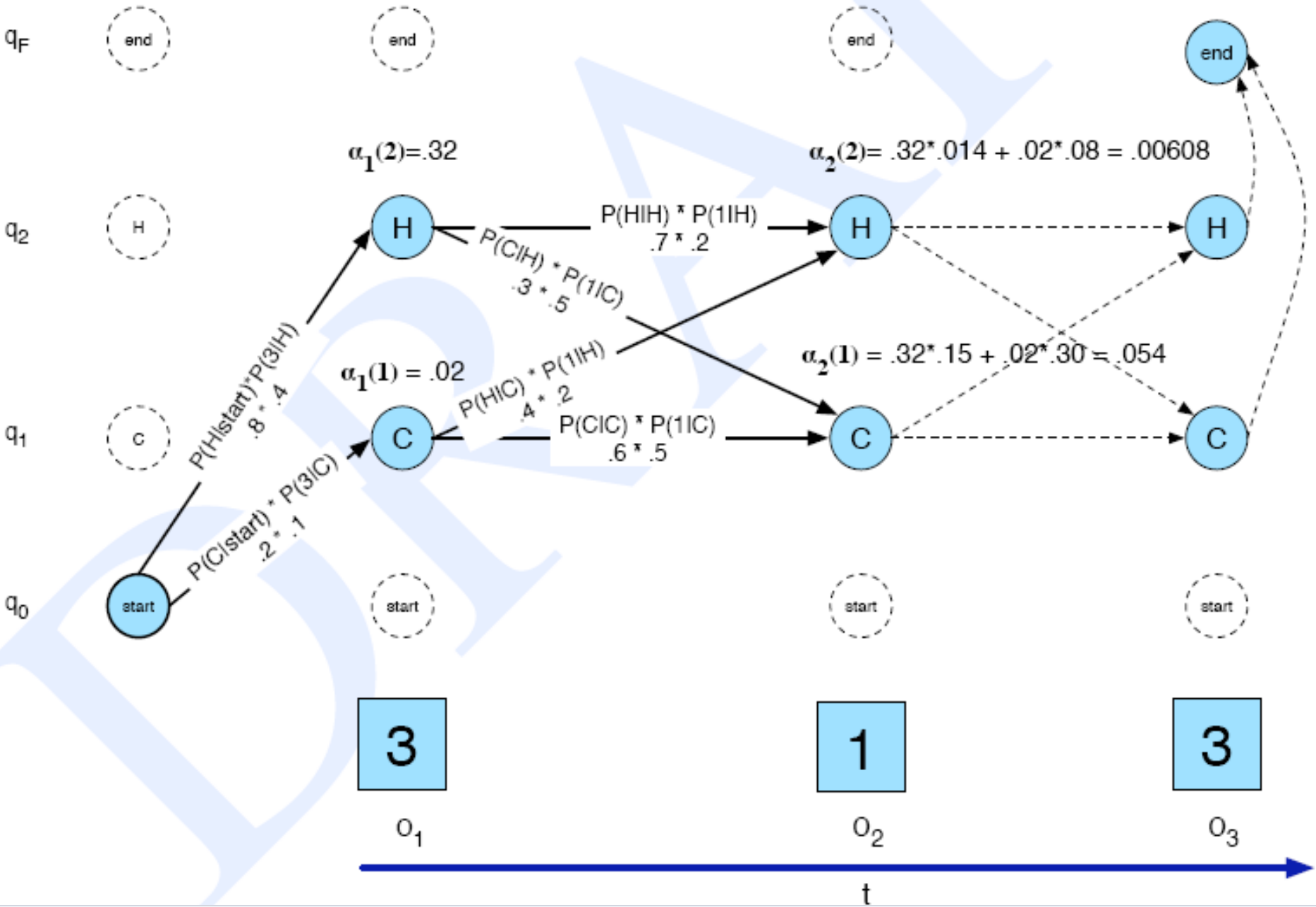
THE FORWARD ALGORITHM

- $\alpha_t(j)$ represents the probability of being in state *j* after seeing the first *t* observations, given the automaton λ
- $q_t = j$ means "the probability that the *t* th state in the sequence of states is state *j*"

$$\alpha_t(j) = P(o_1, o_2 \ldots o_t, q_t = j | \lambda)$$

$$\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t)$$

*


q_F
q_2
q_1
q_0
end
H
C
start
$\alpha_1(2)$=.32
$\alpha_1(1)$ = .02
$\alpha_2(2)$= .32*.014 + .02*.08 = .00608
$\alpha_2(1)$ = .32*.15 + .02*.30 = .054
P(H|start)*P(3|H)
.8 * .4
P(C|start) * P(3|C)
.2 * .1
P(H|H) * P(1|H)
.7 * .2
P(C|H) * P(1|C)
.3 * .5
P(H|C) * P(1|H)
.4 * .2
P(C|C) * P(1|C)
.6 * .5
3
1
3
O_1
O_2
O_3
t

THE FORWARD ALGORITHM

1. Initialization:

$$\alpha_1(j) = a_{0j} b_j(o_1) \quad 1 \le j \le N$$

2. Recursion (since states 0 and F are non-emitting):

$$\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t); \quad 1 \le j \le N, 1 < t \le T$$

3. Termination:

$$P(O|\lambda) = \alpha_T(q_F) = \sum_{i=1}^{N} \alpha_T(i)\, a_{iF}$$

N= num of hidden states
T= num of observation in O seq
$\alpha_t(j)$ represents the probability of being in state j after seeing the first t observations, given the automaton λ
bj(t)= the prob of obser bt generating in state j

Figure A.7 The forward algorithm, where *forward*$[s,t]$ represents $\alpha_t(s)$.

1. Initialization:

$$\alpha_1(j) = \pi_j b_j(o_1) \quad 1 \le j \le N$$

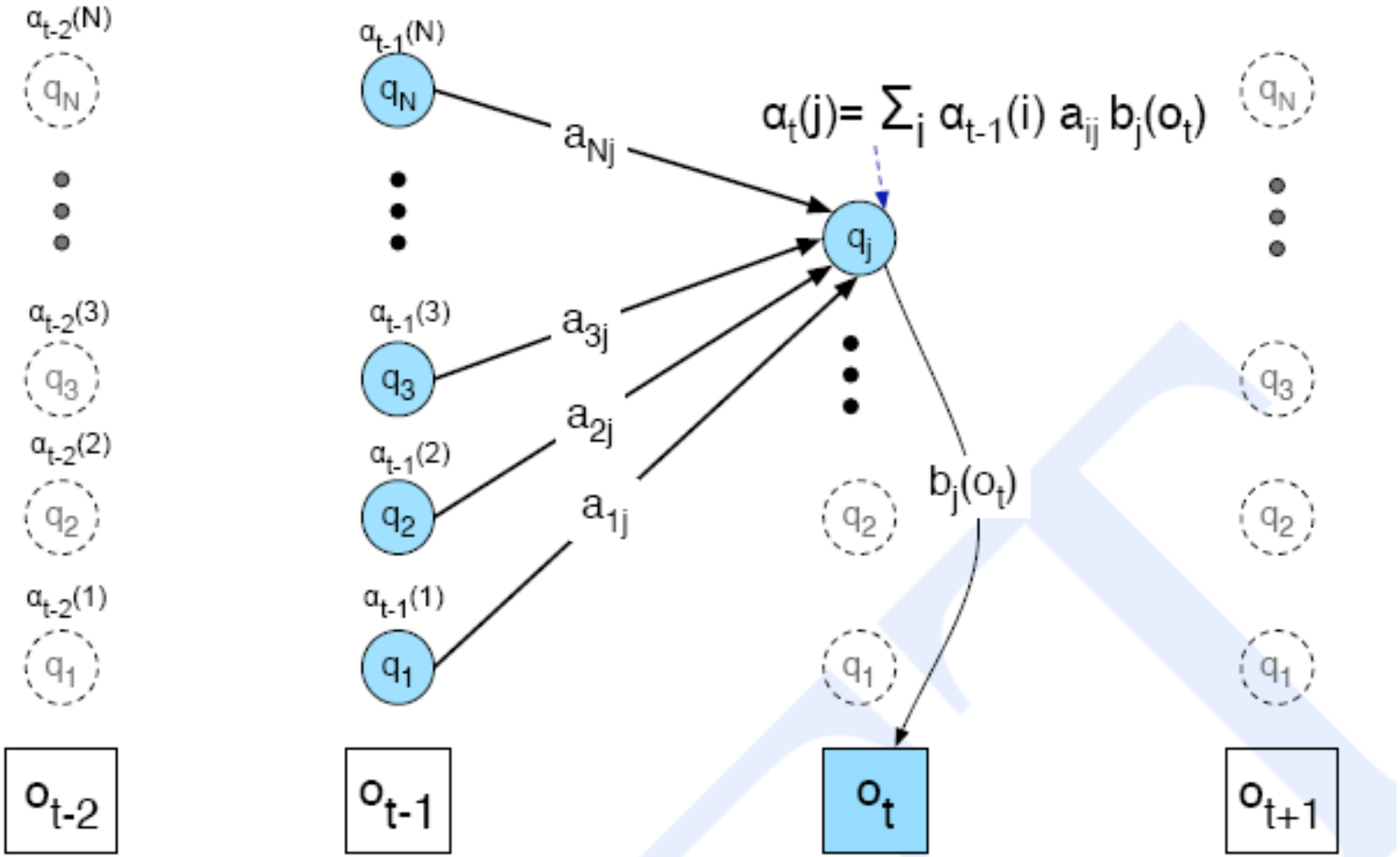
2. Recursion:

$$\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t); \quad 1 \le j \le N, 1 < t \le T$$

3. Termination:

$$P(O|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$$

THE FORWARD ALGORITHM



THE FORWARD ALGORITHM

function FORWARD(*observations* of len T, *state-graph* of len N) **returns** *forward-prob*

create a probability matrix *forward*$[N+2,T]$
for each state s **from** 1 **to** N **do** ;initialization step
$\quad forward[s,1] \leftarrow a_{0,s} * b_s(o_1)$
for each time step t **from** 2 **to** T **do** ;recursion step
$\quad$ **for** each state s **from** 1 **to** N **do**

$$forward[s,t] \leftarrow \sum_{s'=1}^{N} forward[s',t-1] * a_{s',s} * b_s(o_t)$$

$$forward[q_F,\mathrm{T}] \leftarrow \sum_{s=1}^{N} forward[s,T] * a_{s,q_F} \quad \text{; termination step}$$

return $forward[q_F,T]$

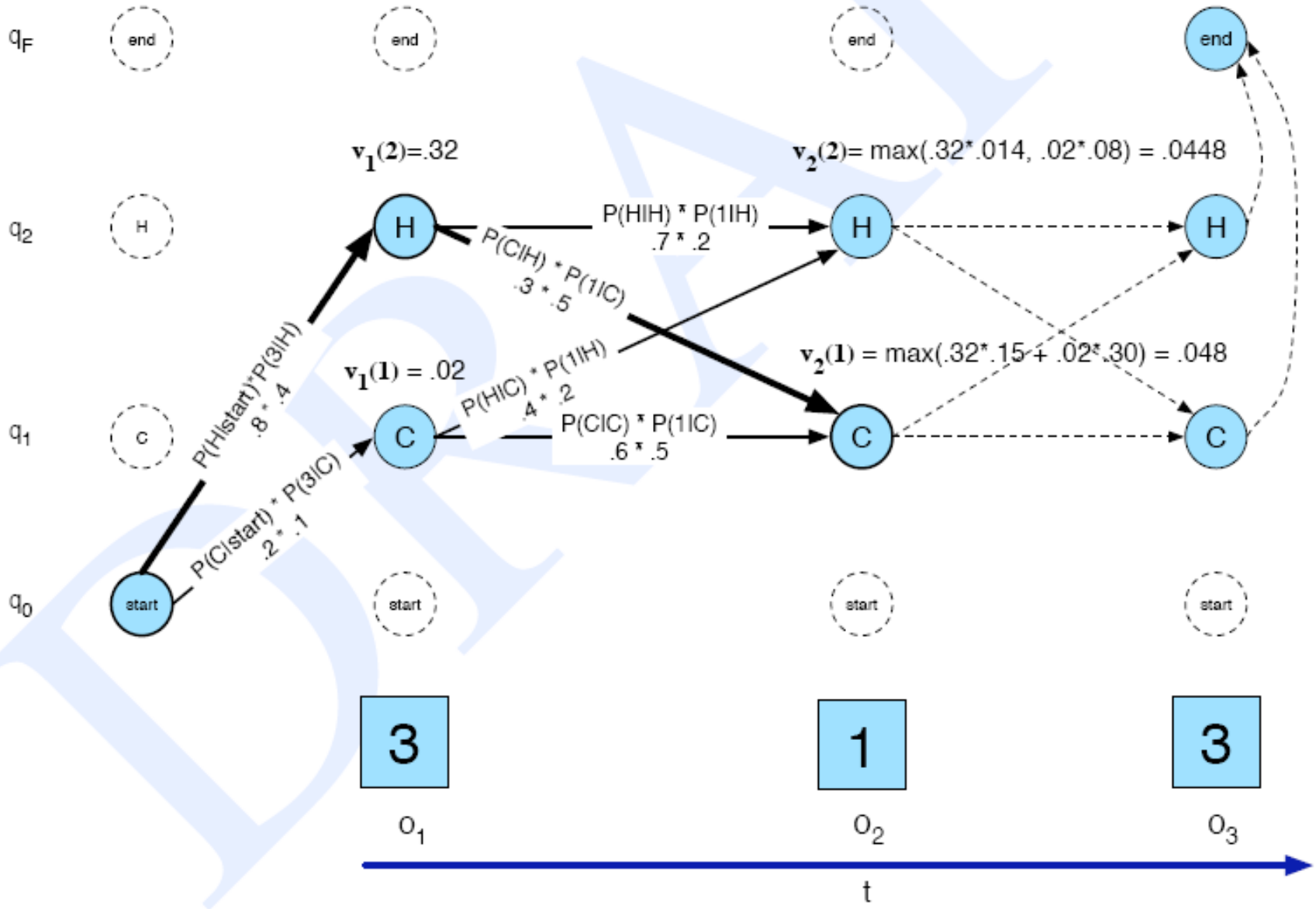
Figure 6.9 The forward algorithm. We've used the notation $forward[s,t]$ to represent $\alpha_t(s)$.

DECODING: THE VITERBI

- In the ice-cream domain, given a sequence of ice-cream observations 3 1 3 and an HMM, the task of the decoder is to find the best hidden weather sequence (like: H H H).
- The most common decoding algorithms for HMMs is the Viterbi algorithm.

*

DECODING: THE VITERBI



DECODING: THE VITERBI ALGORITHM

- $v_t(j)$ represents the probability that the HMM is in state j after seeing the first t observations and passing through the most probable state sequence $q_0,q_1,\ldots,q_{t-1}$, given the automaton λ

$$v_t(j) = \max_{q_0,q_1,\cdots,q_{t-1}} P(q_0,q_1\ldots q_{t-1},o_1,o_2\ldots o_t,q_t=j|\lambda)$$

$$v_t(j) = \max_{i=1}^{N} v_{t-1}(i)\, a_{ij}\, b_j(o_t)$$

*

Figure A.9 shows pseudocode for the Viterbi algorithm. Note that the Viterbi algorithm is identical to the forward algorithm except that it takes the **max** over the previous path probabilities whereas the forward algorithm takes the **sum**. Note also that the Viterbi algorithm has one component that the forward algorithm doesn't

have: **backpointers**. The reason is that while the forward algorithm needs to produce an observation likelihood, the Viterbi algorithm must produce a probability and also the most likely state sequence. We compute this best state sequence by keeping track of the path of hidden states that led to each state, as suggested in Fig. A.10, and then at the end backtracing the best path to the beginning (the Viterbi **backtrace**).

erbi
race

Finally, we can give a formal definition of the Viterbi recursion as follows:

1. **Initialization:**

$$\begin{aligned} v_1(j) &= \pi_j b_j(o_1) \qquad 1 \le j \le N \\ bt_1(j) &= 0 \qquad 1 \le j \le N \end{aligned}$$

2. **Recursion**

$$\begin{aligned} v_t(j) &= \max_{i=1}^{N} v_{t-1}(i)\, a_{ij}\, b_j(o_t); \quad 1 \le j \le N, 1 < t \le T \\ bt_t(j) &= \operatorname*{argmax}_{i=1}^{N} v_{t-1}(i)\, a_{ij}\, b_j(o_t); \quad 1 \le j \le N, 1 < t \le T \end{aligned}$$

3. **Termination:**

$$\begin{aligned} \text{The best score:} \quad P* &= \max_{i=1}^{N} v_T(i) \\ \text{The start of backtrace:} \quad q_T* &= \operatorname*{argmax}_{i=1}^{N} v_T(i) \end{aligned}$$

TRAINING HMMS: THE FORWARD-BACKWARD ALGORITHM

- Given an observation sequence *O* and the set of possible states in the HMM, learn the HMM parameters *A* and *B*
- Ice-Cream task: we would start with a sequence of observations *O* = {1,3,2, ...,}, and the set of hidden states *H* and *C*.
- part-of-speech tagging task: we would start with a sequence of observations *O* = {*w*1,*w*2,*w*3 . . .} and a set of hidden states *NN, NNS, VBD, IN,...*

*

forward-backward

- **Forward-backward** or **Baum-Welch** algorithm (Baum, 1972), a special case of the **Expectation-Maximization (EM algorithm)**
- **Start on Markov Chain**: no emission probabilities *B* (alternatively we could view a Markov chain as a degenerate Hidden Markov Model where all the *b* probabilities are 1.0 for the observed symbol and 0 for all other symbols)
  - Only need to train transition probability A

$$a_{ij} = \frac{C(i \rightarrow j)}{\sum_{q \in Q} C(i \rightarrow q)}$$

*

forward-backward

- **For Hidden Markov Model**: we can not count this directly from an observation sequence since we don't know which path of sates was taken through the machine for a given input.
- **Baum-Welch algorithm** uses two intuitions:
  - The first idea is to *iteratively* estimate the counts
  - computing the forward probability for an observation and then dividing that probability mass among all the different paths that contributed to this forward probability

*

backward probability.

$$\beta_t(i) = P(o_{t+1}, o_{t+2} \ldots o_T | q_t = i, \lambda)$$

1. **Initialization:**

$$\beta_T(i) = a_{i,F}, \quad 1 \leq i \leq N$$

2. **Recursion** (again since states 0 and q_F are non-emitting):

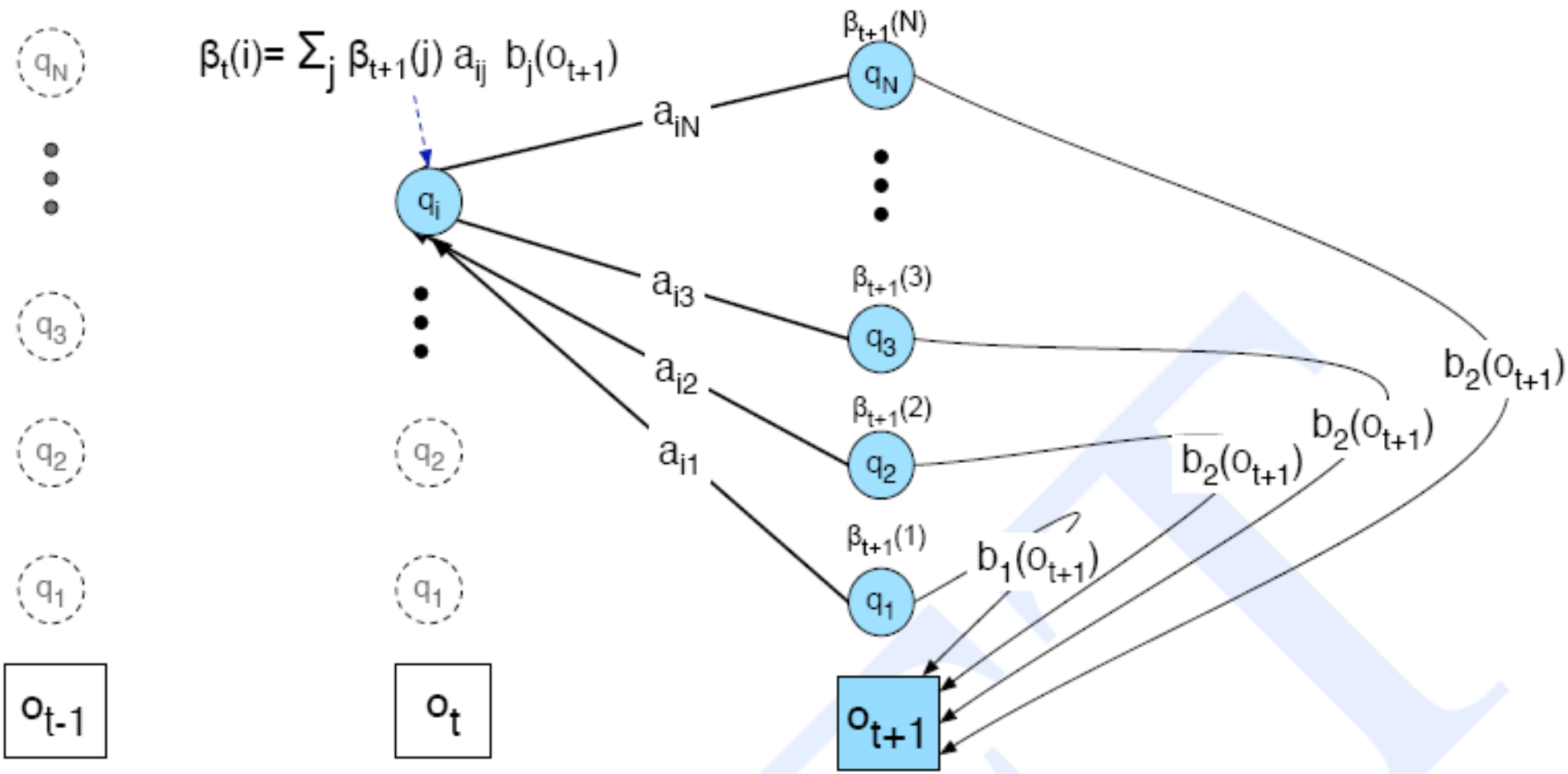
$$\beta_t(i) = \sum_{j=1}^{N} a_{ij}\, b_j(o_{t+1})\, \beta_{t+1}(j), \quad 1 \leq i \leq N, 1 \leq t < T$$

3. **Termination:**

$$P(O|\lambda) = \alpha_T(q_F) = \beta_1(0) = \sum_{j=1}^{N} a_{0j}\, b_j(o_1)\, \beta_1(j)$$

*

backward probability.



*

backward probability.

$$\hat{a}_{ij} = \frac{\text{expected number of transitions from state } i \text{ to state } j}{\text{expected number of transitions from state } i}$$

*

forward-backward

More formally, let's define the probability ξ_t as the probability of being in state i at time t and state j at time $t+1$, given the observation sequence and of course the model:

$$\xi_t(i,j) = P(q_t = i, q_{t+1} = j | O, \lambda)$$

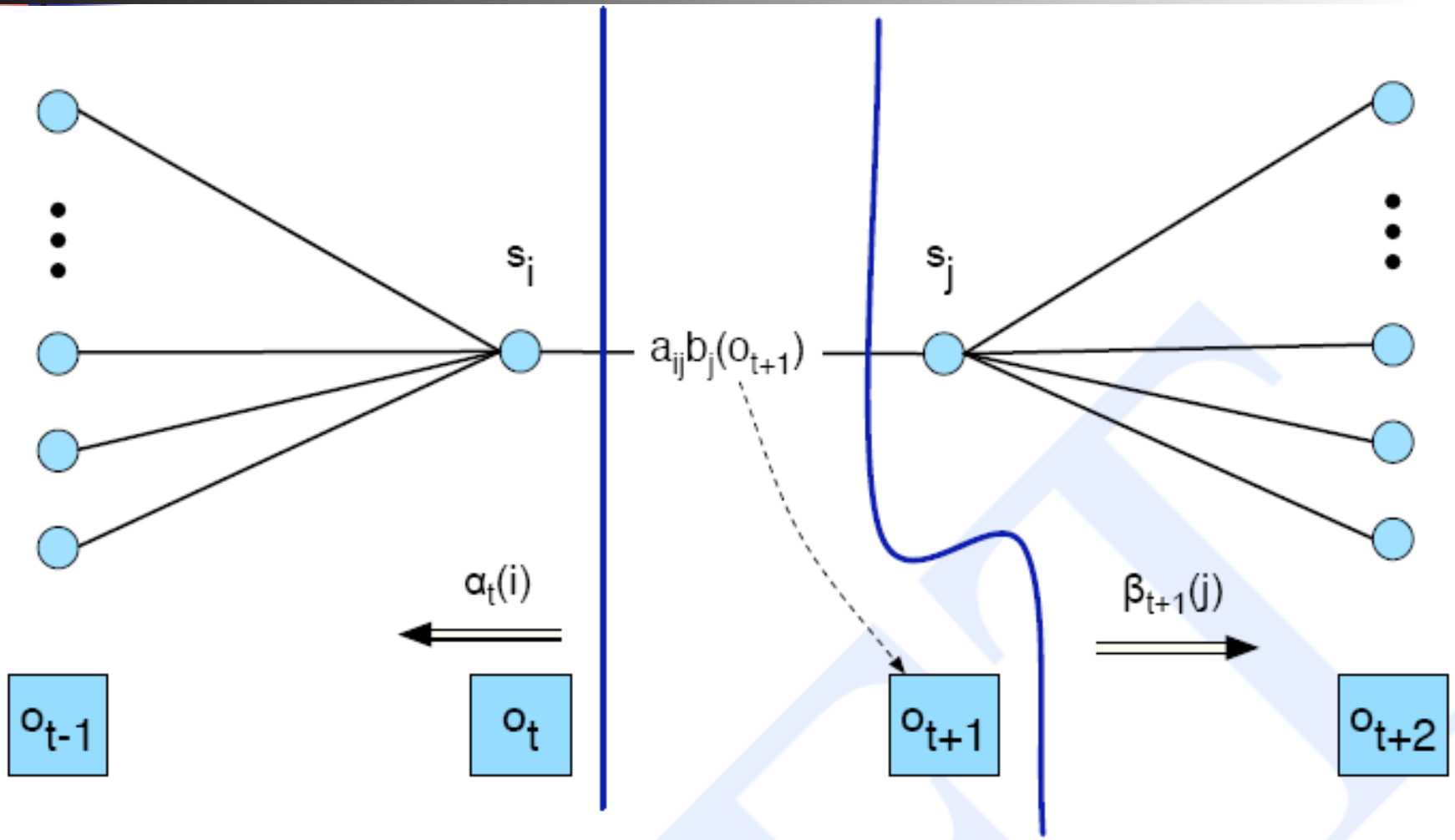
In order to compute ξ_t, we first compute a probability which is similar to ξ_t, but differs in including the probability of the observation; note the different conditioning of O from Equation (6.33):

$$\text{not-quite-}\xi_t(i,j) = P(q_t = i, q_{t+1} = j, O | \lambda)$$

$$\text{not-quite-}\xi_t(i,j) = \alpha_t(i)\, a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)$$

*

forward-backward



forward-backward

$$P(X|Y,Z) = \frac{P(X,Y|Z)}{P(Y|Z)}$$

$$P(O|\lambda) = \alpha_T(N) = \beta_T(1) = \sum_{j=1}^{N} \alpha_t(j)\beta_t(j)$$

$$\xi_t(i,j) = \frac{\alpha_t(i)\, a_{ij} b_j(o_{t+1})\beta_{t+1}(j)}{\alpha_T(N)}$$

$$\hat{a}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \sum_{j=1}^{N} \xi_t(i,j)}$$

*

forward-backward

$$\hat{b}_j(v_k) = \frac{\text{expected number of times in state } j \text{ and observing symbol } v_k}{\text{expected number of times in state } j}$$

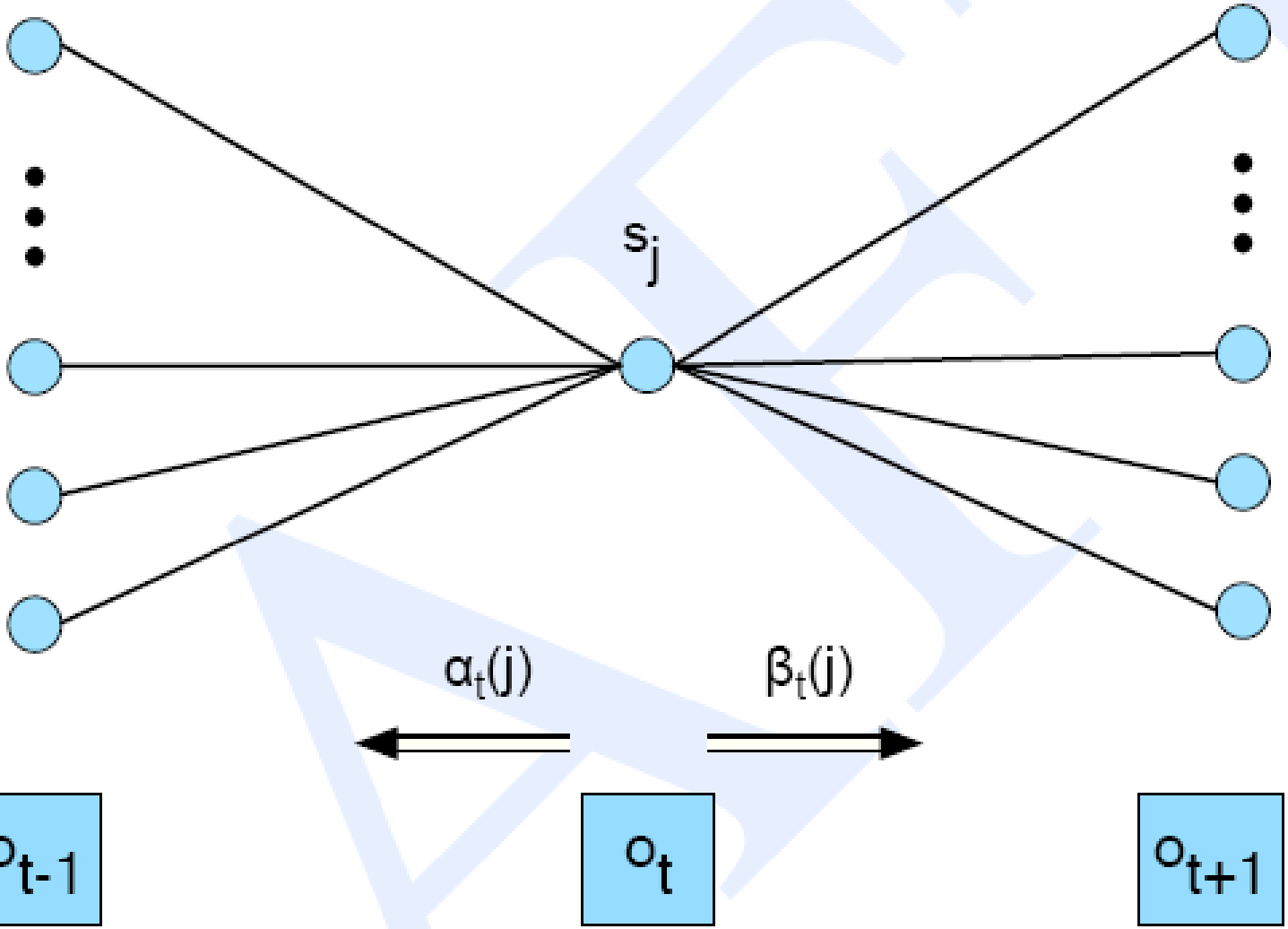
- The probability of being in state j at time t, which we will call $\gamma_t(j)$

$$\gamma_t(j) = P(q_t = j|O, \lambda)$$

$$\gamma_t(j) = \frac{P(q_t = j, O|\lambda)}{P(O|\lambda)}$$

*

forward-backward



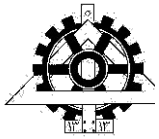
*

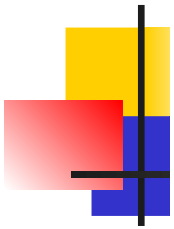
forward-backward

$$\gamma_t(j) = \frac{\alpha_t(j)\beta_t(j)}{P(O|\lambda)}$$

$$\hat{b}_j(v_k) = \frac{\sum_{t=1 s.t. O_t=v_k}^{T} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)}$$

*





function FORWARD-BACKWARD(*observations* of len T, *output vocabulary* V, *hidden state set* Q) **returns** *HMM=(A,B)*

initialize A and B

iterate until convergence

E-step

$$\gamma_t(j) = \frac{\alpha_t(j)\beta_t(j)}{P(O|\lambda)} \quad \forall\, t \text{ and } j$$

$$\xi_t(i,j) = \frac{\alpha_t(i)\, a_{ij} b_j(o_{t+1})\beta_{t+1}(j)}{\alpha_T(N)} \quad \forall\, t,\ i, \text{ and } j$$

M-step

$$\hat{a}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \sum_{j=1}^{N} \xi_t(i,j)}$$

$$\hat{b}_j(v_k) = \frac{\sum_{t=1 s.t.\ O_t=v_k}^{T} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)}$$

return A, B

MAXIMUM ENTROPY MODELS

- Machine learning framework called **Maximum Entropy** modeling, **MAXEnt**
- Used for **Classification**
  - The task of classification is to take a single observation, extract some useful features describing the observation, and then based on these features, to **classify** the observation into one of a set of discrete classes.
- **Probabilistic** classifier: gives the **probability** of the observation being in that class
- Non-sequential classification
  - in **text classification** we might need to decide whether a particular email should be classified as spam or not
  - In **sentiment analysis** we have to determine whether a particular sentence or document expresses a positive or negative **opinion**.
  - we'll need to classify a period character ('.') as either a sentence boundary or not

*

MaxEnt

- MaxEnt belongs to the family of classifiers known as the **exponential** or **log-linear** classifiers

- **MaxEnt** works by extracting some set of features from the input, combining them **linearly** (meaning that we multiply each by a weight and then add them up), and then using this sum as an exponent

*

MaxEnt

- Example: tagging
  - A feature for tagging might be *this word ends in -ing* or *the previous word was 'the'.*
  - For each such feature **f**i, we have some weight **w**i.
  - Given the features and weights, our goal is to choose the class for the word.
  - MaxEnt does this by choosing the most probable tag. The probability of a particular class c given the observation x is

$$p(c|x) = \frac{1}{Z}\exp(\sum_i w_i f_i)$$

  - Here, Z is a normalizing factor, used to make the probabilities correctly sum to 1 and as usual exp(x) = e^x

*

Linear Regression

- Two different names for tasks that map some input features into some output value: **regression** when the output is real-valued, and **classification** when the output is one of a discrete set of classes

*

Linear Regression, Example

Number of vague adjectives	Amount house sold over asking price
4	0
3	$1000
2	$1500
2	$6000
1	$14000
0	$18000

Figure 6.17 Some made-up data on the number of vague adjectives (*fantastic*, *cute*, *charming*) in a real estate ad, and the amount the house sold for over the asking price.

*

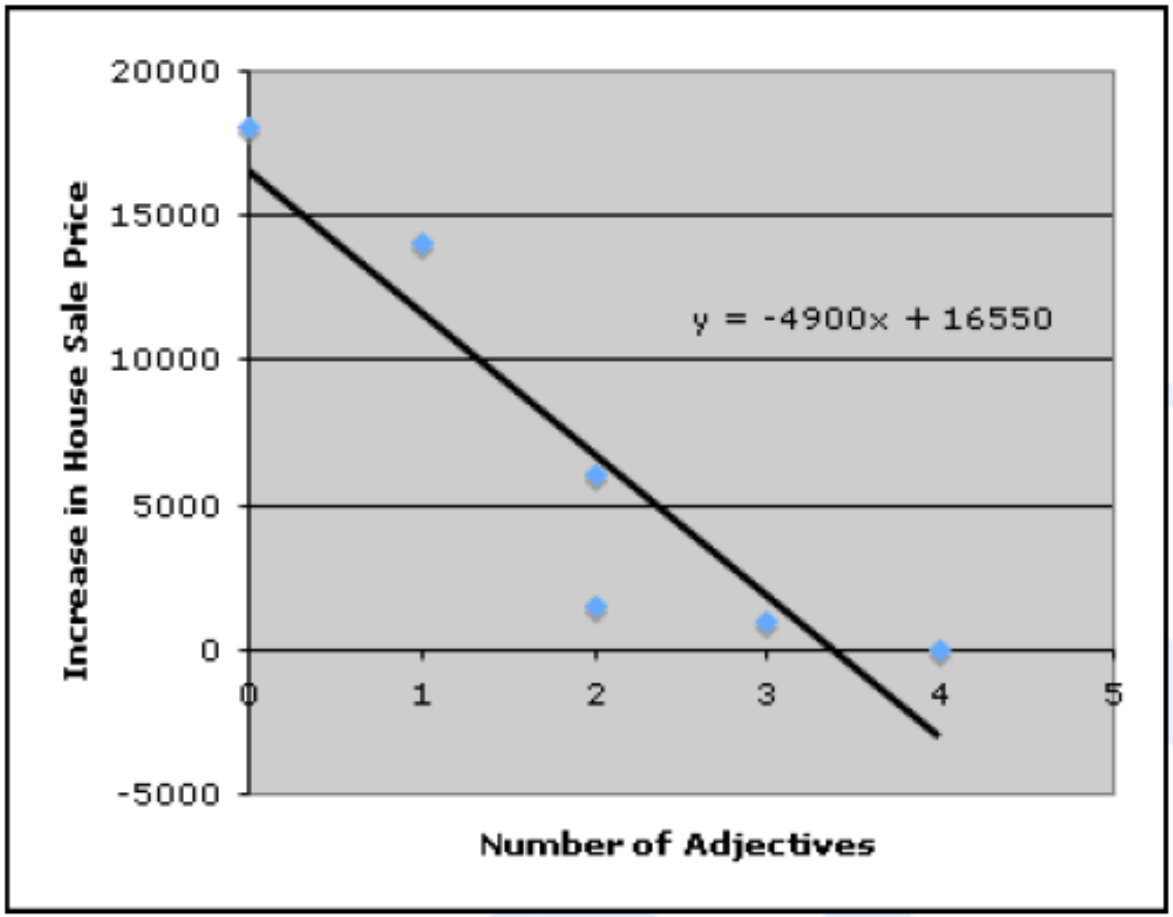


Figure 6.18 A plot of the (made-up) points in Fig. 6.17 and the regression line that best fits them, with the equation $y = -4900x + 16550$.

Multiple linear regression

- Price = *w*0 +
  *w*1 * Num Adjectives +
  *w*2 * Mortgage Rate +
  *w*3 * Num of Unsold Houses in market

$$\text{price} = w_0 + \sum_{i=1}^{N} w_i \times f_i$$

linear regression: $$y = \sum_{i=0}^{N} w_i \times f_i$$

*

Logistic regression

- **Classification** in which the output *y* we are trying to predict takes on one from a small set of discrete values
- **binary classification**:

$$P(y = true|x) = \sum_{i=0}^{N} w_i \times f_i = w \cdot f$$

- **Odds**

$$\frac{p(y = true)|x}{1 - p(y = true|x)} = \overset{exp}{w} \cdot f$$

(***dot product*** *of weight vector and feature vector)*

- **logit function**

$$\ln\left(\frac{p(y = true|x)}{1 - p(y = true|x)}\right) = w \cdot f$$

$$\text{logit}\,(p(x)) = \ln\left(\frac{p(x)}{1 - p(x)}\right)$$

*

Logistic regression

$$\ln\left(\frac{p(y=\text{true}|x)}{1-p(y=\text{true}|x)}\right)=w\cdot f$$

$$\frac{p(y=\text{true}|x)}{1-p(y=\text{true}|x)}=e^{w\cdot f}$$

$$p(y=\text{true}|x)=(1-p(y=\text{true}|x))e^{w\cdot f}$$

$$p(y=\text{true}|x)=e^{w\cdot f}-p(y=\text{true}|x)e^{w\cdot f}$$

$$p(y=\text{true}|x)+p(y=\text{true}|x)e^{w\cdot f}=e^{w\cdot f}$$

$$p(y=\text{true}|x)(1+e^{w\cdot f})=e^{w\cdot f}$$

$$p(y=\text{true}|x)=\frac{e^{w\cdot f}}{1+e^{w\cdot f}} \qquad p(y=false|x)=\frac{1}{1+e^{w\cdot f}} \qquad *$$

Logistic regression

$$p(y = \text{true}|x) = \frac{\exp(\sum_{i=0}^{N} w_i f_i)}{1 + \exp(\sum_{i=0}^{N} w_i f_i)}$$

$$p(y = \text{false}|x) = \frac{1}{1 + \exp(\sum_{i=0}^{N} w_i f_i)}$$

$$p(y = \text{true}|x) = \frac{e^{w \cdot f}}{1 + e^{w \cdot f}}$$

$$= \frac{1}{1 + e^{-w \cdot f}}$$

*

Logistic regression: Classification

Our observation should be labeled ‘true’ if

$$p(y = true|x) > p(y = false|x)$$

$$\frac{p(y = true|x)}{p(y = false|x)} > 1$$

$$\frac{p(y = true|x)}{1 - p(y = true|x)} > 1$$

$$e^{w \cdot f} > 1$$

$$w \cdot f > 0$$

$$\sum_{i=0}^{N} w_i f_i > 0$$

*

MAXIMUM ENTROPY MODELING

- **multinomial logistic regression(MaxEnt)**
  - Most of the time, classification problems that come up in language processing involve larger numbers of classes (part-of-speech classes)
- y is a value take on C different value corresponding to classes C1.....Cn

$$p(c|x) = \frac{1}{Z}\exp\sum_i w_i f_i \qquad p(c|x) = \frac{\exp\left(\sum_{i=0}^{N} w_{ci} f_i\right)}{\sum_{c' \in C} \exp\left(\sum_{i=0}^{N} w_{c'i} f_i\right)}$$

*

Maximum Entropy Modeling

$$Z = \sum_{C} p(c|x) = \sum_{c' \in C} \exp\left(\sum_{i=0}^{N} w_{c'i} f_i\right)$$

- Indicator function: A feature that only takes on the values 0 and 1

$$p(c|x) = \frac{\exp\left(\sum_{i=0}^{N} w_{ci} f_i(c,x)\right)}{\sum_{c' \in C} \exp\left(\sum_{i=0}^{N} w_{c'i} f_i(c',x)\right)}$$

*

Maximum Entropy Modeling

- Example
  - Secretariat/NNP is/BEZ expected/VBN to/TO **race**/?? tomorrow/

$$f_1(c,x) = \begin{cases} 1 & \text{if } word_i = \text{“race”} \ \& \ c = \text{NN} \\ 0 & \text{otherwise} \end{cases}$$

$$f_2(c,x) = \begin{cases} 1 & \text{if } t_{i-1} = \text{TO} \ \& \ c = \text{VB} \\ 0 & \text{otherwise} \end{cases}$$

$$f_3(c,x) = \begin{cases} 1 & \text{if } \text{suffix}(word_i) = \text{“ing”} \ \& \ c = \text{VBG} \\ 0 & \text{otherwise} \end{cases}$$

$$f_4(c,x) = \begin{cases} 1 & \text{if } \text{is_lower_case}(word_i) \ \& \ c = \text{VB} \\ 0 & \text{otherwise} \end{cases}$$

$$f_5(c,x) = \begin{cases} 1 & \text{if } word_i = \text{”race”} \ \& \ c = \text{VB} \\ 0 & \text{otherwise} \end{cases}$$

$$f_6(c,x) = \begin{cases} 1 & \text{if } t_{i-1} = \text{TO} \ \& \ c = \text{NN} \\ 0 & \text{otherwise} \end{cases}$$

*

Maximum Entropy Modeling

		f1	f2	f3	f4	f5	f6
VB	f	0	1	0	1	1	0
VB	w		.8		.01	.1	
NN	f	1	0	0	0	0	1
NN	w	.8					-1.3

Figure 6.19 Some sample feature values and weights for tagging the word *race* in (6.81).

$$P(NN|x) = \frac{e^{.8}e^{-1.3}}{e^{.8}e^{-1.3} + e^{.8}e^{.01}e^{.1}} = .20$$

$$P(VB|x) = \frac{e^{.8}e^{.01}e^{.1}}{e^{.8}e^{-1.3} + e^{.8}e^{.01}e^{.1}} = .80$$

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|x)$$

*

Why do we call it Maximum Entropy?

- Suppose, we want to assign a tag to the word zzfish (a word made up for this example)
- Intuitively it would be the equiprobable distribution:

NN	JJ	NNS	VB	NNP	IN	MD	UH	SYM	VBG	POS	PRP	CC	CD	...
$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	$\frac{1}{45}$	...

- Suppose, from training data we learned one fact: the set of possible tags for zzfish is NN, JJ, NNS and VB.

$$P(NN) + P(JJ) + P(NNS) + P(VB) = 1$$

NN	JJ	NNS	VB	NNP	IN	MD	UH	SYM	VBG	POS	PRP	CC	CD	...
$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	0	0	0	0	0	0	0	0	0	0	...

Why do we call it Maximum Entropy?

- Suppose we looked at our tagged training data and noticed that 8 times out of 10, zzfish was tagged as either NN or NNS.

$$P(NN) + P(JJ) + P(NNS) + P(VB) = 1$$

$$P(\text{word is } zzfish \text{ and } t_i = \text{NN or } t_i = \text{NNS}) = \frac{8}{10}$$

NN	JJ	NNS	VB	NNP	...
$\frac{4}{10}$	$\frac{1}{10}$	$\frac{4}{10}$	$\frac{1}{10}$	0	...

*

Why do we call it Maximum Entropy?

- Suppose we have no further info. about zzfish.
- But we notice in the training data, verbs (VB) occur as 1 word in 20.
- We can add this constraint.

$$P(VB) = \frac{1}{20}$$

NN	JJ	NNS	VB
$\frac{4}{10}$	$\frac{3}{20}$	$\frac{4}{10}$	$\frac{1}{20}$

*

Maximum Entropy

"To select a model from a set C of allowed probability distributions, choose the model $p^* \in C$ with maximum entropy $H(p)$":

$$p^* = \underset{p \in C}{\operatorname{argmax}} H(p)$$

*

RNNs and LSTMs

Simple Recurrent Networks (RNNs or Elman Nets)

Modeling Time in Neural Networks

Language is inherently temporal

Yet the simple NLP classifiers we've seen (for example for sentiment analysis) mostly ignore time

- (Feedforward neural LMs (and the transformers we'll see later) use a "moving window" approach to time.)

Here we introduce a deep learning architecture with a different way of representing time

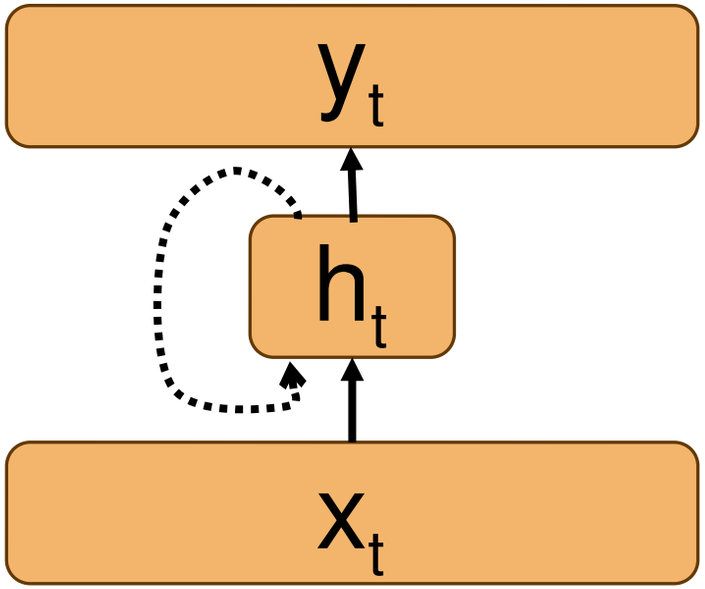
- RNNs and their variants like LSTMs

Recurrent Neural Networks (RNNs)

Any network that contains a cycle within its network connections.

The value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input.

Simple Recurrent Nets (Elman nets)



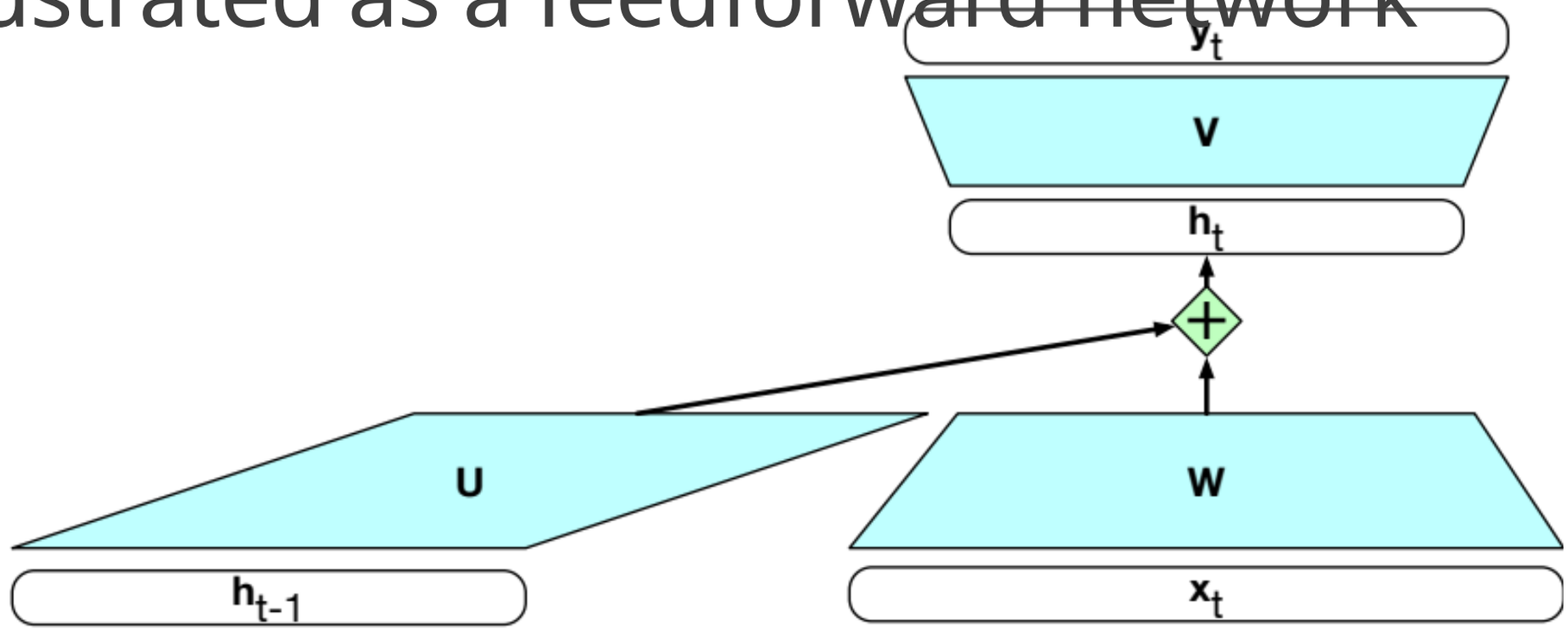
The hidden layer has a recurrence as part of its input

The activation value h_t depends on x_t but also

Forward inference in simple RNNs

Very similar to the feedforward networks we've seen!

Simple recurrent neural network illustrated as a feedforward network



$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

Inference has to be incremental

Computing h at time t requires that we first computed h at the previous time step!

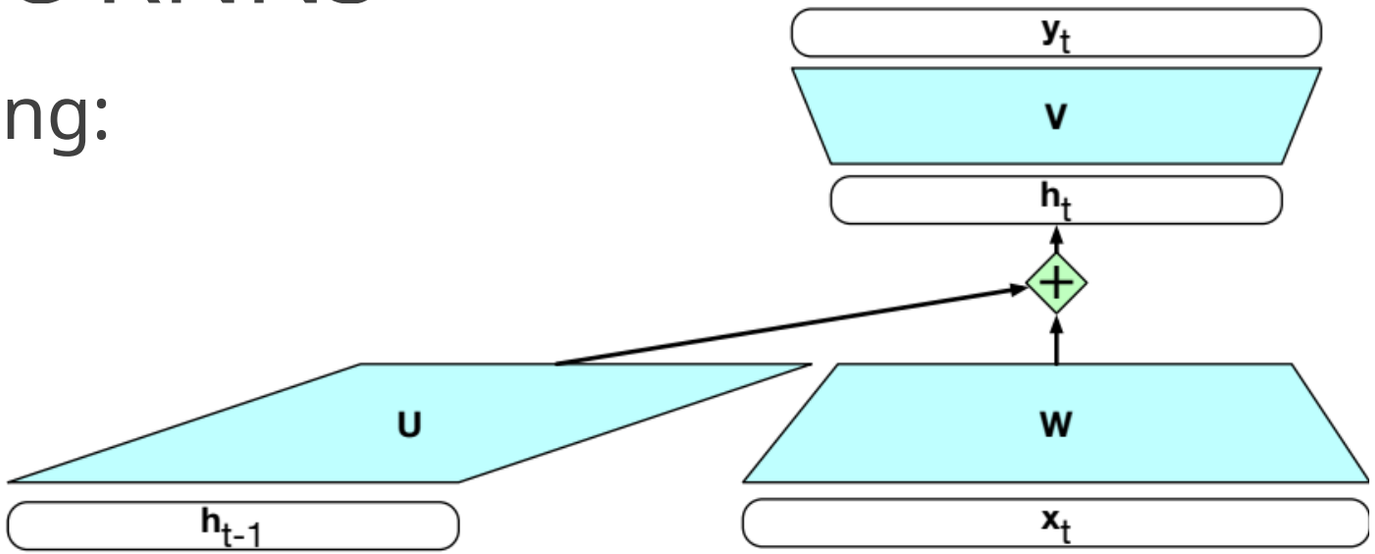
function FORWARDRNN($\mathbf{x}$, *network*) **returns** output sequence $\mathbf{y}$

$\mathbf{h}_0 \leftarrow 0$
for $i \leftarrow 1$ **to** LENGTH($\mathbf{x}$) **do**
  $\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$
  $\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$
return y

Training in simple RNNs

Just like feedforward training:

- training set,
- a loss function,
- backpropagation



Weights that need to be updated:

- **W**, the weights from the input layer to the hidden layer,
- **U**, the weights from the previous hidden layer to the current hidden layer,
- V**,** the weights from the hidden layer to the output layer.

Training in simple RNNs: unrolling in time

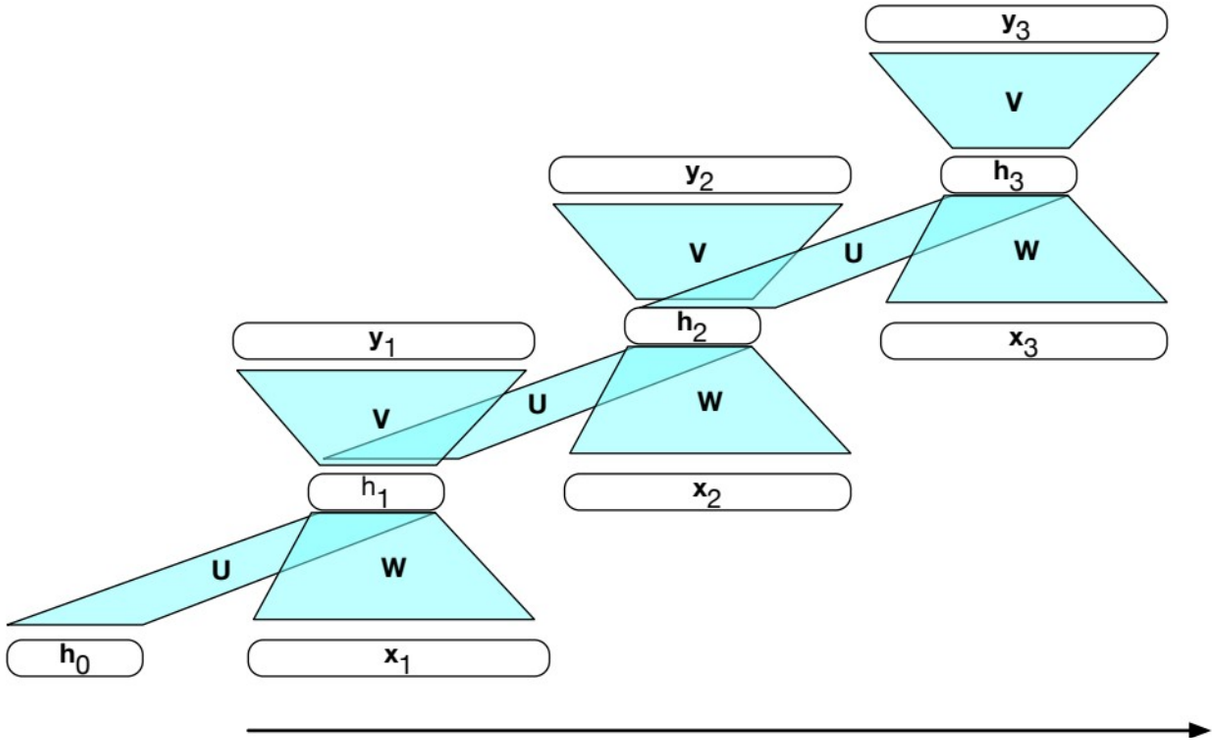
Unlike feedforward networks:

1. To compute loss function for the output at time *t* we need the hidden layer from time *t* − 1.

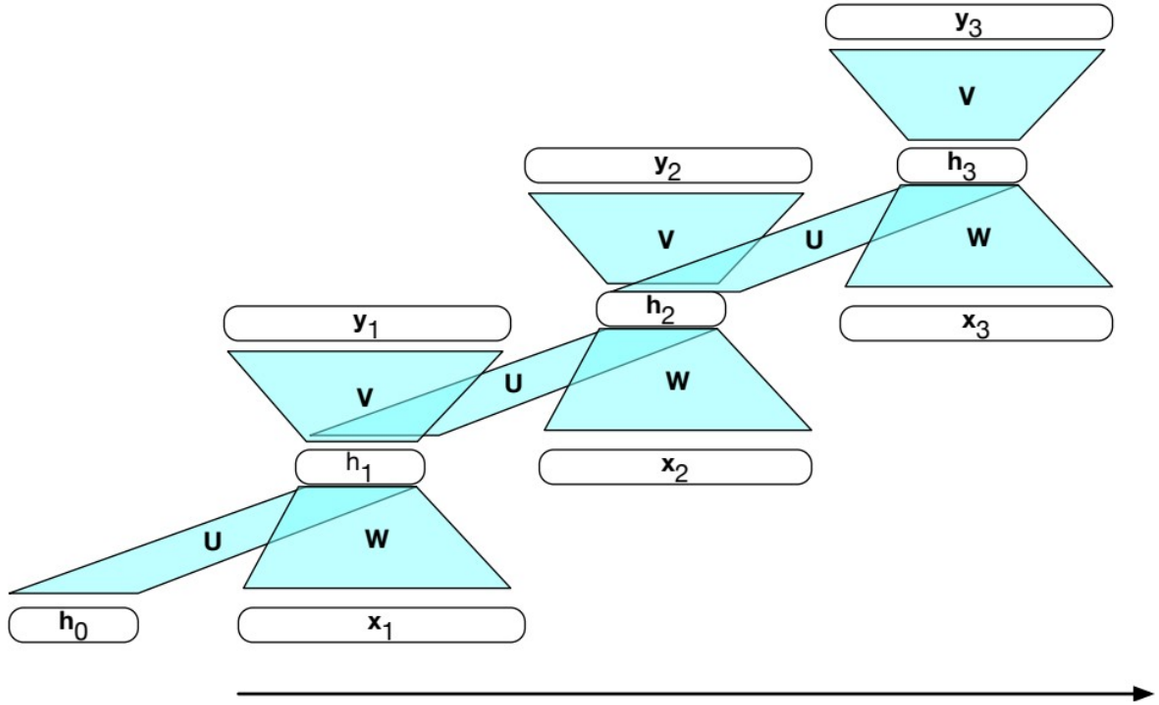
2. hidden layer at time *t* influences the output at time *t* and hidden layer at time *t+1* (and hence the output and loss at *t*+1).

So: to measure error accruing to h_t,

- need to know its influence on both the current output *as well as the ones that follow*.



Unrolling in time (2)



We unroll a recurrent network into a feedforward computational graph eliminating recurrence

1. Given an input sequence,
2. Generate an unrolled feedforward network specific to input
3. Use graph to train weights directly via ordinary backprop (or can do forward inference)

RNNs and LSTMs

Simple Recurrent Networks (RNNs or Elman Nets)

RNNs and LSTMs

RNNs as Language Models

Reminder: Language Modeling

$$P(fish|Thanks\ for\ all\ the)$$

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i|w_{<i})$$

The size of the conditioning context for different LMs

The n-gram LM:

Context size is the $n - 1$ prior words we condition on.

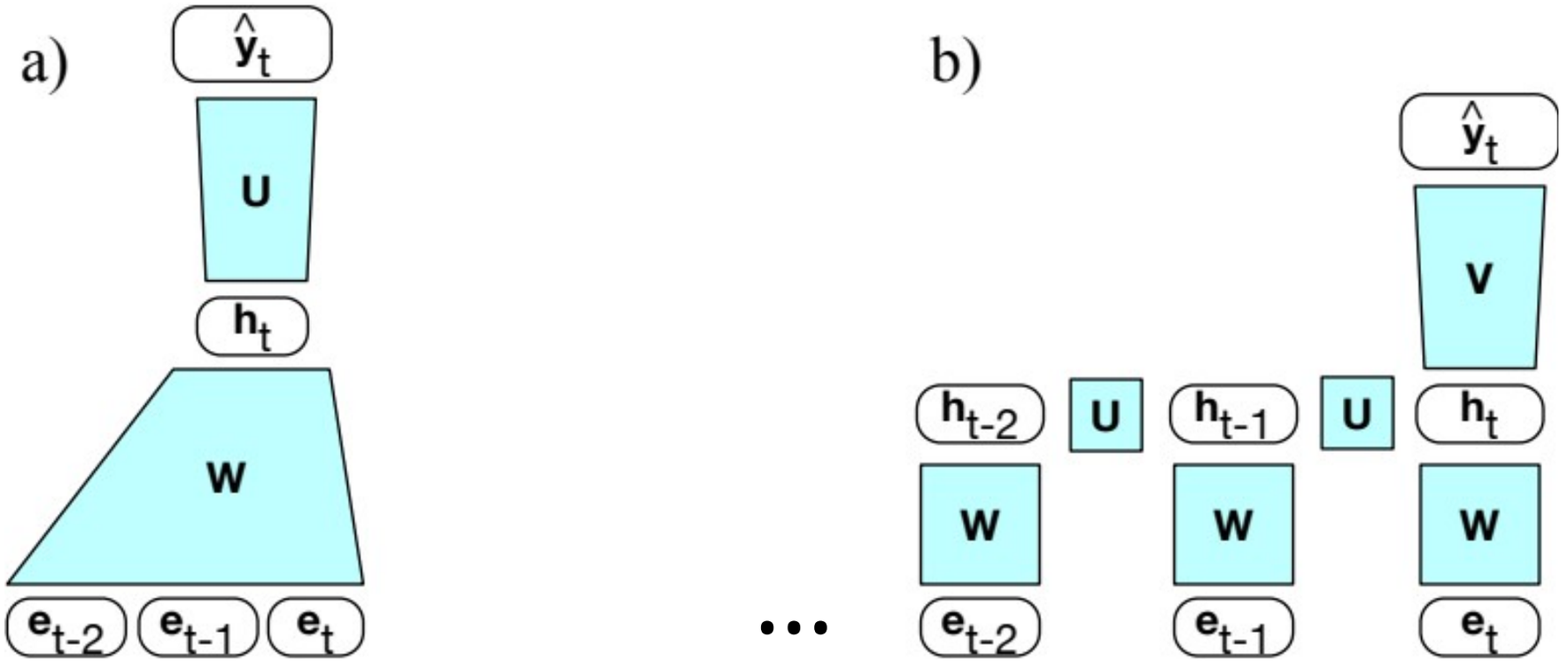
The feedforward LM:

Context is the window size.

The RNN LM:

No fixed context size; h_{t-1} represents entire history

FFN LMs vs RNN LMs



FF
N

RN
N

Forward inference in the RNN LM

Given input X of of N tokens represented as one-hot vectors

$$\mathbf{X} = [\mathbf{x}_1; \ldots; \mathbf{x}_t; \ldots; \mathbf{x}_N$$

Use embedding matrix to get the embedding for current token x_t

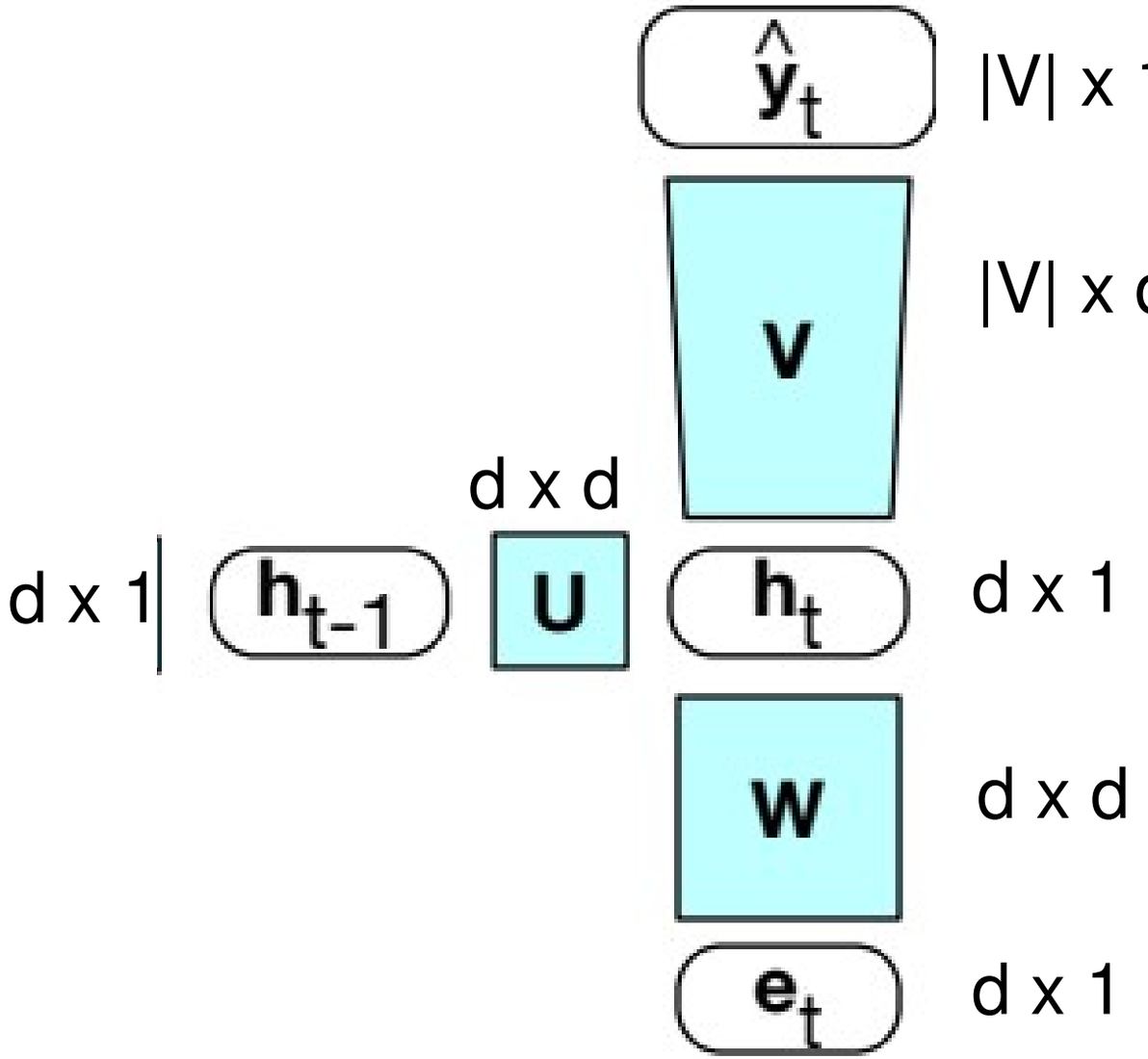
$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

Combine …

$$\hat{\mathbf{y}}_t = \mathbf{softmax}(\mathbf{V}\mathbf{h}_t)$$

Shapes



Computing the probability that the next word is word *k*

$$P(w_{t+1} = k | w_1, \ldots, w_t) = \hat{\mathbf{y}}_t[k]$$

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i | w_{1:i-1})$$

$$= \prod_{i=1}^{n} \hat{\mathbf{y}}_i[w_i]$$

Training RNN LM

- **Self-supervision**
  - take a corpus of text as training material
  - at each time step *t*
  - ask the model to predict the next word.
- **Why called self-supervised:** we don't need human labels; the text is its own supervision signal
- We train the model to
  - minimize the error
  - in predicting the true next word in the training sequence,
  - using cross-entropy as the loss function.

Cross-entropy loss

The difference between:

- a predicted probability distribution
- the correct distribution.

$$L_{CE} = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

CE loss for LMs is simpler!!!

- the correct distribution y_t *is* a one-hot vector over the vocabulary
  - where the entry for the actual next word is 1, and all the other entries are 0.
- So the CE loss for LMs is only determined by the probability of next word.
- So at time t, CE loss is:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

Teacher forcing

We always give the model the correct history to predict the next word (rather than feeding the model the possible buggy guess from the prior time step).

This is called **teacher forcing** (in training we **force** the context to be correct based on the gold words)

What teacher forcing looks like:

- At word position *t*
- the model takes as input the correct word *wt* together with *ht*−1, computes a probability distribution over possible next words
- That gives loss for the next token *wt*+1
- Then we move on to next word, ignore what the model predicted for the next word and instead use the correct word *wt*+1 along with the prior history encoded to estimate the probability of token *wt*+2.

Weight tying

The input embedding matrix E and the final layer matrix V, are similar

- The columns of E represent the word embeddings for each word in vocab. E is [d x |V|]
- The final layer matrix V helps give a score (logit) for each word in vocab . V is [|V| x d]

Instead of having separate E and V, we just tie them together, using E^T instead of V:

$$\begin{aligned} \mathbf{e}_t &= \mathbf{E}\mathbf{x}_t \\ \mathbf{h}_t &= g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \\ \hat{\mathbf{y}}_t &= \text{softmax}(\mathbf{E}^T\mathbf{h}_t) \end{aligned}$$

RNNs and LSTMs

RNNs as Language Models

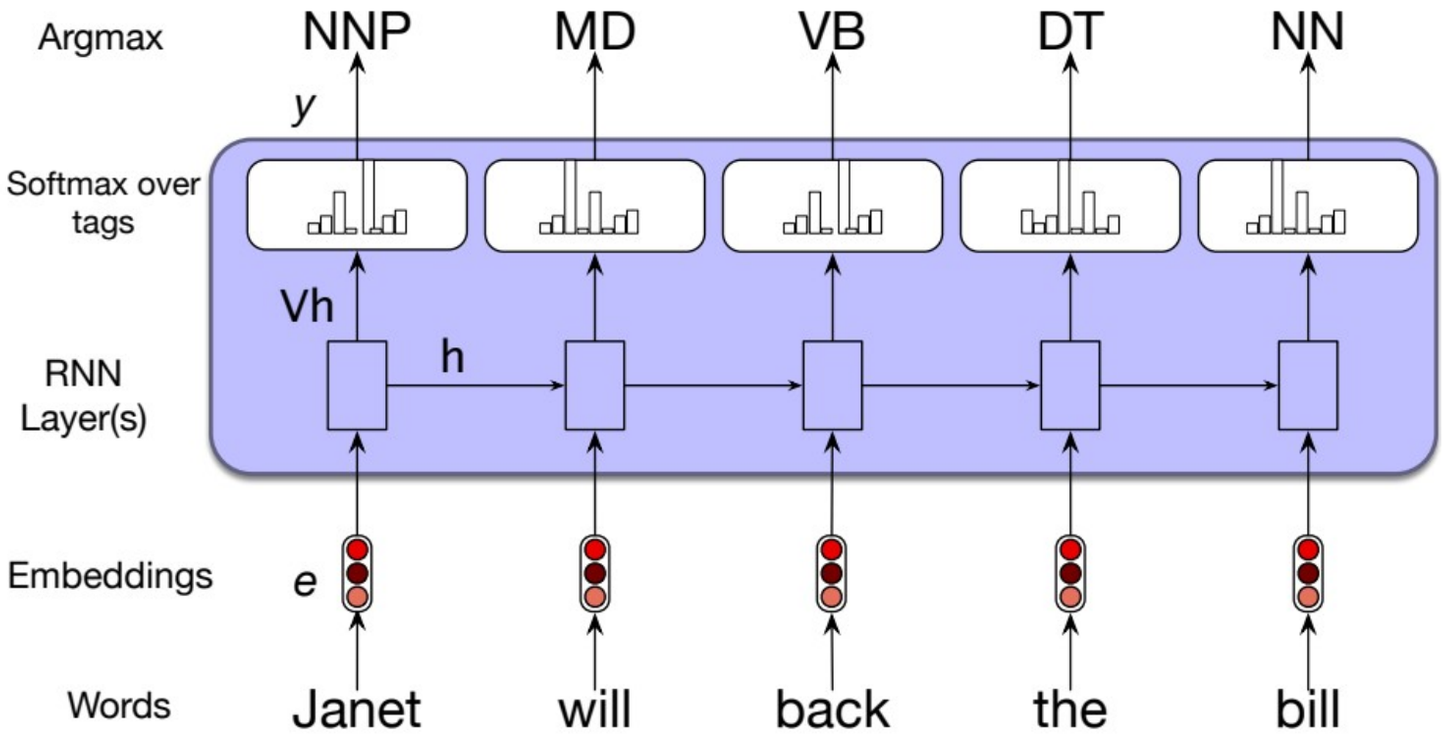
RNNs and LSTMs

RNNs for Sequences

RNNs for sequence labeling

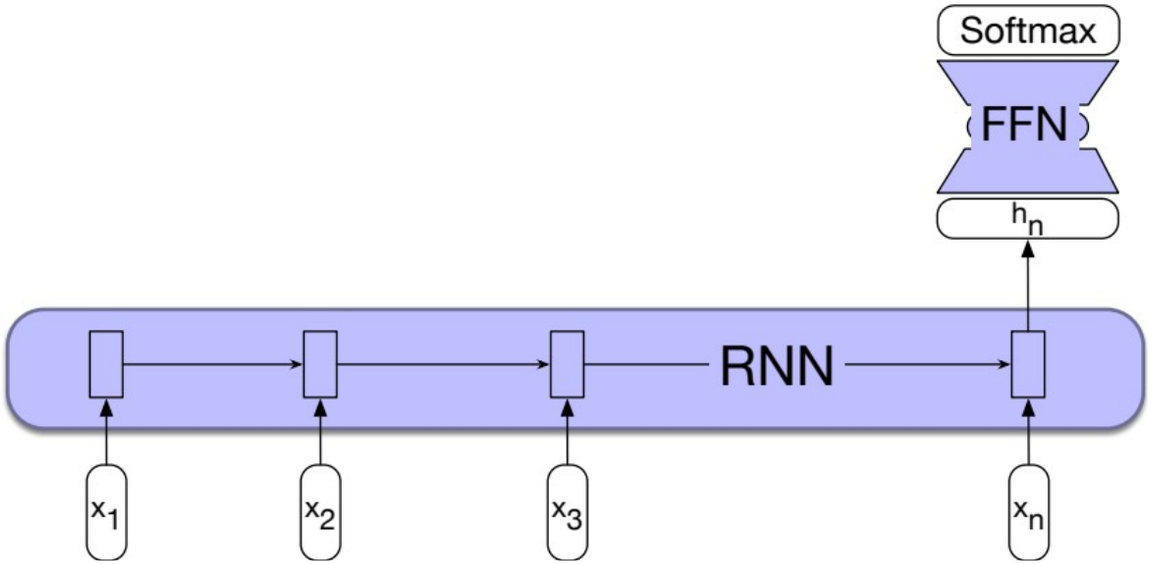
Assign a label to each element of a sequence

Part-of-speech tagging



RNNs for sequence classification

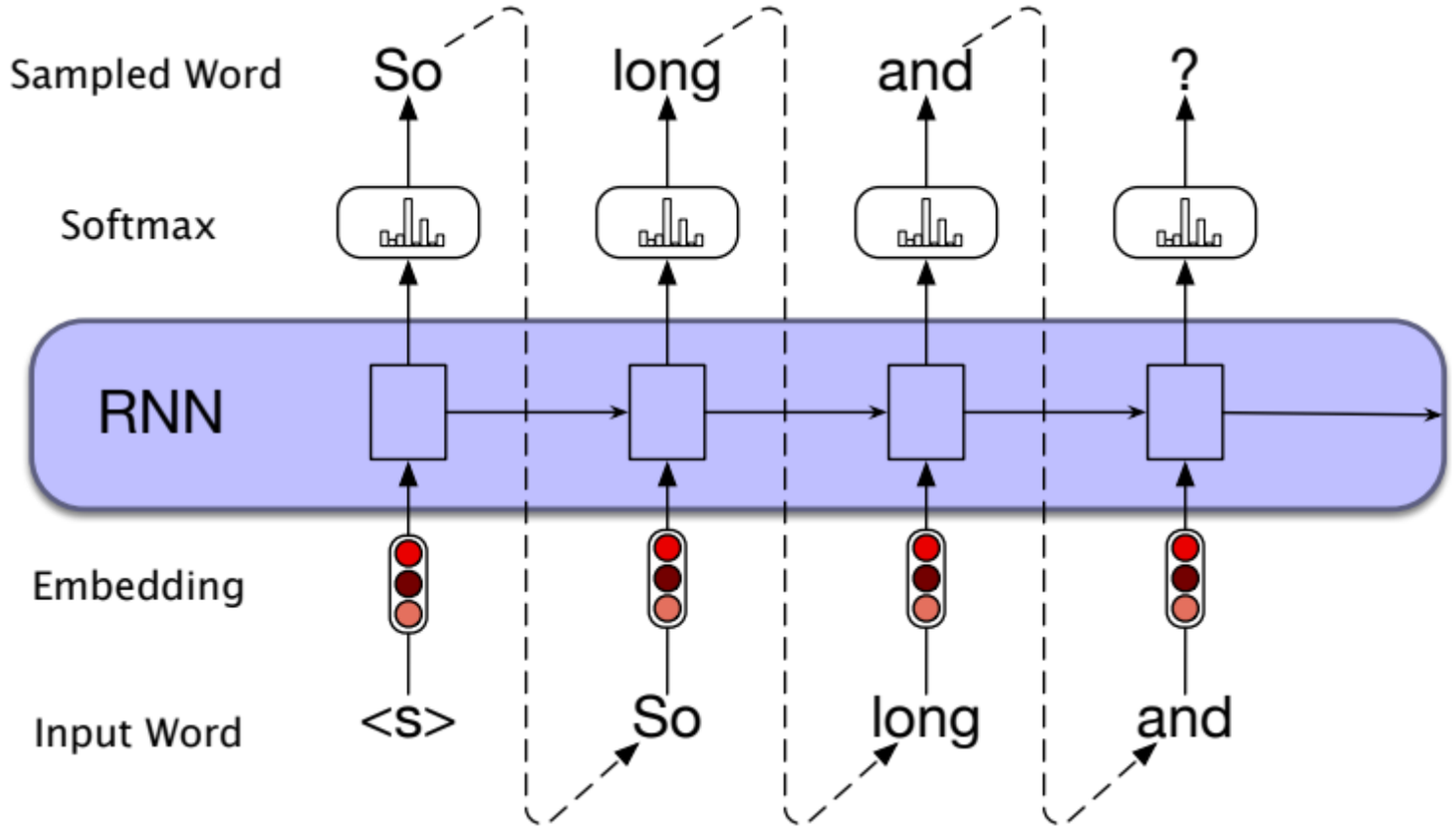
Text classification



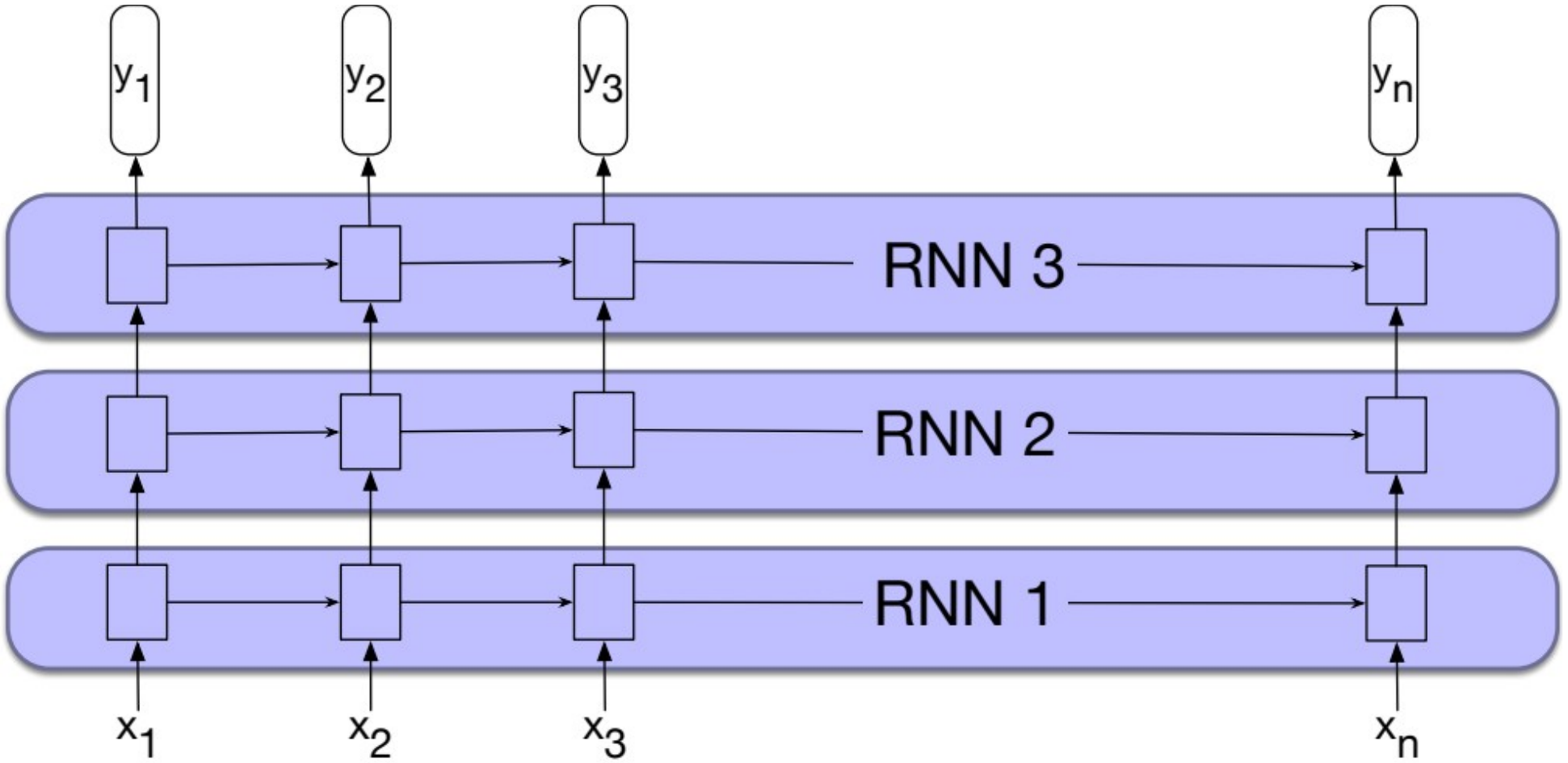
Instead of taking the last state, could use some pooling function of all the output states, like **mean pooling**

$$\mathbf{h}_{mean} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{h}_i$$

Autoregressive generation



Stacked RNNs

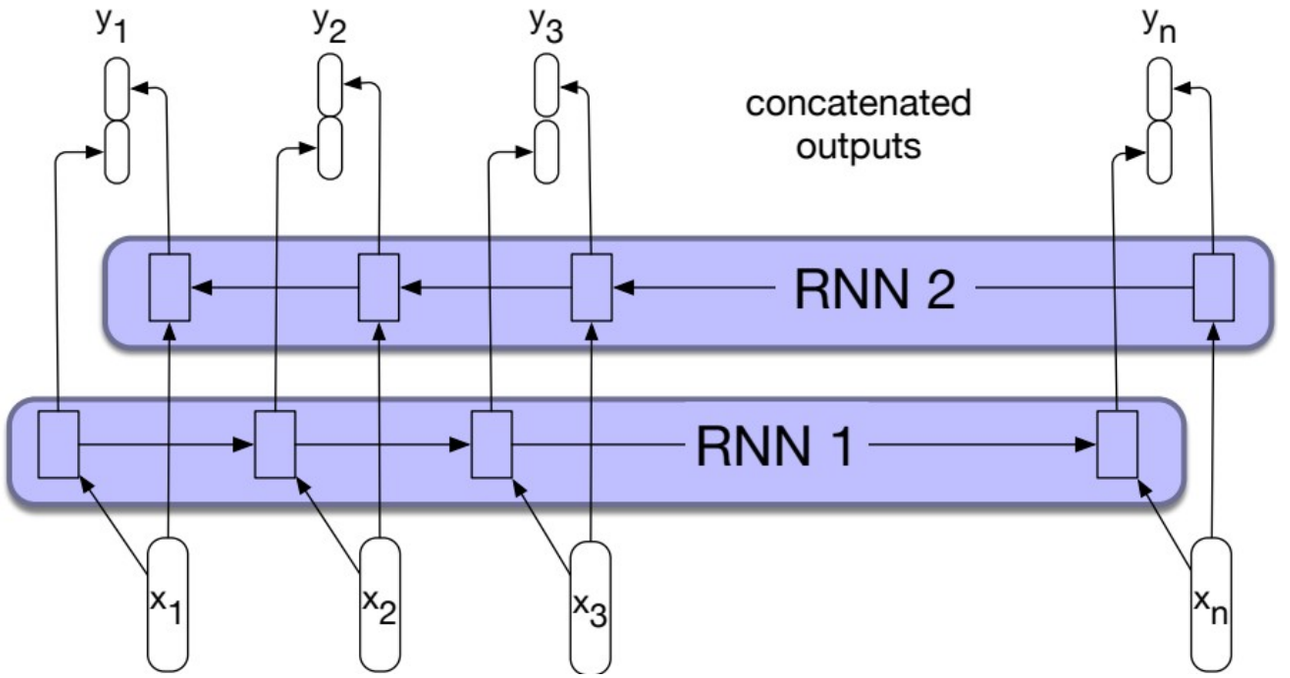


Bidirectional RNNs

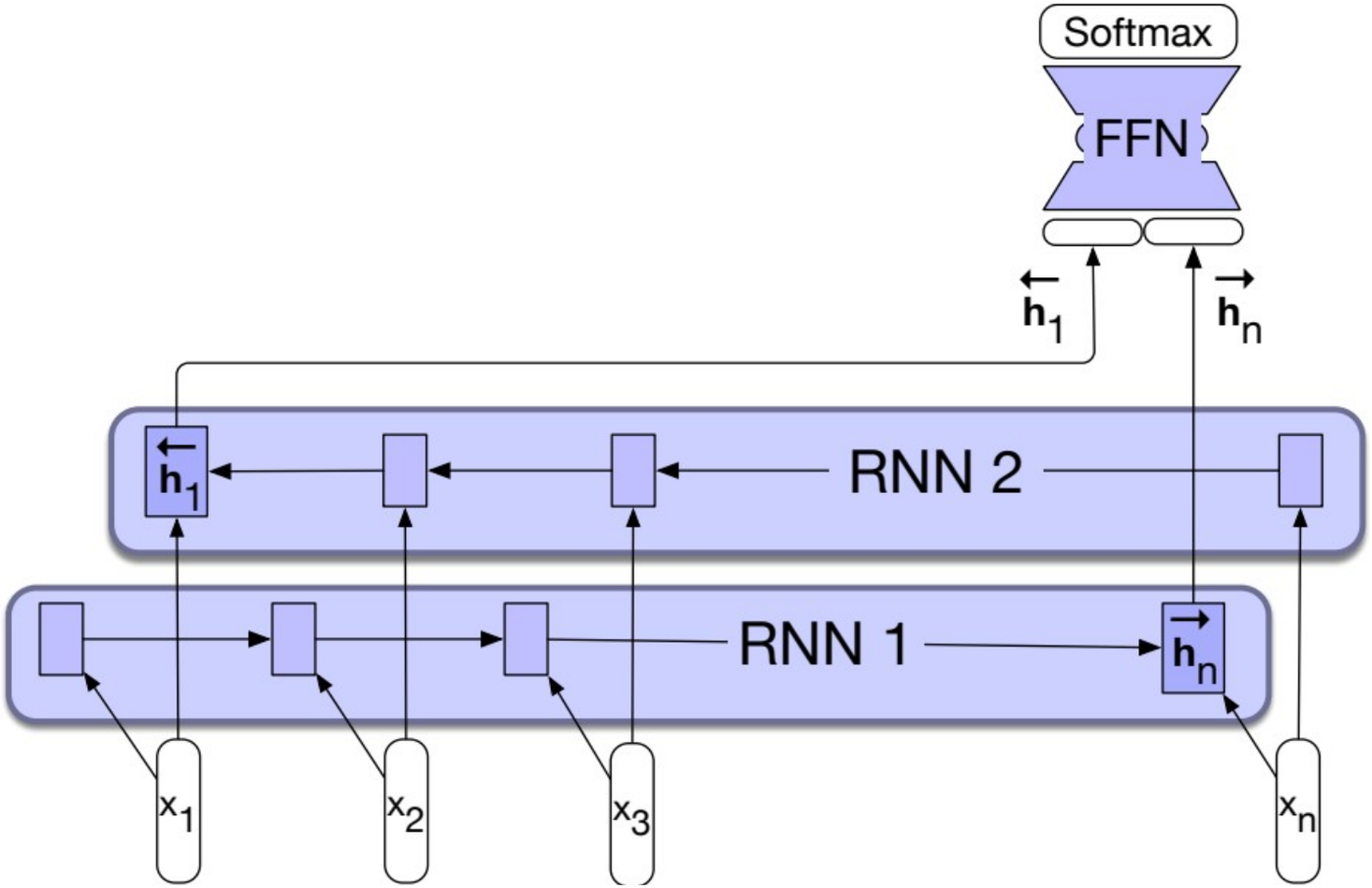
$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(\mathbf{x}_1, \ldots, \mathbf{x}_t)$$

$$\mathbf{h}_t^b = \text{RNN}_{\text{backward}}(\mathbf{x}_t, \ldots \mathbf{x}_n)$$

$$\begin{aligned}\mathbf{h}_t &= [\mathbf{h}_t^f ; \mathbf{h}_t^b] \\ &= \mathbf{h}_t^f \oplus \mathbf{h}_t^b\end{aligned}$$



Bidirectional RNNs for classification



RNNs and LSTMs

RNNs for Sequences

RNNs and LSTMs

The LSTM

Motivating the LSTM: dealing with distance

- It's hard to assign probabilities accurately when context is very far away:
  - The flights the airline was canceling were full.
- Hidden layers are being forced to do two things:
  - Provide information useful for the current decision,
  - Update and carry forward information required for future decisions.
- Another problem: During backprop, we have to repeatedly multiply gradients through time and many h's
  - The "vanishing gradient" problem

The LSTM: Long short-term memory network

LSTMs divide the context management problem into two subproblems:

- removing information no longer needed from the context,
- adding information likely to be needed for later decision making
- LSTMs add:
  - explicit context layer
  - Neural circuits with **gates** to control information flow

Forget gate

Deletes information from the context that is no longer needed.

$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t)$$
$$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t$$

Regular passing of information

$$\mathbf{g}_r = \tanh(\mathbf{U}_g \mathbf{h}_{r-1} + \mathbf{W}_g \mathbf{x}_r)$$

Add gate

Selecting information to add to current context

$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t)$$
$$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t$$

Add this to the modified context vector to get our new context vector.

$$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t$$

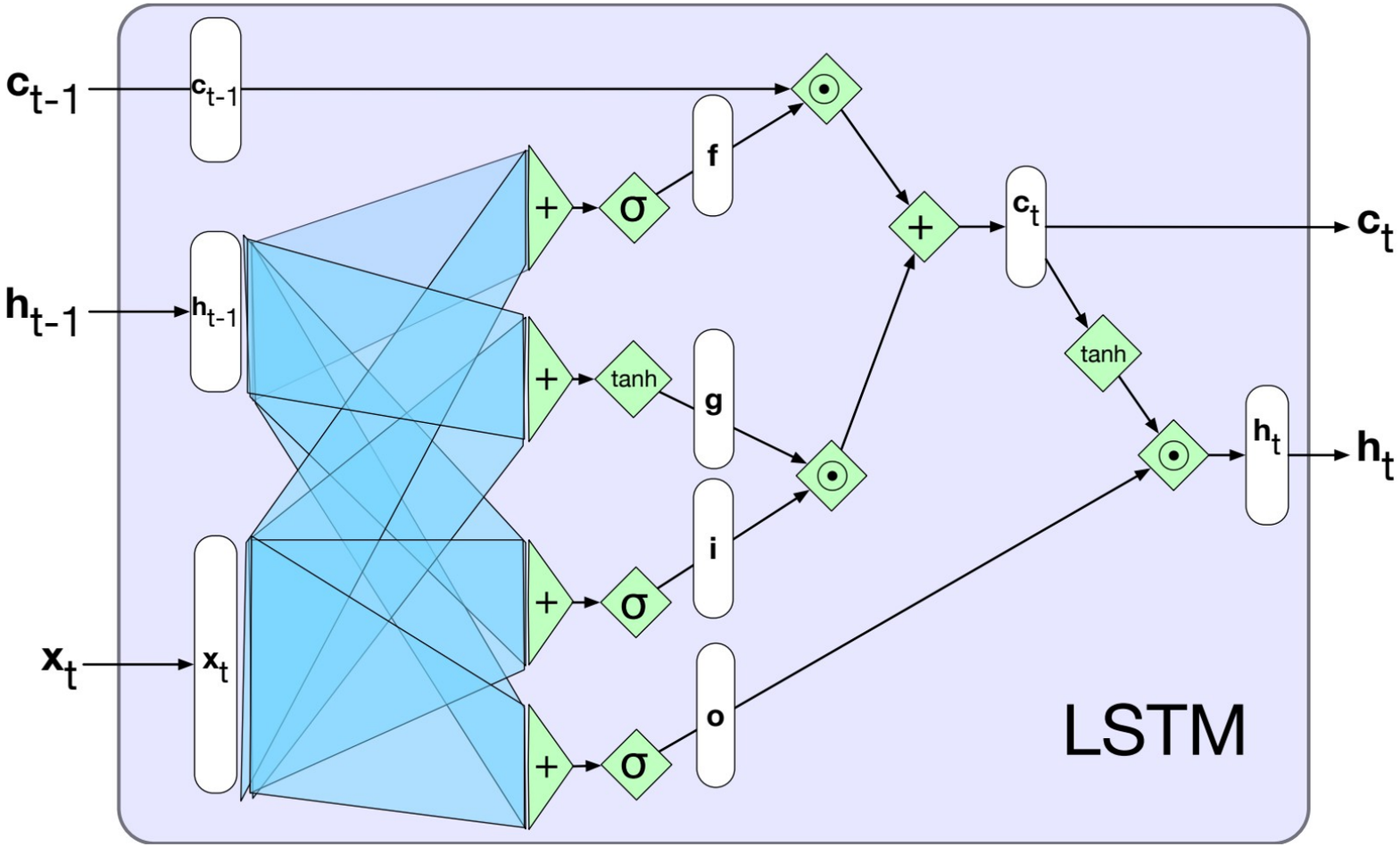
Output gate

Decide what information is required for the current hidden state (as opposed to what information needs to be preserved for future decisions).

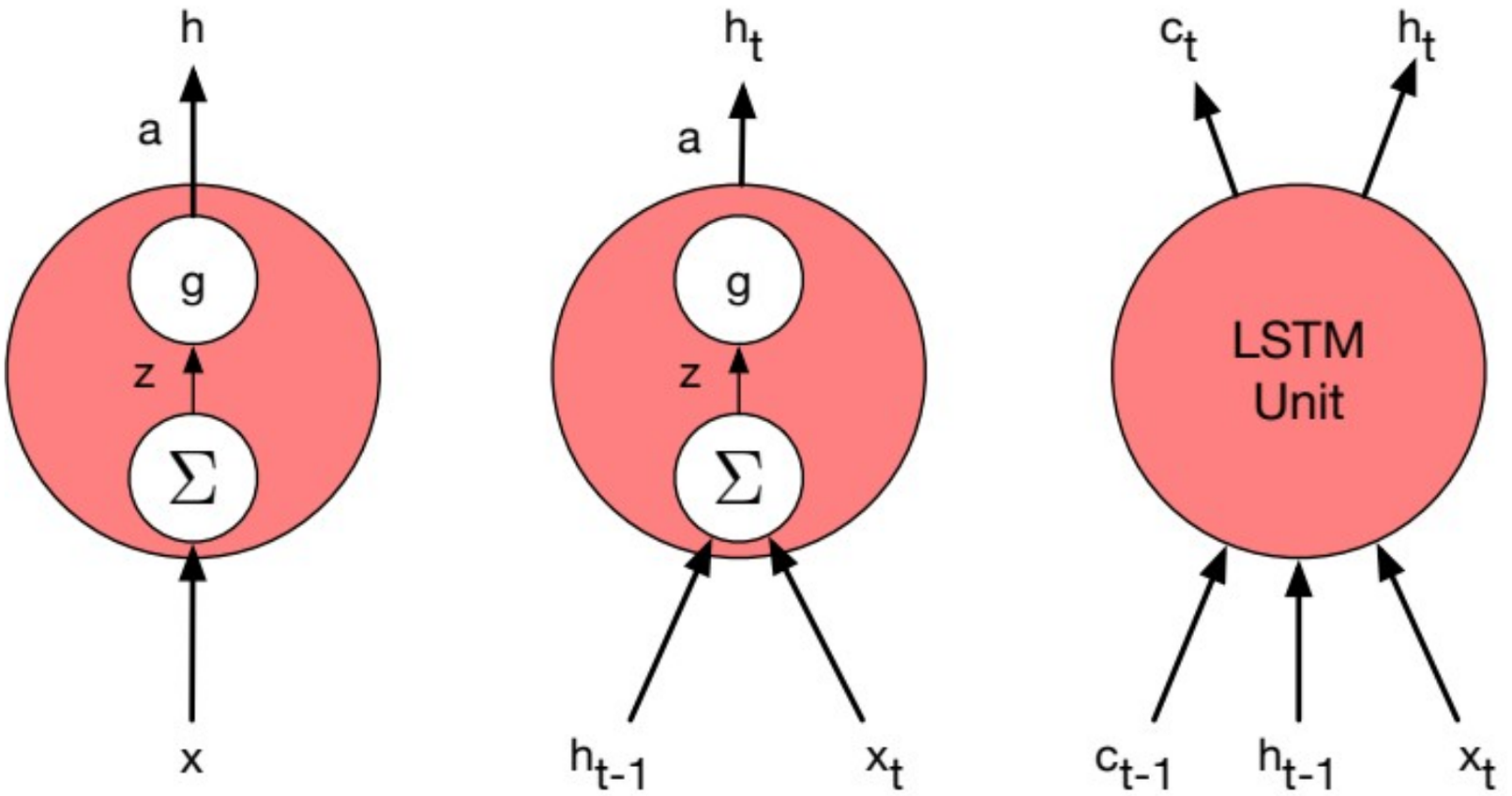
$$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

The LSTM



Units



(a) (b) (c)

FF SR LST
N N M

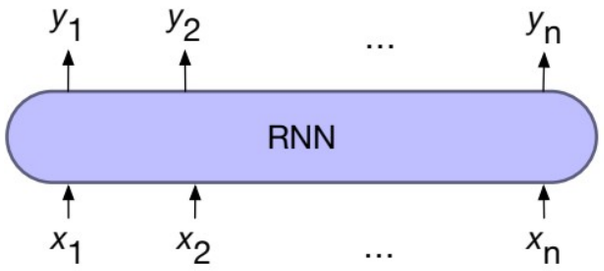
RNNs and LSTMs

The LSTM

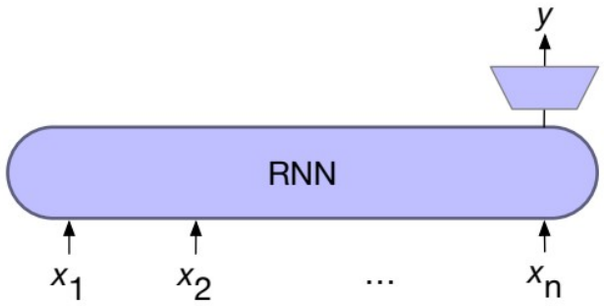
RNNs and LSTMs

The LSTM Encoder-Decoder Architecture

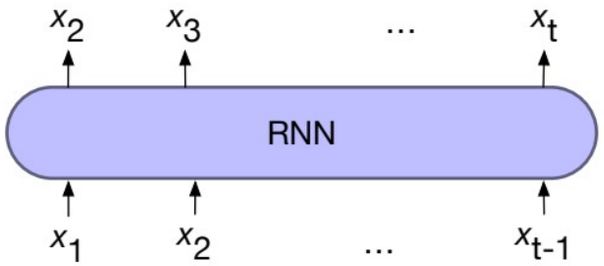
Four architectures for NLP tasks with RNNs



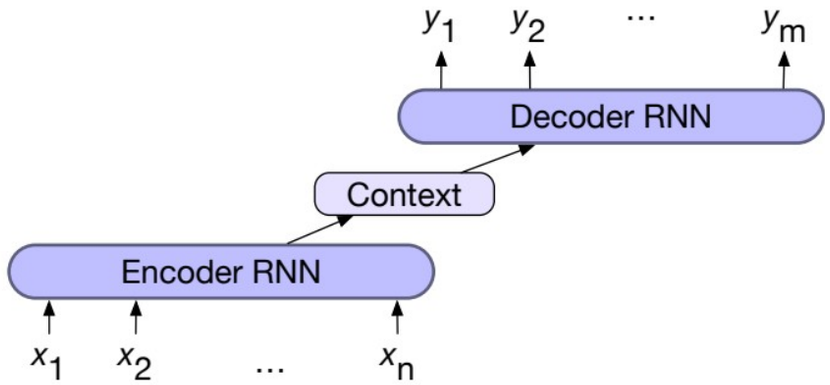
a) sequence labeling



b) sequence classification



c) language modeling

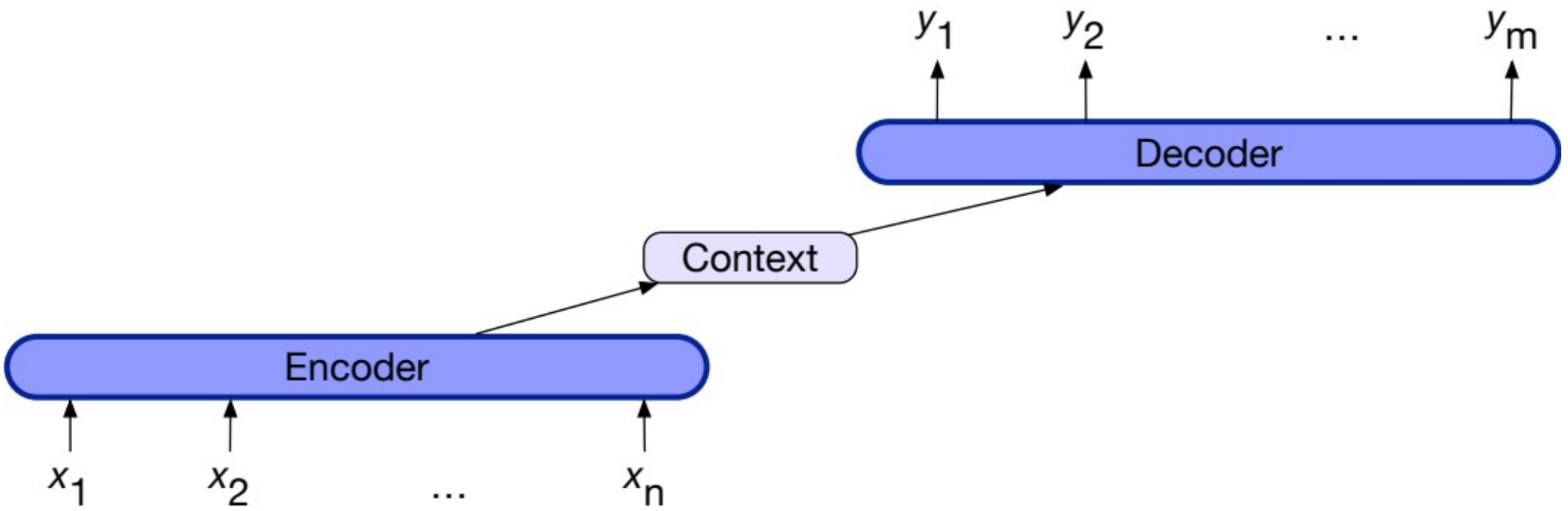


d) encoder-decoder

3 components of an encoder-decoder

1. An encoder that accepts an input sequence, $x_{1:n}$, and generates a corresponding sequence of contextualized representations, $h_{1:n}$.
2. A context vector, c, which is a function of $h_{1:n}$, and conveys the essence of the input to the decoder.
3. A decoder, which accepts c as input and generates an arbitrary length sequence of hidden states $h_{1:m}$, from which a corresponding sequence of output states $y_{1:m}$, can be obtained

Encoder-decoder



Encoder-decoder for translation

Regular language modeling

$$p(y) = p(y_1)p(y_2|y_1)p(y_3|y_1,y_2)\cdots p(y_m|y_1,\cdots,y_{m-1})$$

$$\mathbf{h}_t = g(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{h}_t)$$

Encoder-decoder for translation

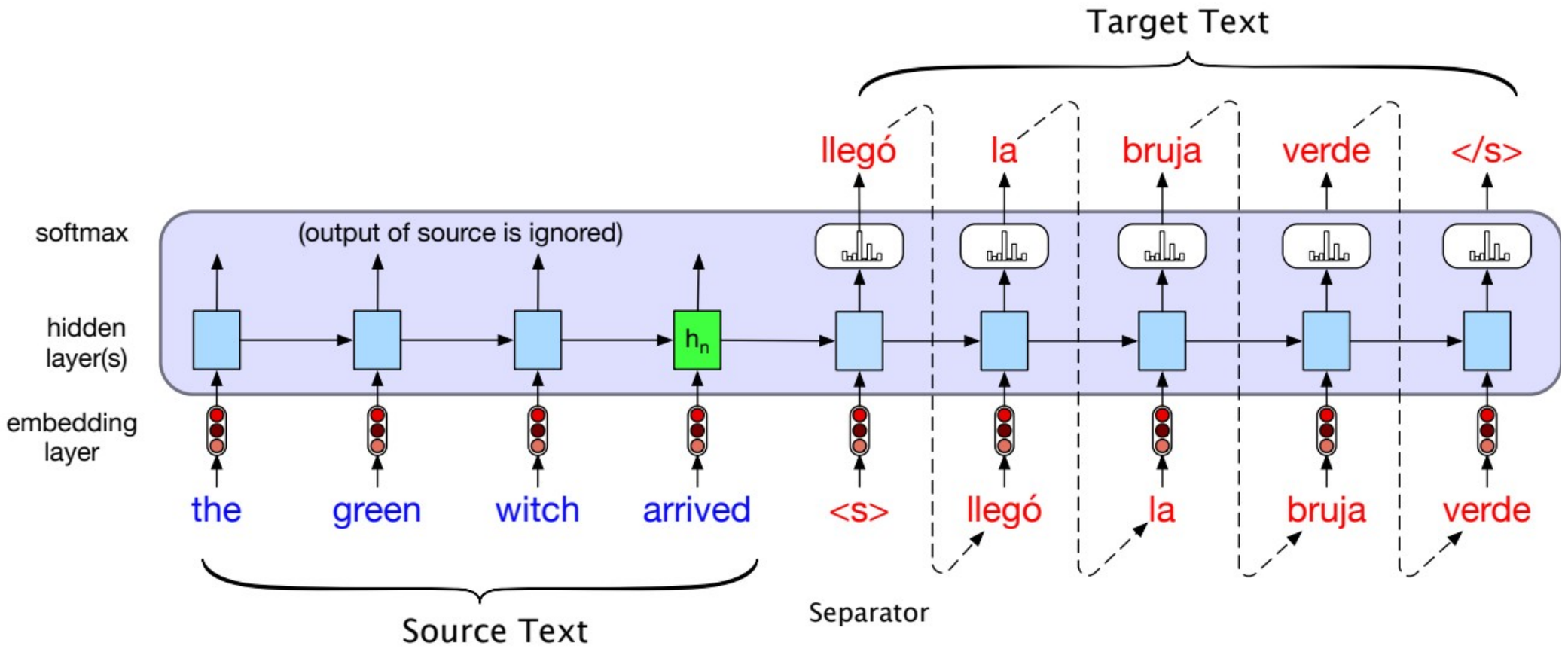
Let x be the source text plus a separate token <s> and y the target

Let x = The green witch arrive <s>

Let y = *llego´ la bruja verde*

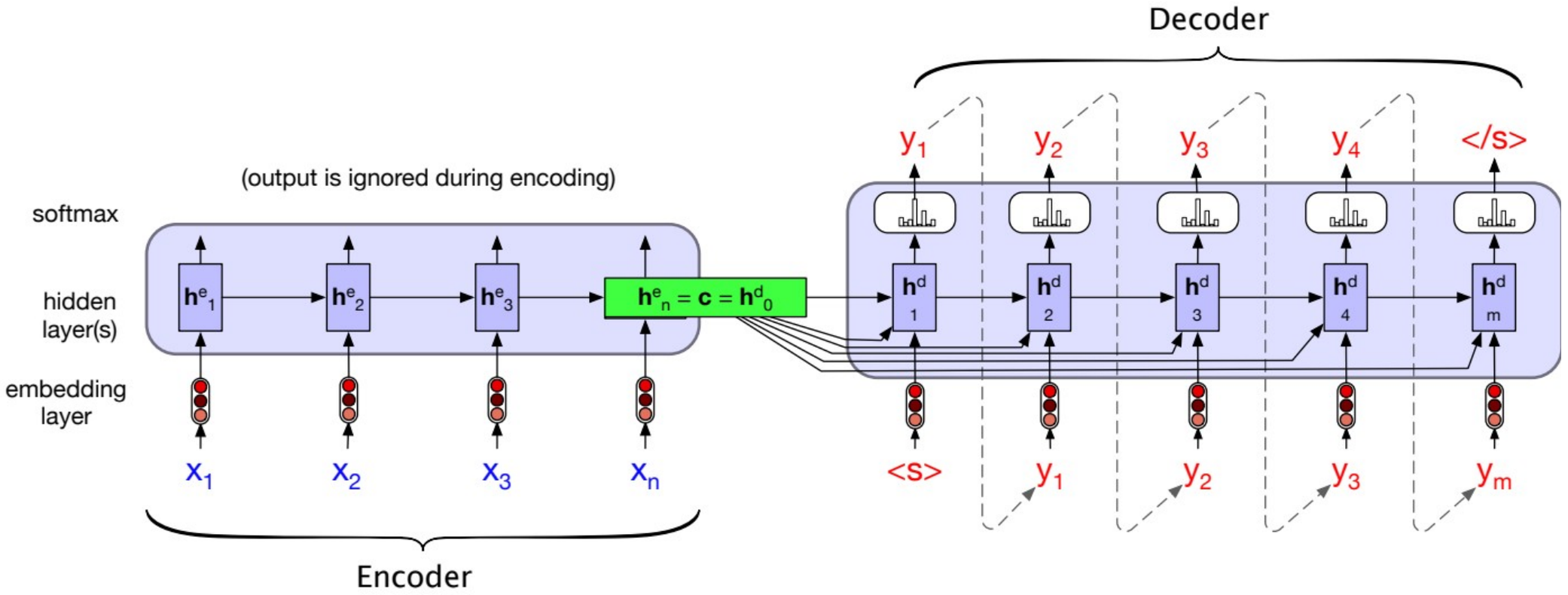
$$p(y|x) = p(y_1|x)p(y_2|y_1,x)p(y_3|y_1,y_2,x)\cdots p(y_m|y_1,\cdots,y_{m-1},x)$$

Encoder-decoder simplified



Encoder-decoder showing context

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$



Encoder-decoder equations

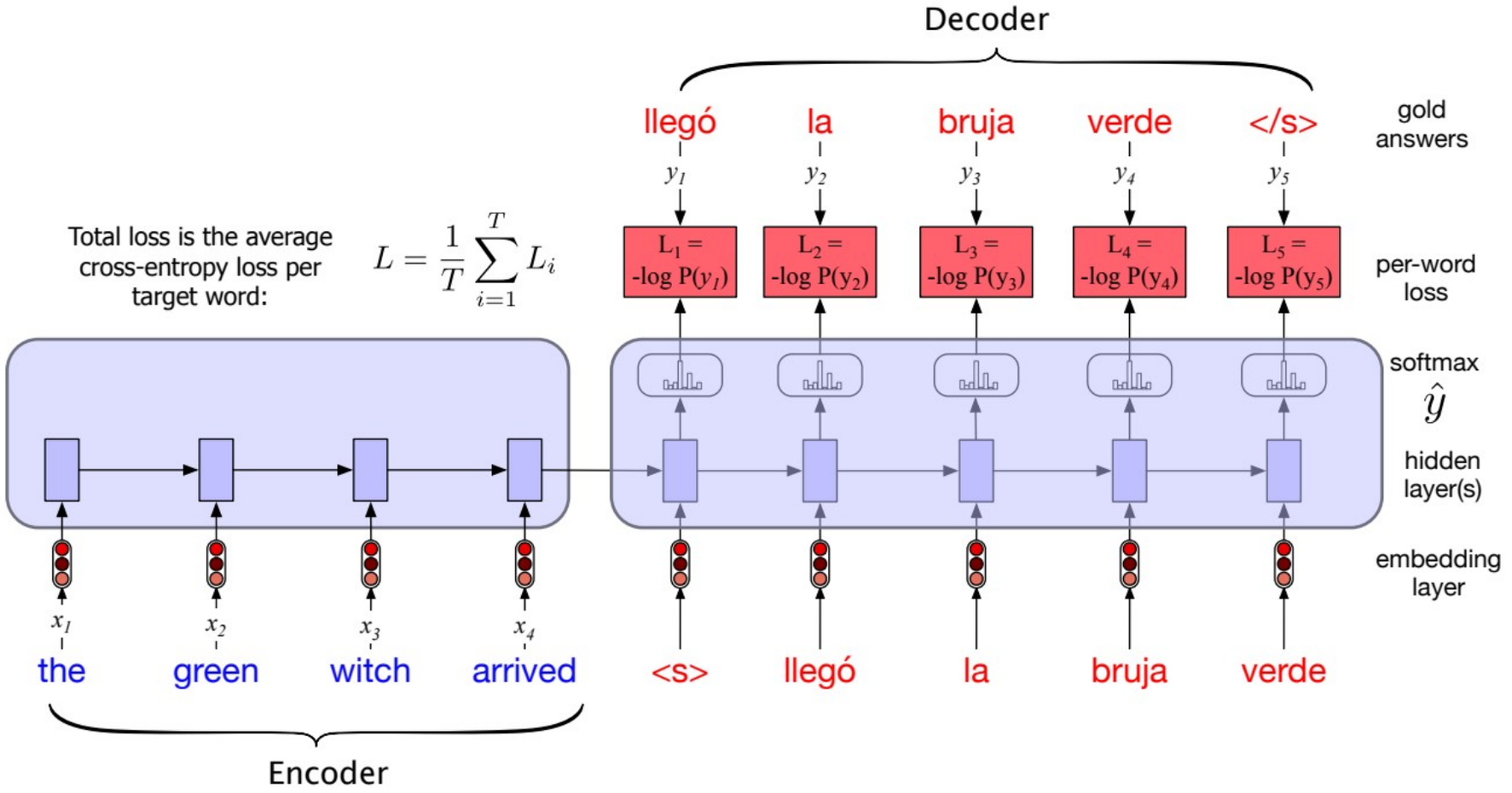
$$
\begin{aligned}
\mathbf{c} &= \mathbf{h}_n^e \\
\mathbf{h}_0^d &= \mathbf{c} \\
\mathbf{h}_t^d &= g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \\
\hat{\mathbf{y}}_t &= \mathbf{softmax}(\mathbf{h}_t^d)
\end{aligned}
$$

g is a stand-in for some flavor of RNN

$\hat{y}_{t-1}$ is the embedding for the output sampled from the softmax at the previous step

$\hat{y}_t$ is a vector of probabilities over the vocabulary, representing the probability of each word occurring at time t. To generate text, we sample from this distribution $\hat{y}_t$.

Training the encoder-decoder with teacher forcing



RNNs and LSTMs

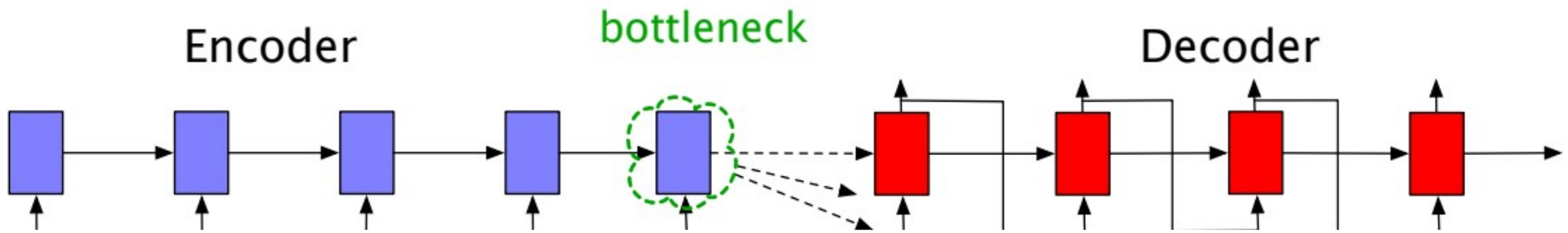
The LSTM Encoder-Decoder Architecture

RNNs and LSTMs

LSTM Attention

Problem with passing context c only from end

Requiring the context *c* to be only the encoder's final hidden state forces all the information from the entire source sentence to pass through this representational bottleneck.



Solution: attention

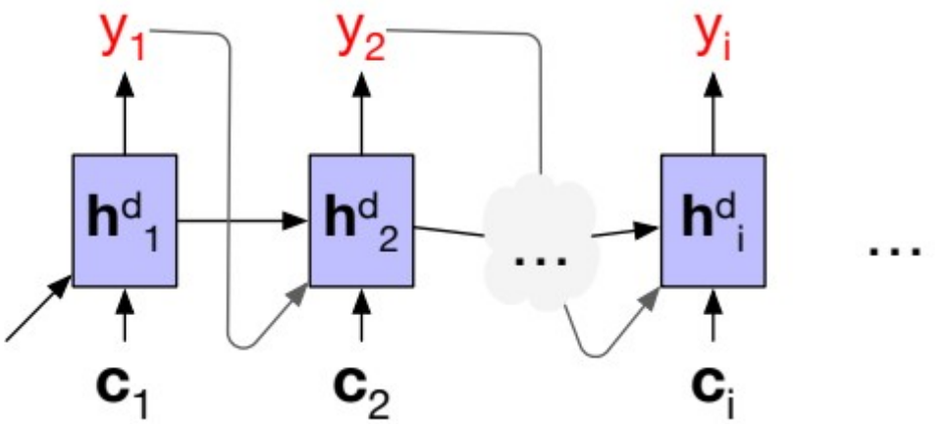
instead of being taken from the last hidden state, the context it’s a weighted average of all the hidden states of the decoder.

this weighted average is also informed by part of the decoder state as well, the state of the decoder right before the current token *i*.

$$\mathbf{c} = f(\mathbf{h}_1^e \dots \mathbf{h}_n^e, \mathbf{h}_{i-1}^d)$$

Attention

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



How to compute c?

We'll create a score that tells us how much to focus on each encoder state, how *relevant* each encoder state is to the decoder state:

$$\text{score}(\mathbf{h}^d_{i-1}, \mathbf{h}^e_j) = \mathbf{h}^d_{i-1} \cdot \mathbf{h}^e_j$$

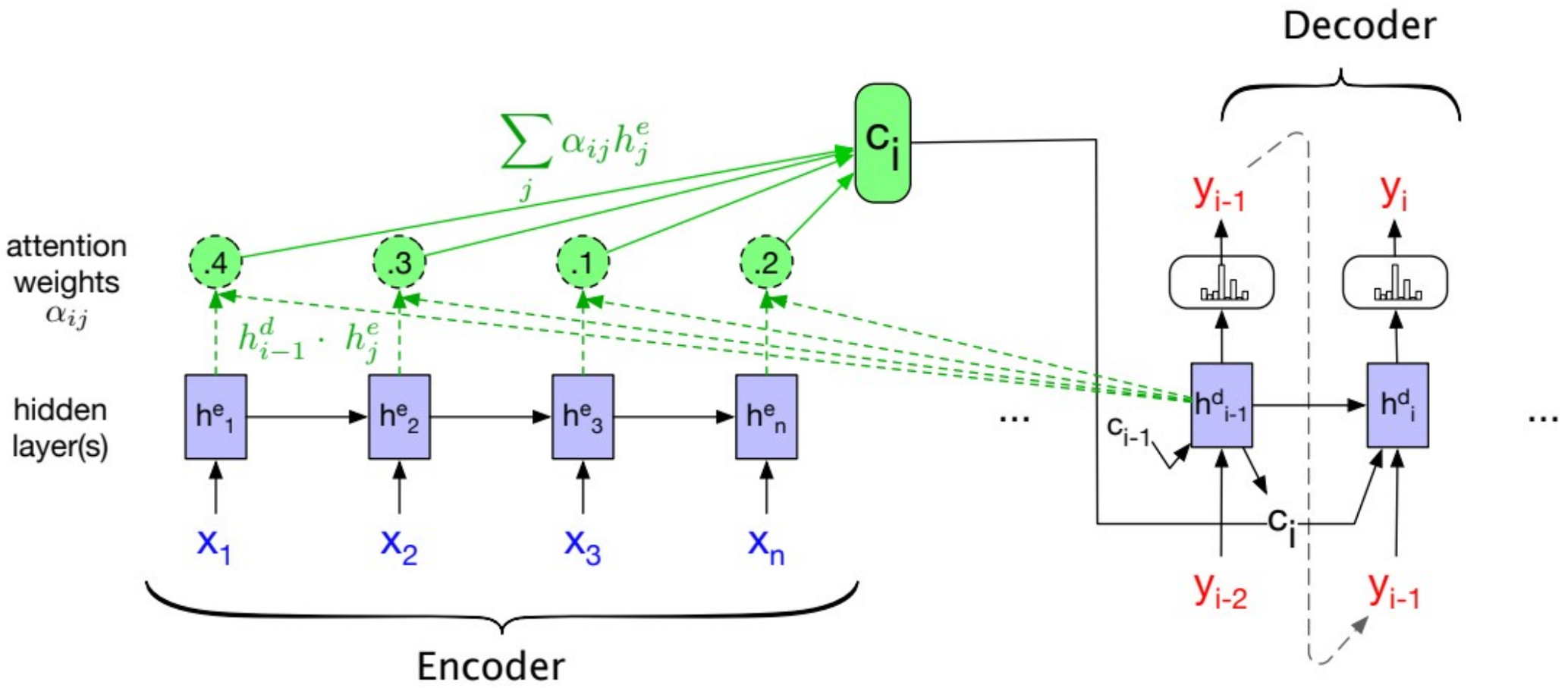
We'll normalize them with a softmax to create weights $\alpha_{i\,j}$, that tell us the relevance of encoder hidden state *j* to hidden decoder state, h^d_{i-1}

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}^d_{i-1}, \mathbf{h}^e_j))$$

$$\mathbf{c}_i = \sum_j \alpha_{ij}\, \mathbf{h}^e_j$$

And then use this to help create a weighted average:

Encoder-decoder with attention, focusing on the computation of c



RNNs and LSTMs

LSTM Attention

Transformers

Introduction to Transformers

LLMs are built out of transformers

Transformer: a specific kind of network architecture, like a fancier feedforward network, but based on attention

---

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

A very approximate timeline

1990 Static Word Embeddings

2003 Neural Language Model

2008 Multi-Task Learning

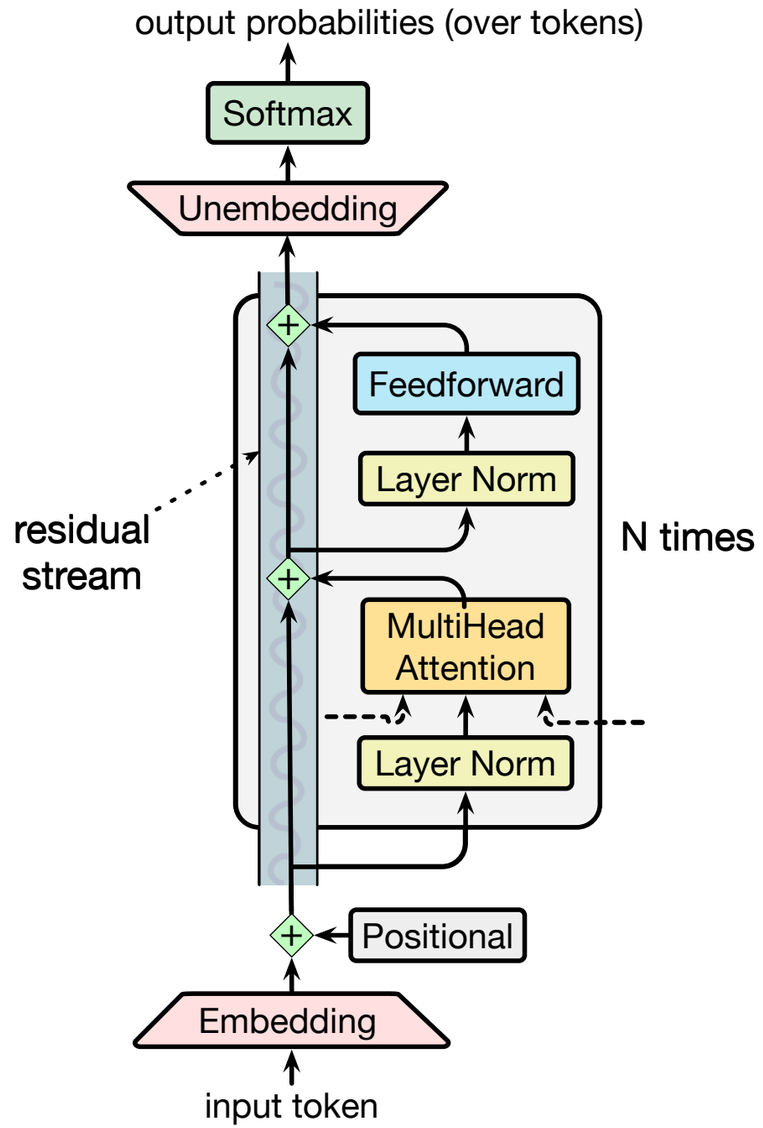
2015 Attention

2017 Transformer

2018 Contextual Word Embeddings and Pretraining

2019 Prompting

The transformer

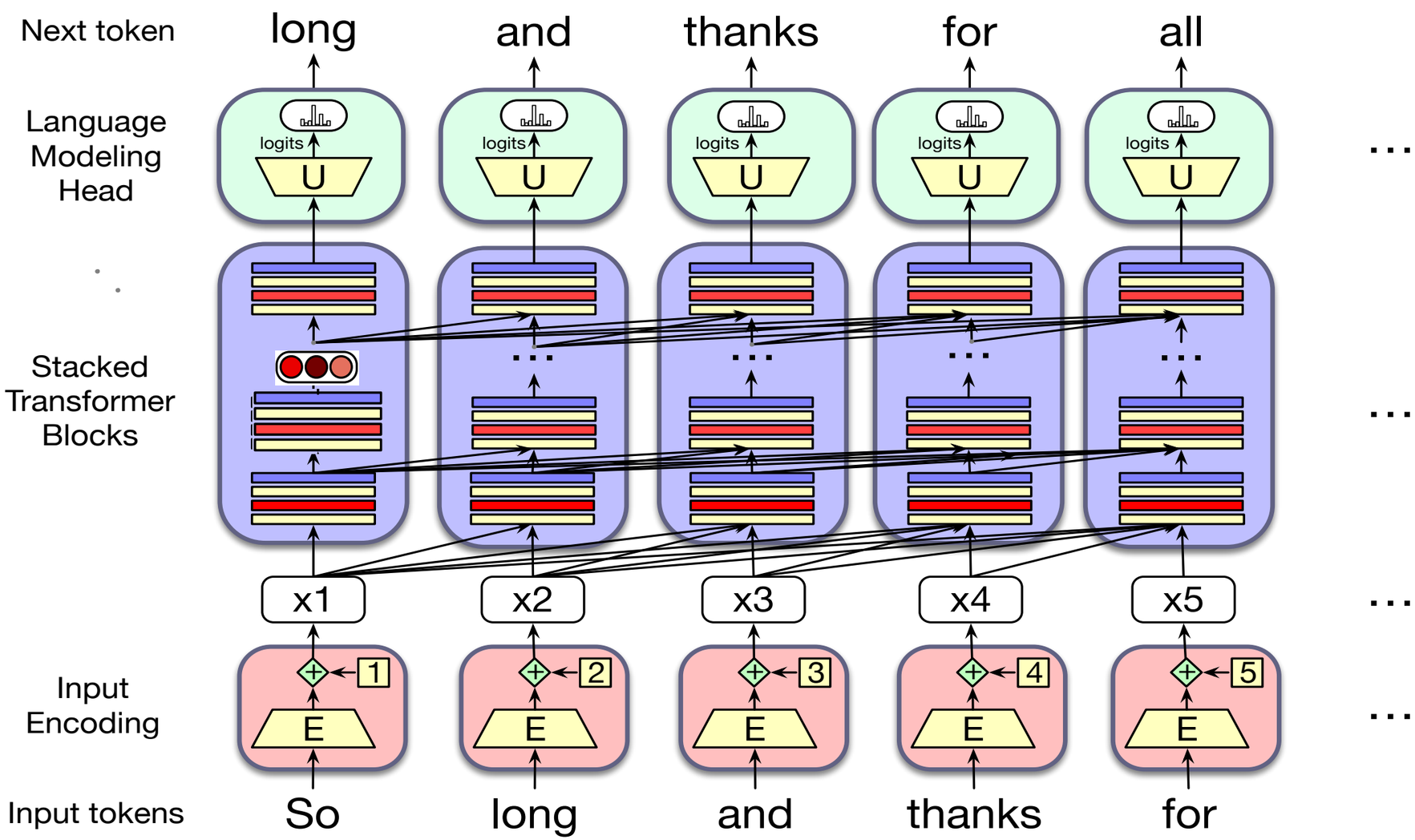


Transformers

Attention

Instead of starting with the big picture

Let's consider the embeddings for an individual word from a particular layer



Problem with static embeddings (word2vec)

They are static! The embedding for a word doesn't reflect how its meaning changes in context.

The chicken didn't cross the road because it was too tired

What is the meaning represented in the static embedding for "it"?

Contextual Embeddings

- Intuition: a representation of meaning of a word should be different in different contexts!
- **Contextual Embedding**: each word has a different vector that expresses different meanings depending on the surrounding words
- How to compute contextual embeddings?
  - **Attention**

Contextual Embeddings

```
The chicken didn't cross the road because it
```

What should be the properties of "it"?

```
The chicken didn't cross the road because it was too tired
The chicken didn't cross the road because it was too wide
```

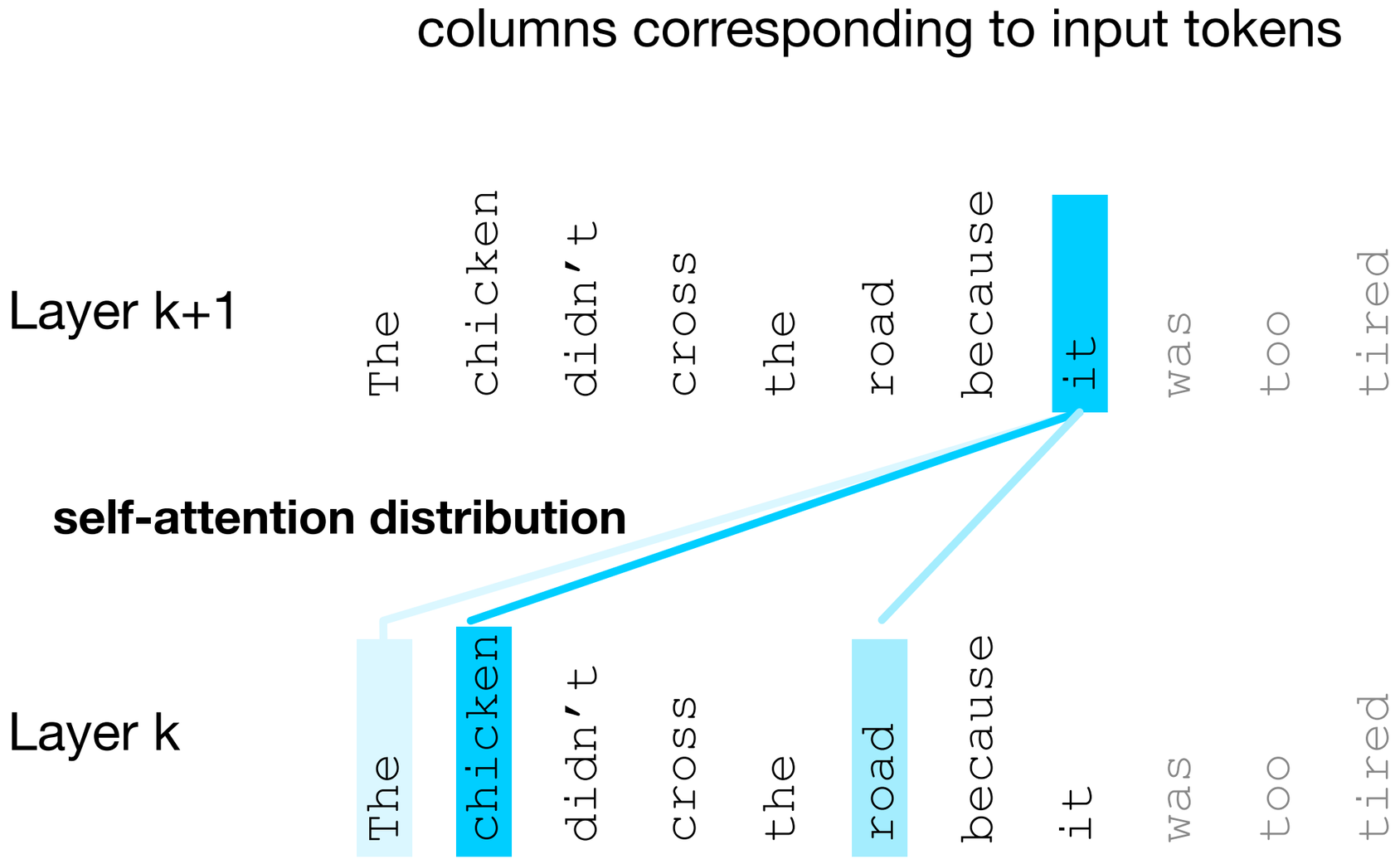
At this point in the sentence, it's probably referring to either the chicken or the street

Intuition of attention

Build up the contextual embedding from a word by selectively integrating information from all the neighboring words

We say that a word "attends to" some neighboring words more than others

Intuition of attention:



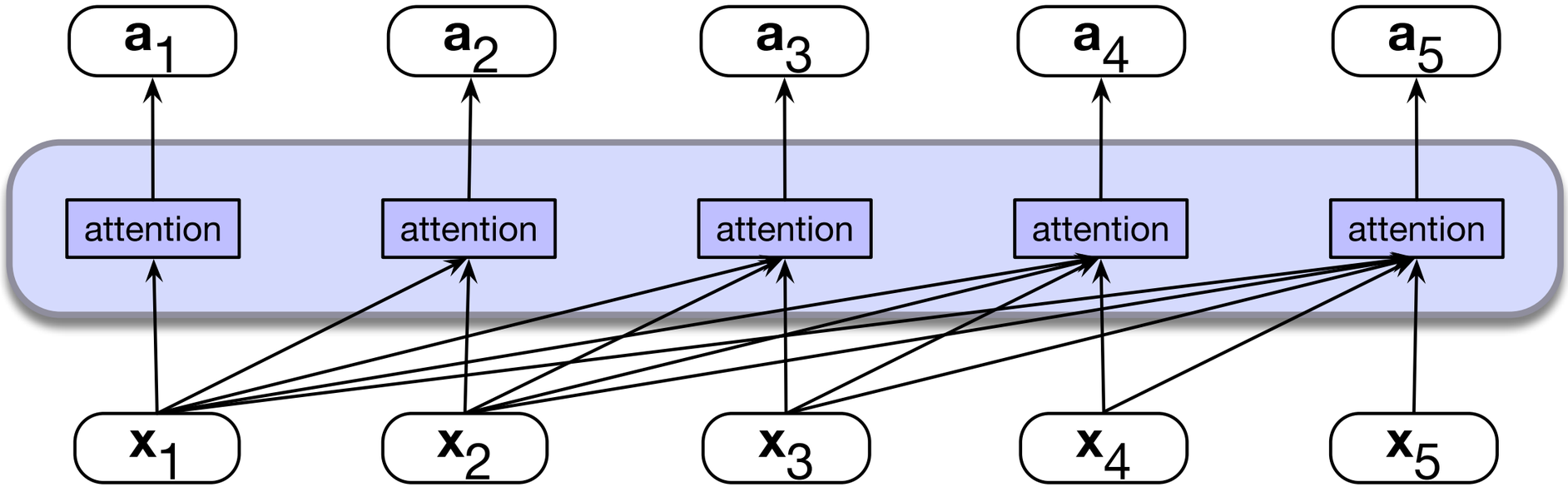
Attention definition

A mechanism for helping compute the embedding for a token by selectively attending to and integrating information from surrounding tokens (at the previous layer).

More formally: a method for doing a weighted sum of vectors.

Attention is left-to-right

Self-Attention Layer



Simplified version of attention: a sum of prior words weighted by their similarity with the current word

Given a sequence of token embeddings:

$$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$$

Produce: $\mathbf{a}_i$ = a weighted sum of $\mathbf{x}_1$ through $\mathbf{x}_7$ (and $\mathbf{x}_i$)

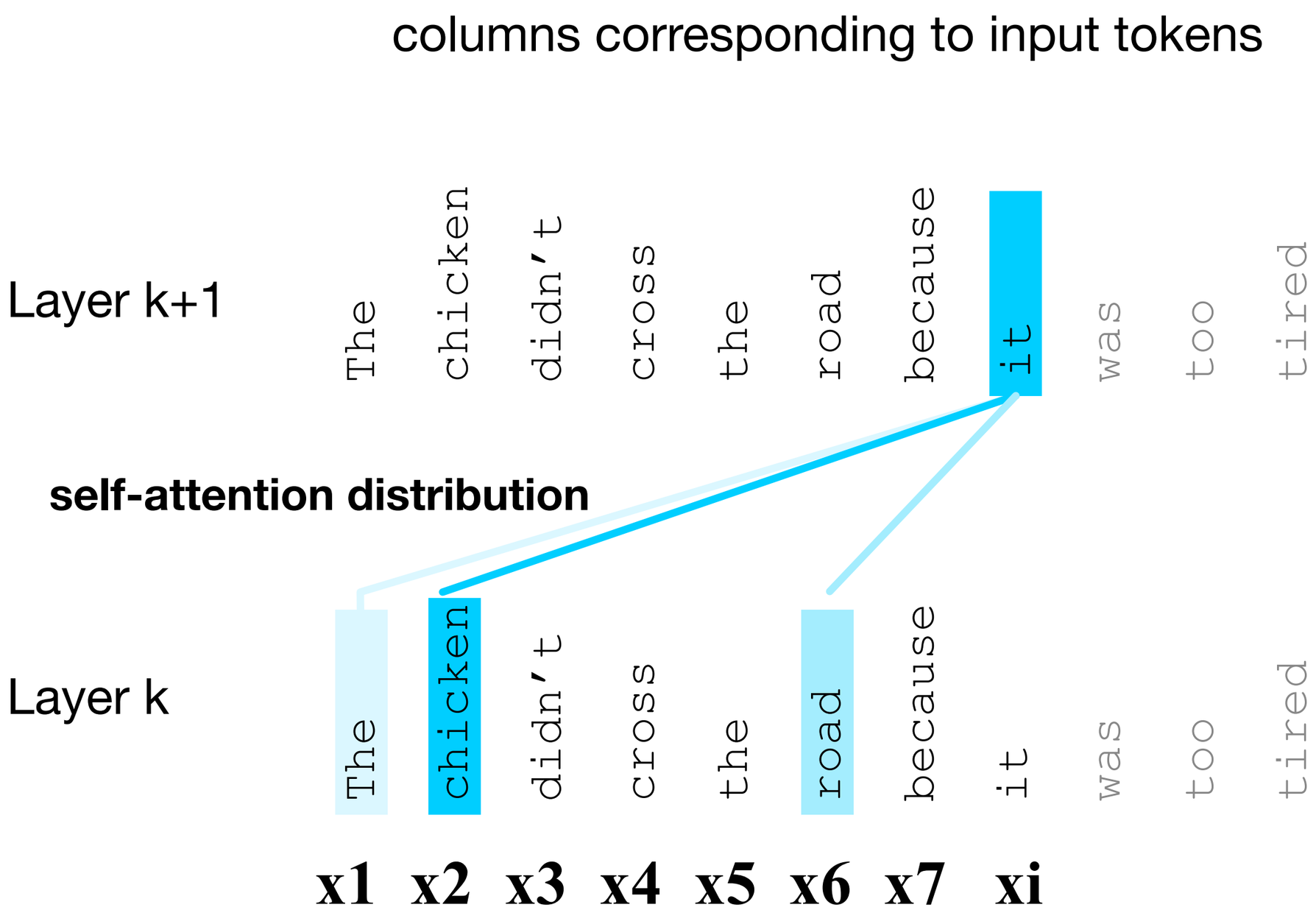
Weighted by their similarity to $\mathbf{x}_i$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

Intuition of attention:

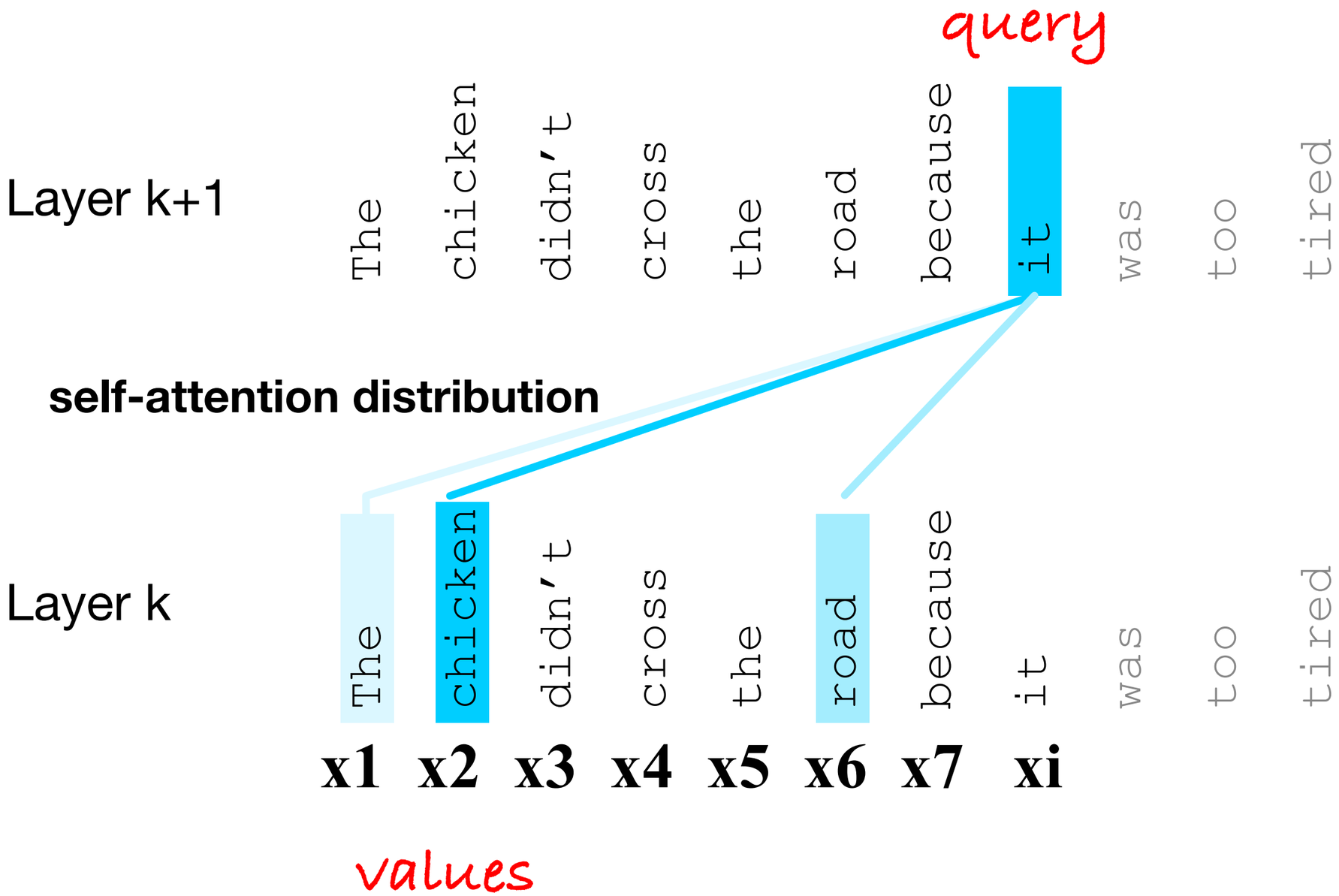


An Actual Attention Head: slightly more complicated

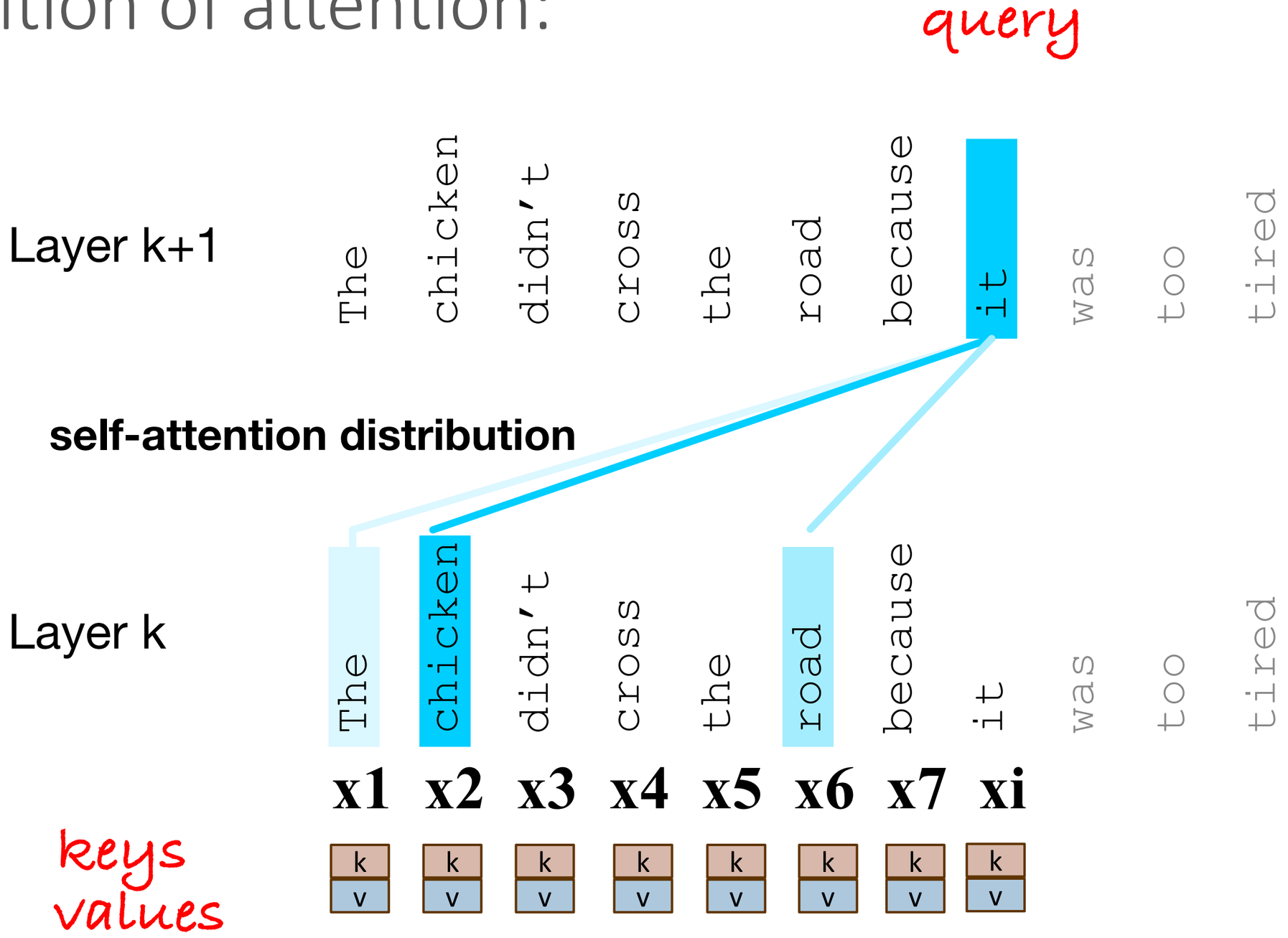
High-level idea: instead of using vectors (like x_i and x_4) directly, we'll represent 3 separate roles each vector $\mathbf{x}_i$ plays:

- **query**: As *the current element* being compared to the preceding inputs.
- **key**: as *a preceding input* that is being compared to the current element to determine a similarity
- **value**: a value of a preceding element that gets weighted and summed

Attention intuition



Intuition of attention:



An Actual Attention Head: slightly more complicated

We'll use matrices to project each vector $\mathbf{x}_i$ into a representation of its role as query, key, value:

- **query**: $\mathbf{W}^Q$
- **key**: $\mathbf{W}^K$
- **value**: $\mathbf{W}^V$

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^V$$

An Actual Attention Head: slightly more complicated

Given these 3 representation of $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{V}}$$

To compute similarity of current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$

We'll use dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$.

And instead of summing up $\mathbf{x}_j$, we'll sum up $\mathbf{v}_j$

Final equations for one attention head

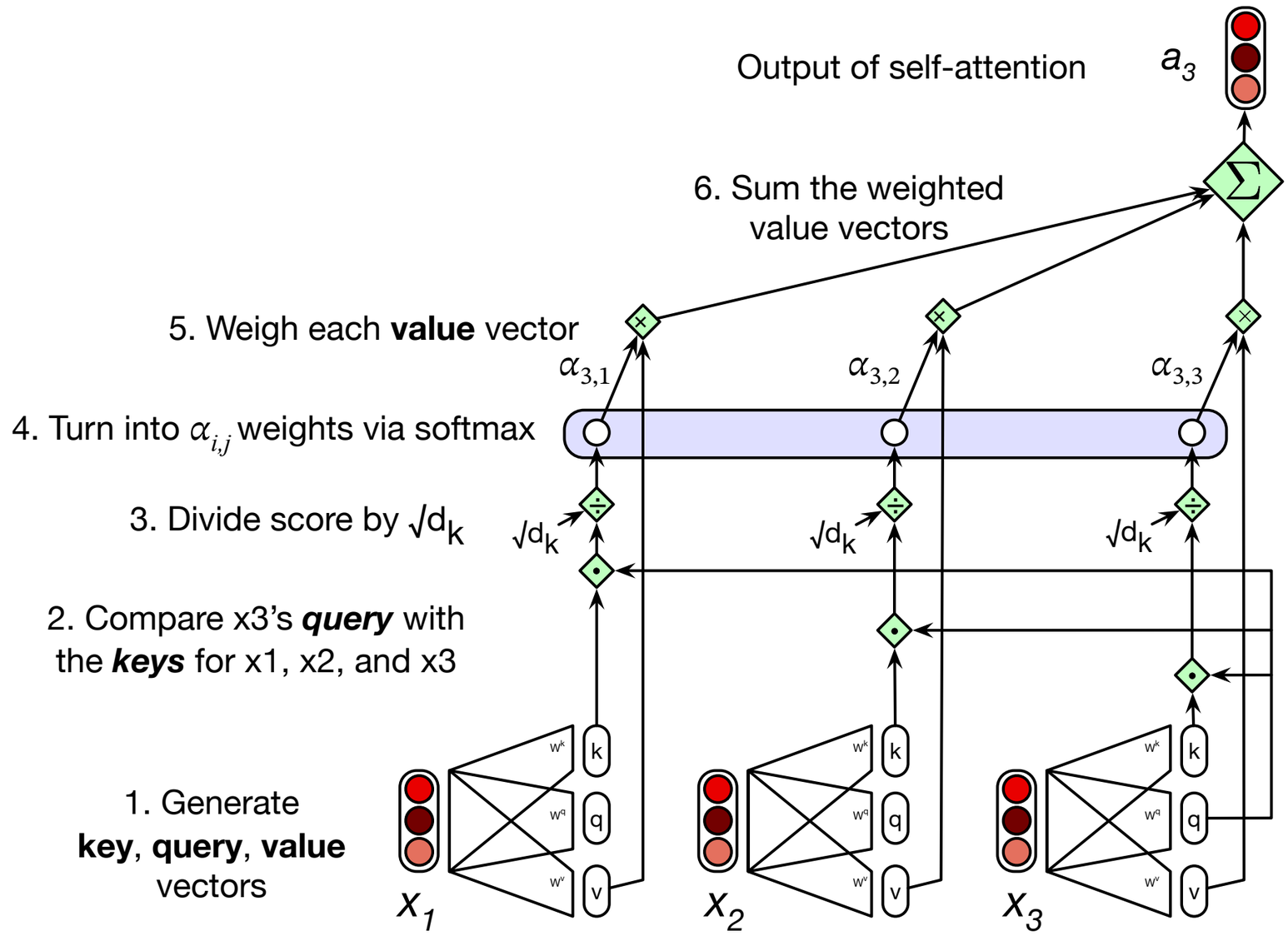
$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^{\mathbf{V}}$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Calculating the value of a3



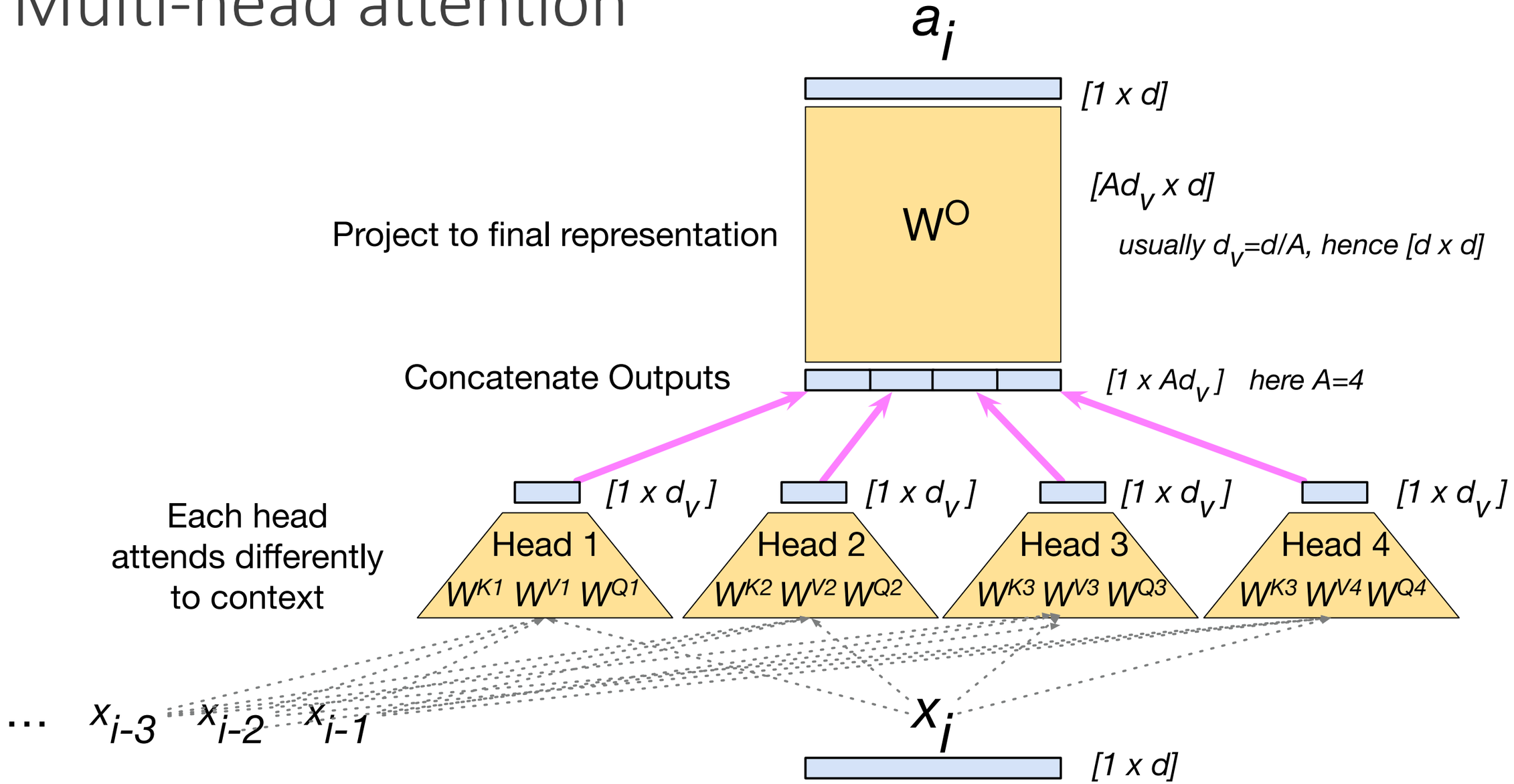
Actual Attention: slightly more complicated

- Instead of one attention head, we'll have lots of them!
- Intuition: each head might be attending to the context for different purposes
  - Different linguistic relationships or patterns in the context

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W^{Qc}}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W^{Kc}}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W^{Vc}}; \quad \forall\, c \;\; 1 \leq c \leq h$$

$$\begin{aligned}
\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) &= \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}} \\
\alpha_{ij}^c &= \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \;\; \forall j \leq i \\
\mathbf{head}_i^c &= \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c \\
\mathbf{a}_i &= (\mathbf{head}^1 \oplus \mathbf{head}^2 \ldots \oplus \mathbf{head}^h)\mathbf{W}^O \\
\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \cdots, \mathbf{x}_N]) &= \mathbf{a}_i
\end{aligned}$$

Multi-head attention



Summary

Attention is a method for enriching the representation of a token by incorporating contextual information

The result: the embedding for each word will be different in different contexts!

Contextual embeddings: a representation of word meaning in its context.

We'll see in the next lecture that attention can also be viewed as a way to move information from one token to another.

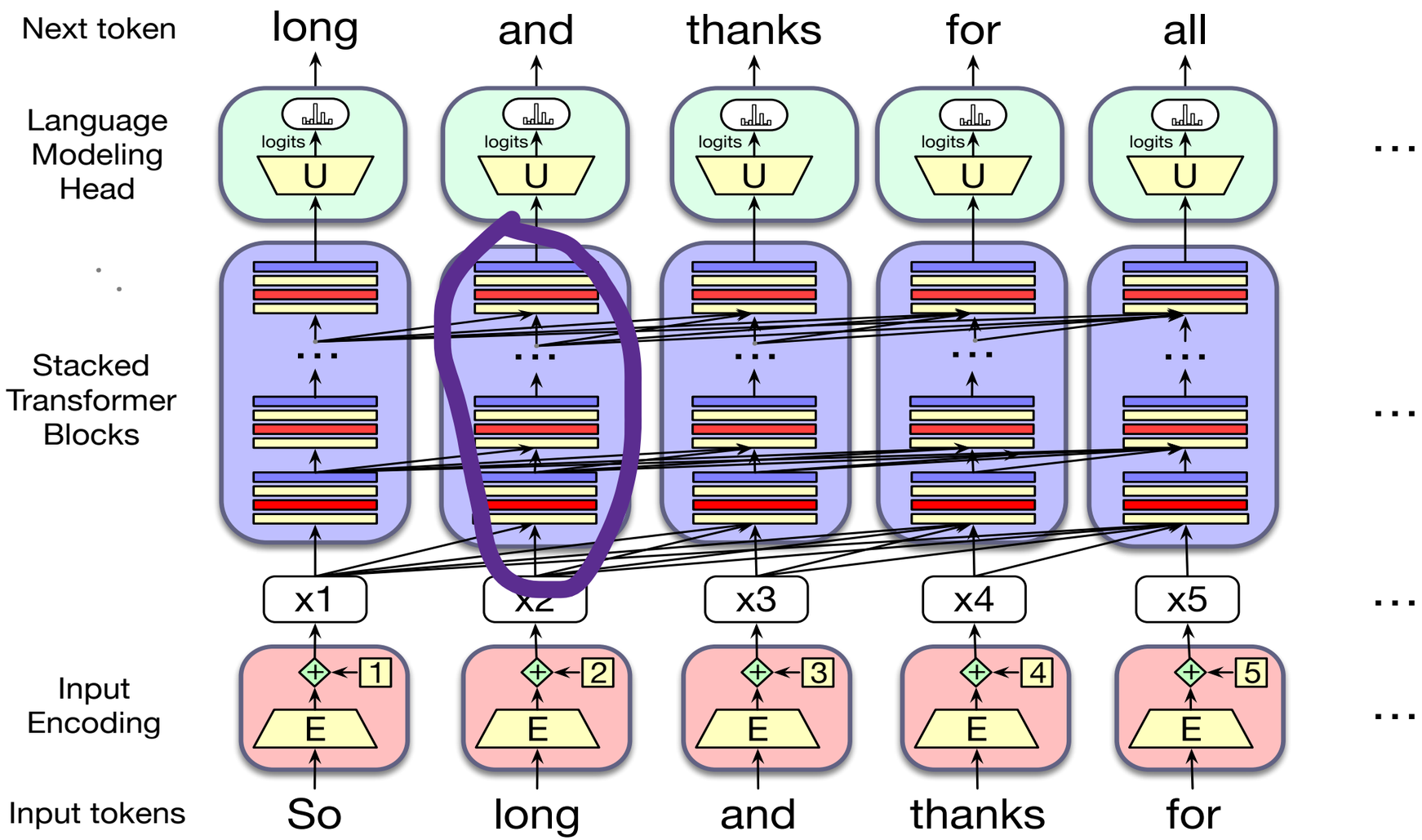
Transformers

Attention

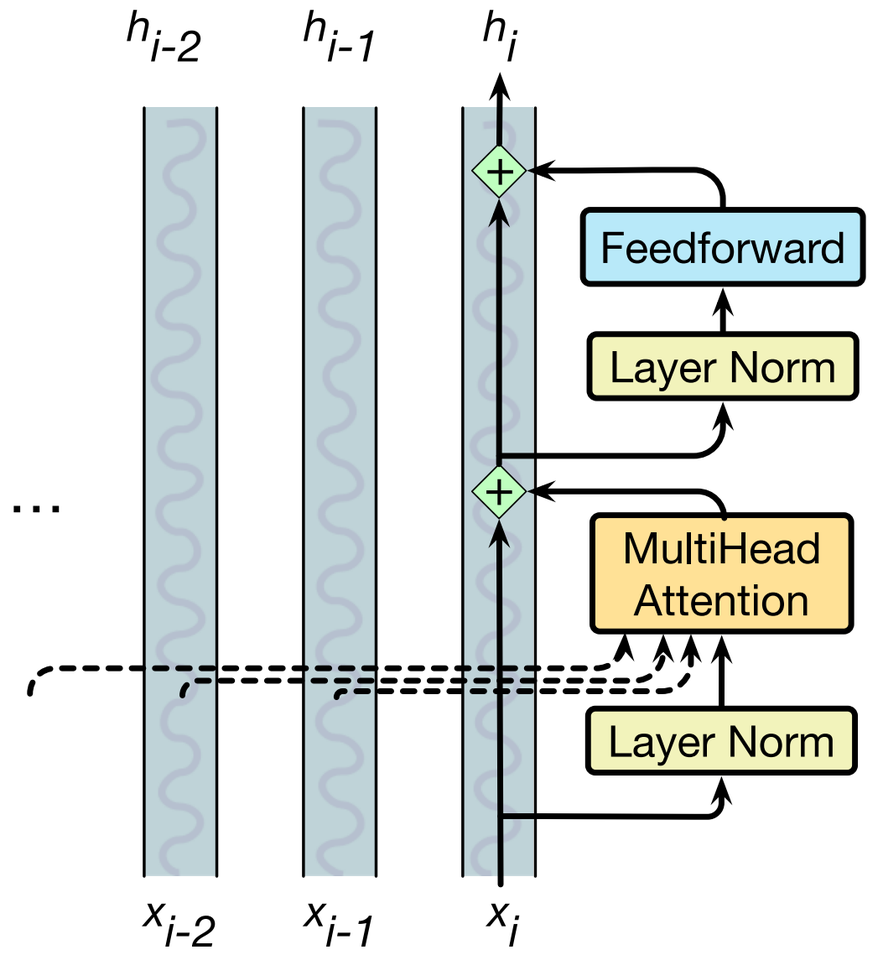
Transformers

The Transformer Block

Reminder: transformer language model

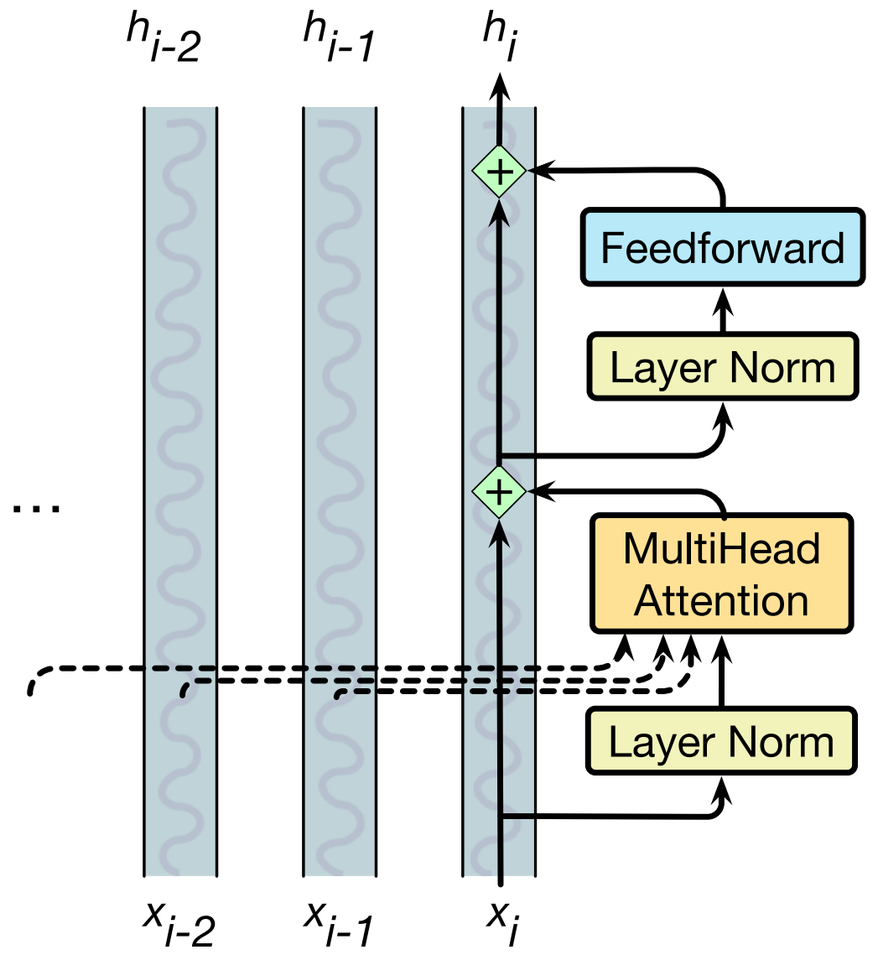


The residual stream: each token gets passed up and modified

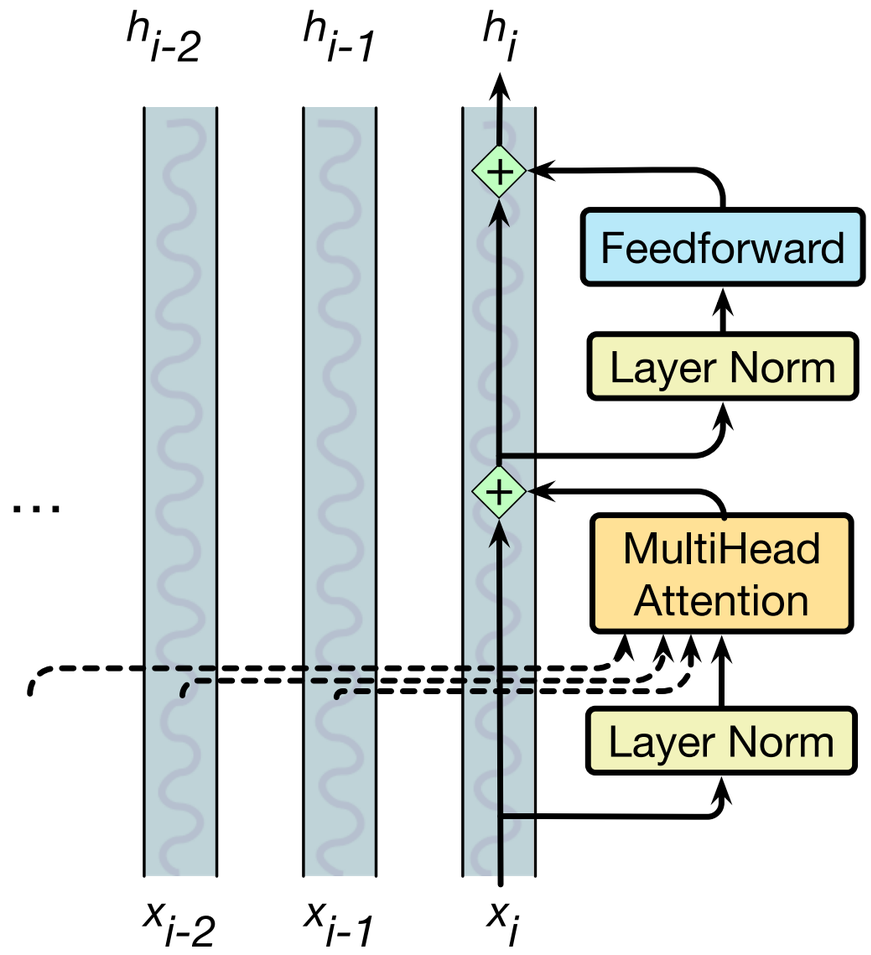


We'll need nonlinearities, so a feedforward layer

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$



Layer norm: the vector $\mathbf{x}_i$ is normalized twice



Layer Norm

Layer norm is a variation of the z-score from statistics, applied to a single vec- tor in a hidden layer

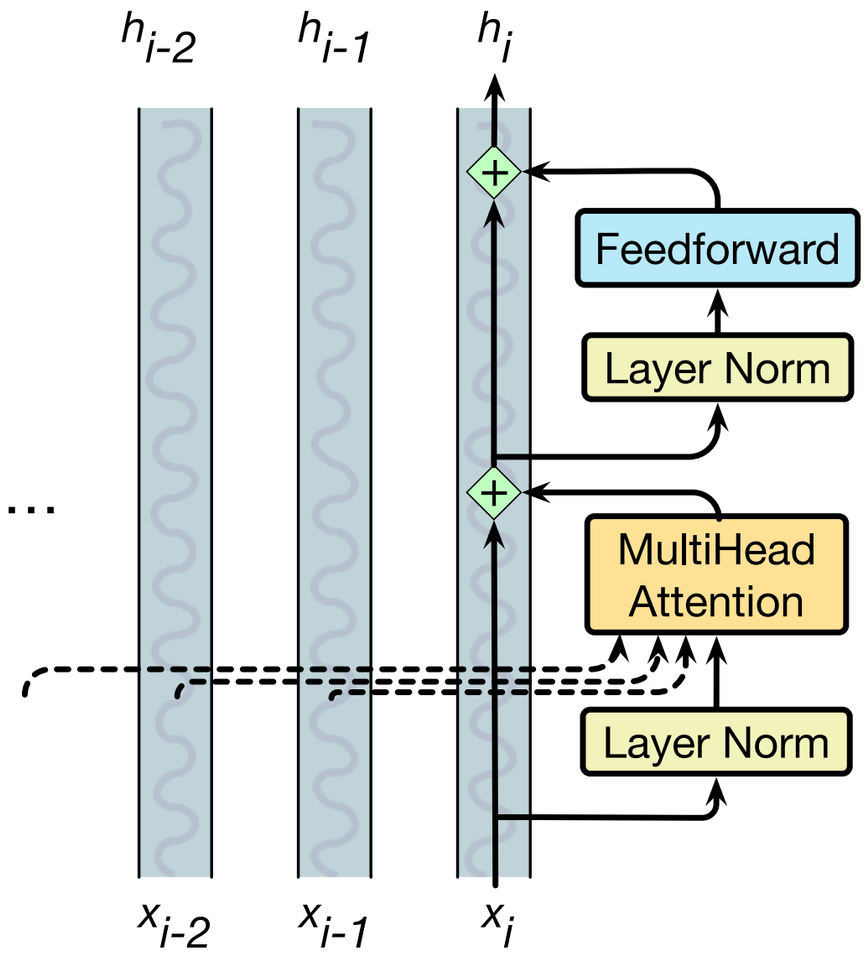
$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

$$\sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

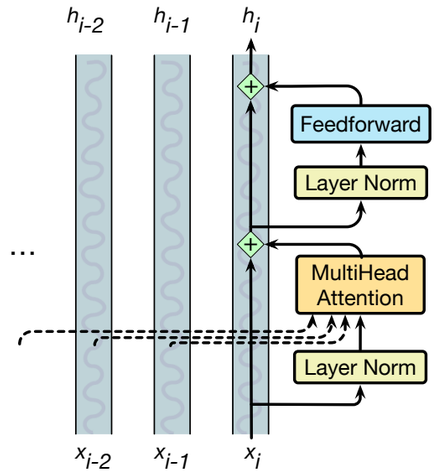
Putting together a single transformer block



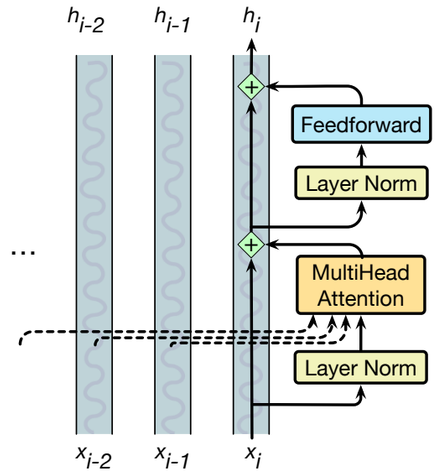
$$\mathbf{t}_i^1 = \text{LayerNorm}(\mathbf{x}_i)$$
$$\mathbf{t}_i^2 = \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{x}_1^1, \cdots, \mathbf{x}_N^1])$$
$$\mathbf{t}_i^3 = \mathbf{t}_i^2 + \mathbf{x}_i$$
$$\mathbf{t}_i^4 = \text{LayerNorm}(\mathbf{t}_i^3)$$
$$\mathbf{t}_i^5 = \text{FFN}(\mathbf{t}_i^4)$$
$$\mathbf{h}_i = \mathbf{t}_i^5 + \mathbf{t}_i^3$$

A transformer is a stack of these blocks
so all the vectors are of the same dimensionality d

Block 2



Block 1

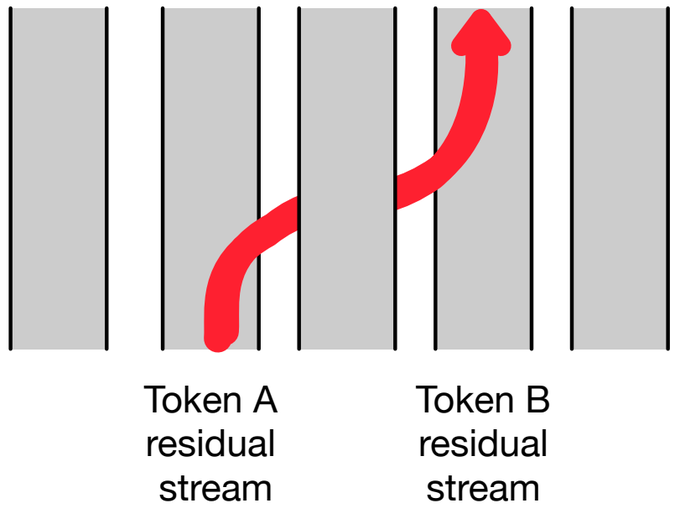


Residual streams and attention

Notice that all parts of the transformer block apply to 1 residual stream (1 token).

Except attention, which takes information from other tokens

Elhage et al. (2021) show that we can view attention heads as literally moving information from the residual stream of a neighboring token into the current stream .



Transformers

The Transformer Block

Transformers

Parallelizing Attention Computation

Parallelizing computation using X

For attention/transformer block we've been computing a **single** output at a **single** time step *i* in a **single** residual stream.

But we can pack the *N* tokens of the input sequence into a single matrix **X** of size [$N \times d$].

Each row of X is the embedding of one token of the input.

X can have 1K-32K rows, each of the dimensionality of the embedding *d* (the **model dimension**)

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{\mathbf{Q}};\ \ \mathbf{K} = \mathbf{X}\mathbf{W}^{\mathbf{K}};\ \ \mathbf{V} = \mathbf{X}\mathbf{W}^{\mathbf{V}}$$

QK^T

Now can do a single matrix multiply to combine Q and K^T

N	**q1·k1**	**q1·k2**	**q1·k3**	**q1·k4**
	q2·k1	**q2·k2**	**q2·k3**	**q2·k4**
	q3·k1	**q3·k2**	**q3·k3**	**q3·k4**
	q4·k1	**q4·k2**	**q4·k3**	**q4·k4**

N

Parallelizing attention

- Scale the scores, take the softmax, and then multiply the result by V resulting in a matrix of shape $N \times d$
  - An attention vector for each input token

$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

Masking out the future

$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{QK}^\mathsf{T}}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

- What is this mask function?
  QK^T has a score for each query dot every key, *including those that follow the query*.
- Guessing the next word is pretty simple if you already know it!

Masking out the future

$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

Add $-\infty$ to cells in upper triangle

The softmax will turn it to 0

q1·k1	−∞	−∞	−∞
q2·k1	q2·k2	−∞	−∞
q3·k1	q3·k2	q3·k3	−∞
q4·k1	q4·k2	q4·k3	q4·k4

N

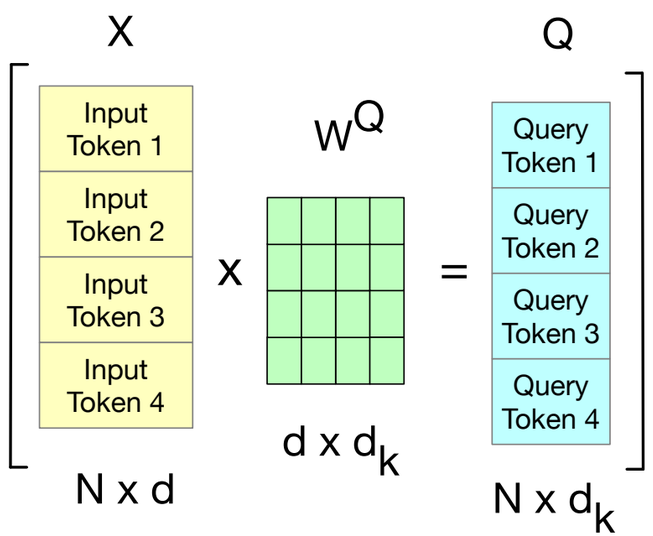
N

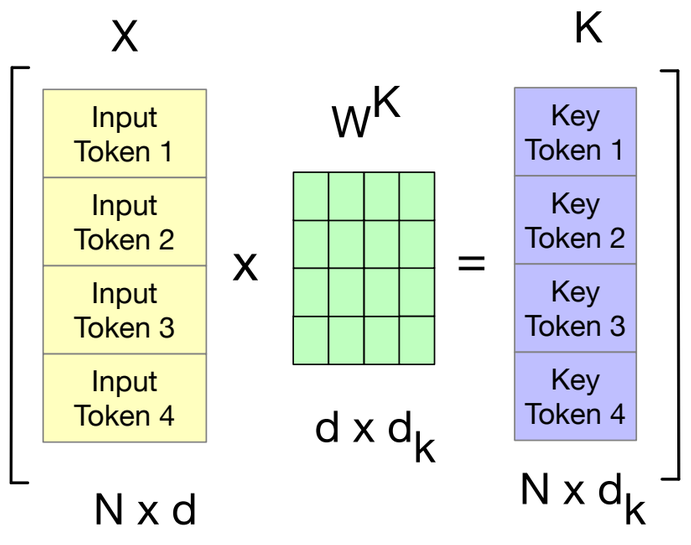
Another point: Attention is quadratic in length

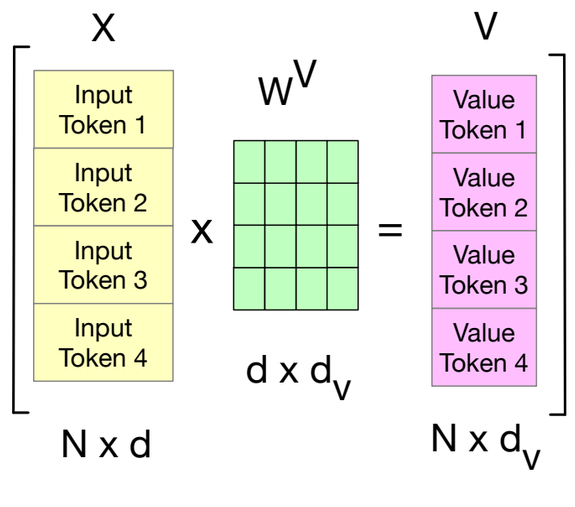
$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

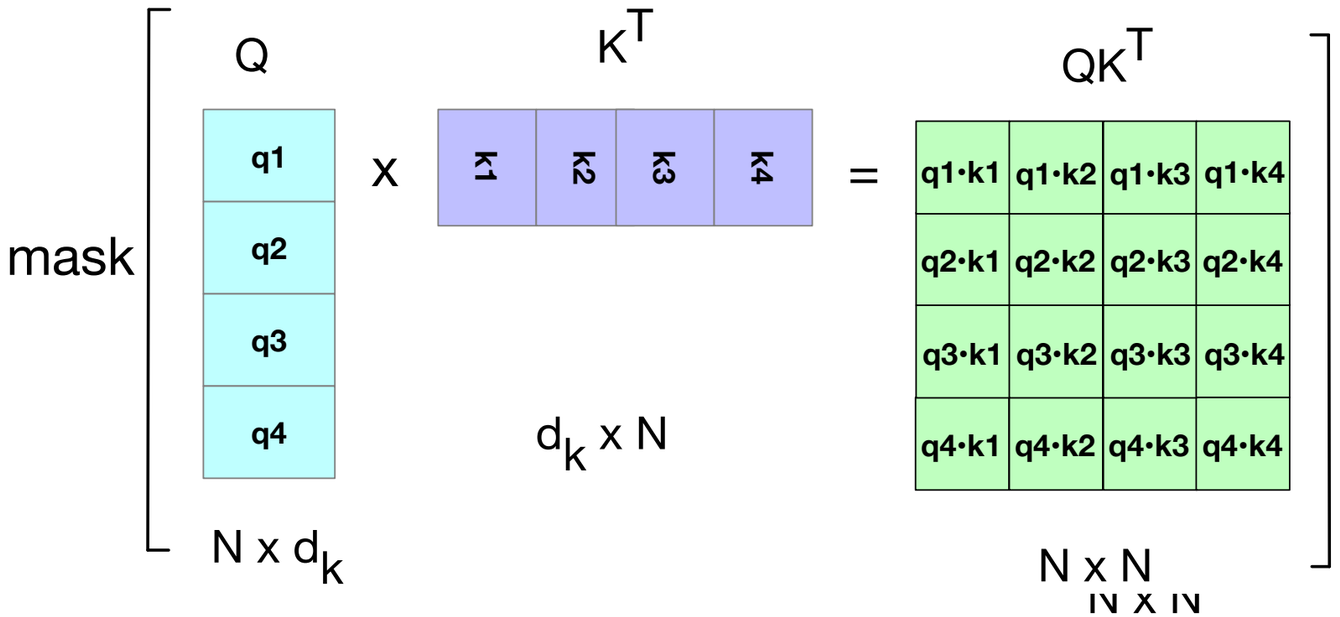
N	q1·k1	−∞	−∞	−∞
	q2·k1	q2·k2	−∞	−∞
	q3·k1	q3·k2	q3·k3	−∞
	q4·k1	q4·k2	q4·k3	q4·k4
		N		

Attention again









Parallelizing Multi-head Attention

$$\mathbf{Q^i} = \mathbf{XW^{Qi}};\ \mathbf{K^i} = \mathbf{XW^{Ki}};\ \mathbf{V^i} = \mathbf{XW^{Vi}}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q^i}, \mathbf{K^i}, \mathbf{V^i}) = \text{softmax}\left(\frac{\mathbf{Q^i K^{i\,T}}}{\sqrt{d_k}}\right)\mathbf{V^i}$$

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_h)\mathbf{W^O}$$

Parallelizing Multi-head Attention

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{MultiHeadAttention}(\mathbf{X}))$$
$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O}))$$

or

$$\mathbf{T}^1 = \text{MultiHeadAttention}(\mathbf{X})$$
$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$
$$\mathbf{T}^3 = \text{LayerNorm}(\mathbf{T}^2)$$
$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$
$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$
$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

Transformers

Parallelizing Attention Computation

Transformers

Input and output: Position embeddings and the Language Model Head

Token and Position Embeddings

The matrix X (of shape [$N \times d$]) has an embedding for each word in the context.

This embedding is created by adding two distinct embedding for each input

- token embedding
- positional embedding

Token Embeddings

Embedding matrix E has shape [$|V| \times d$].

- One row for each of the $|V|$ tokens in the vocabulary.
- Each word is a row vector of d dimensions

Given: string "*Thanks for all the*"

1. Tokenize with BPE and convert into vocab indices

w = [5,4000,10532,2224]

2. Select the corresponding rows from E, each row an embedding

- (row 5, row 4000, row 10532, row 2224).

Position Embeddings

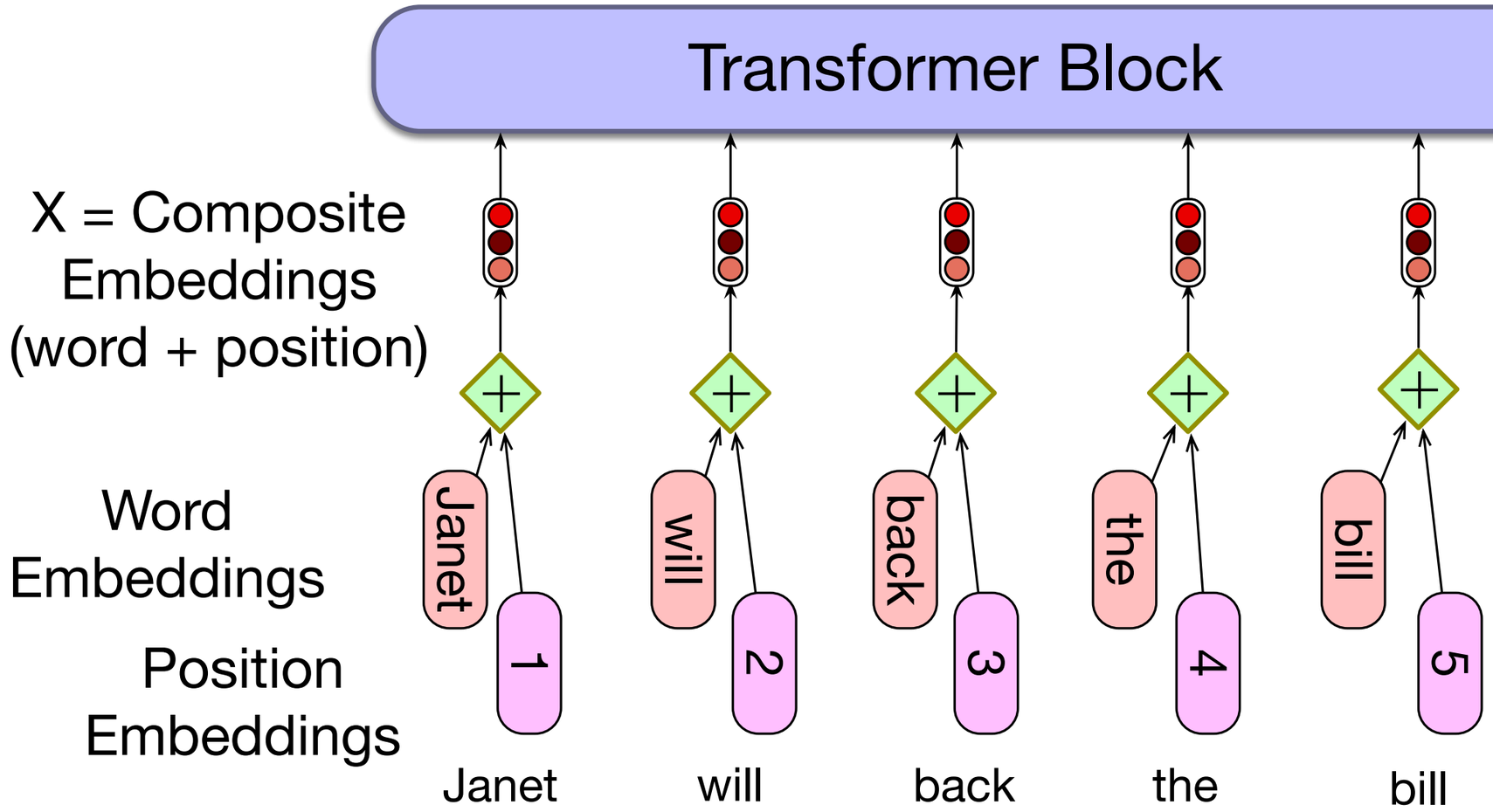
There are many methods, but we'll just describe the simplest: absolute position.

Goal: learn a position embedding matrix E_{pos} of shape $[1 \times N]$.

Start with randomly initialized embeddings

- one for each integer up to some maximum length.
- i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17.
- As with word embeddings, these position embeddings are learned along with other parameters during training.

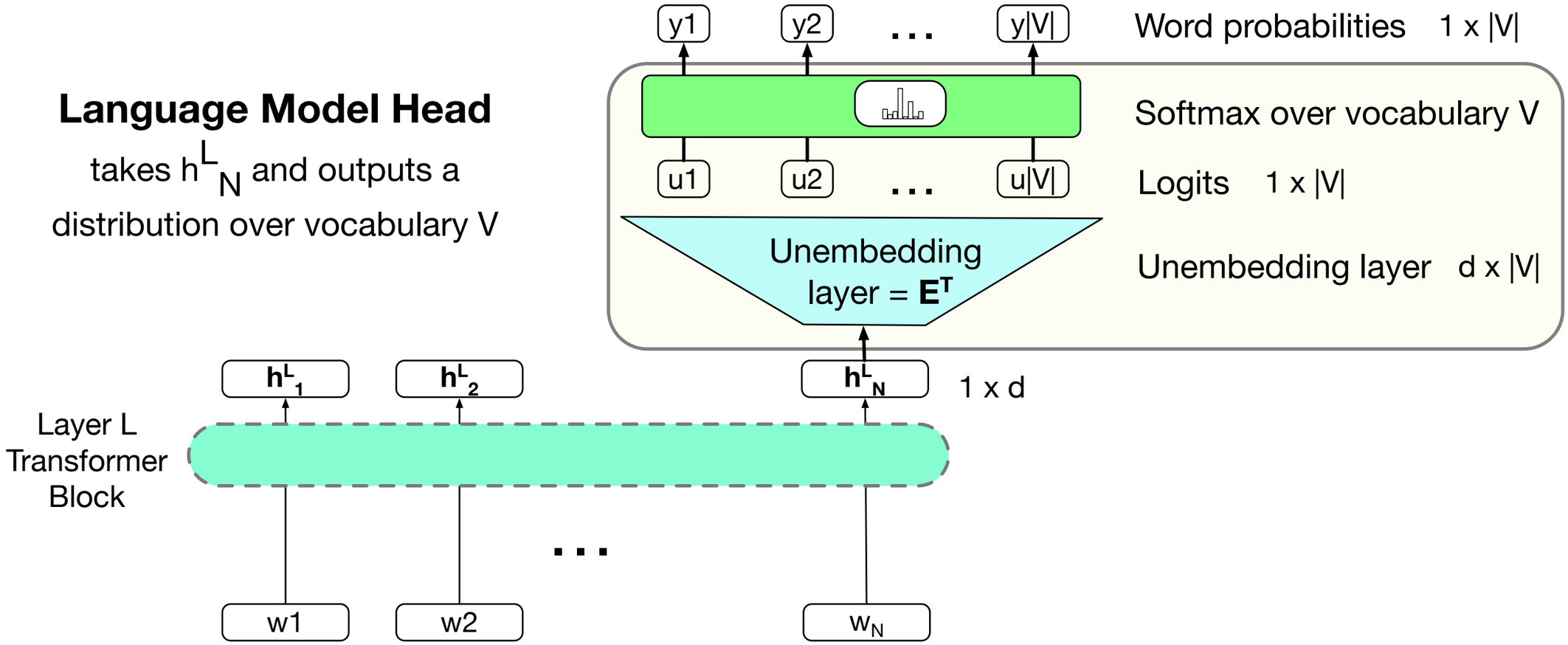
Each **x** is just the sum of word and position embeddings



Language modeling head

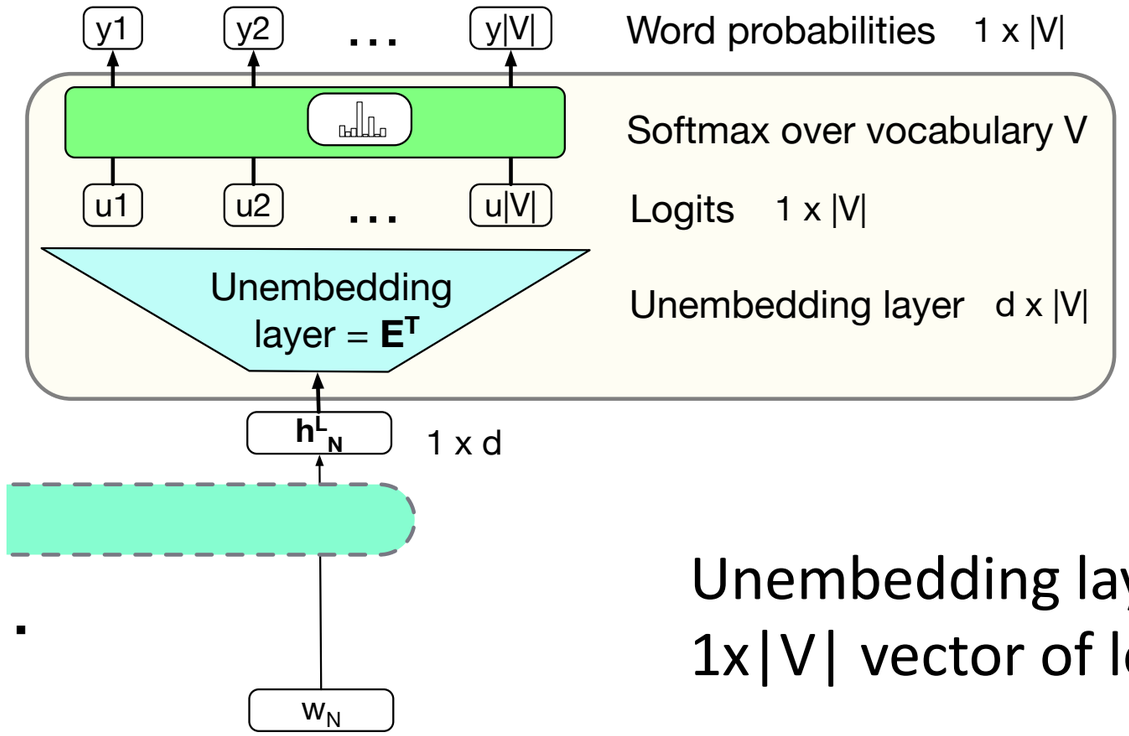
Language Model Head

takes h^L_N and outputs a distribution over vocabulary V



Language modeling head

Unembedding layer: linear layer projects from h^L_N (shape [1 × d]) to logit vector

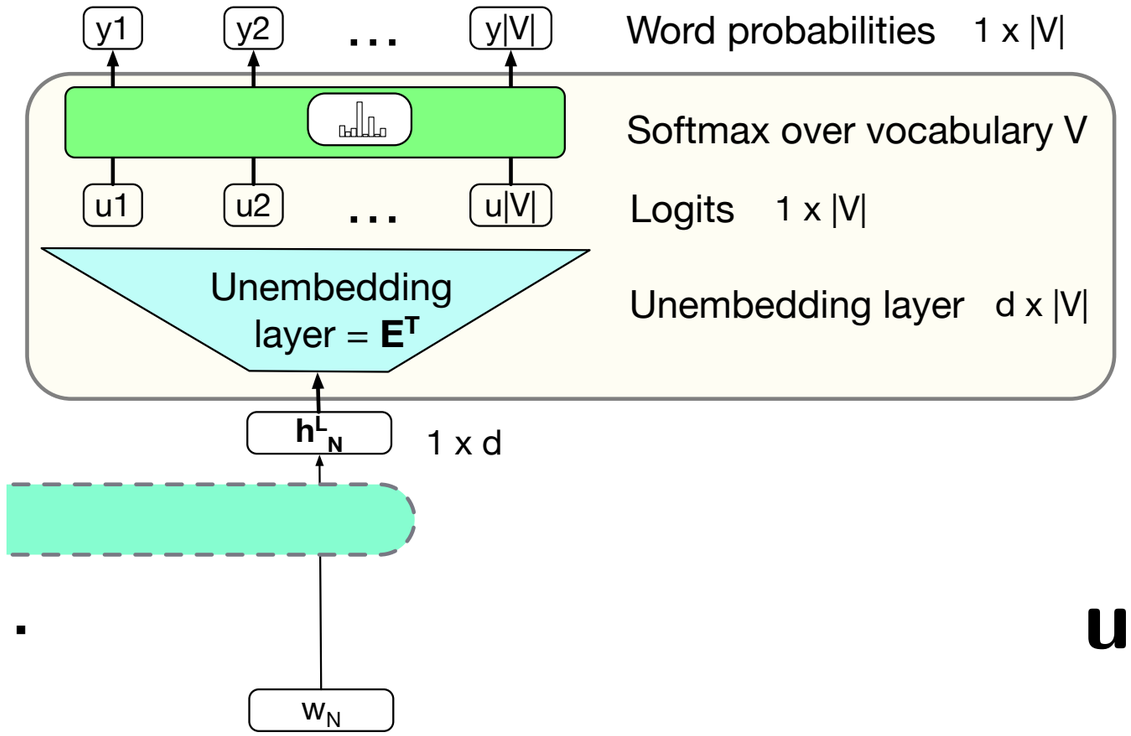


Why "unembedding"? **Tied** to $\mathbf{E}^T$

Weight tying, we use the same weights for two different matrices

Unembedding layer maps from an embedding to a 1x|V| vector of logits

Language modeling head



Logits, the score vector u

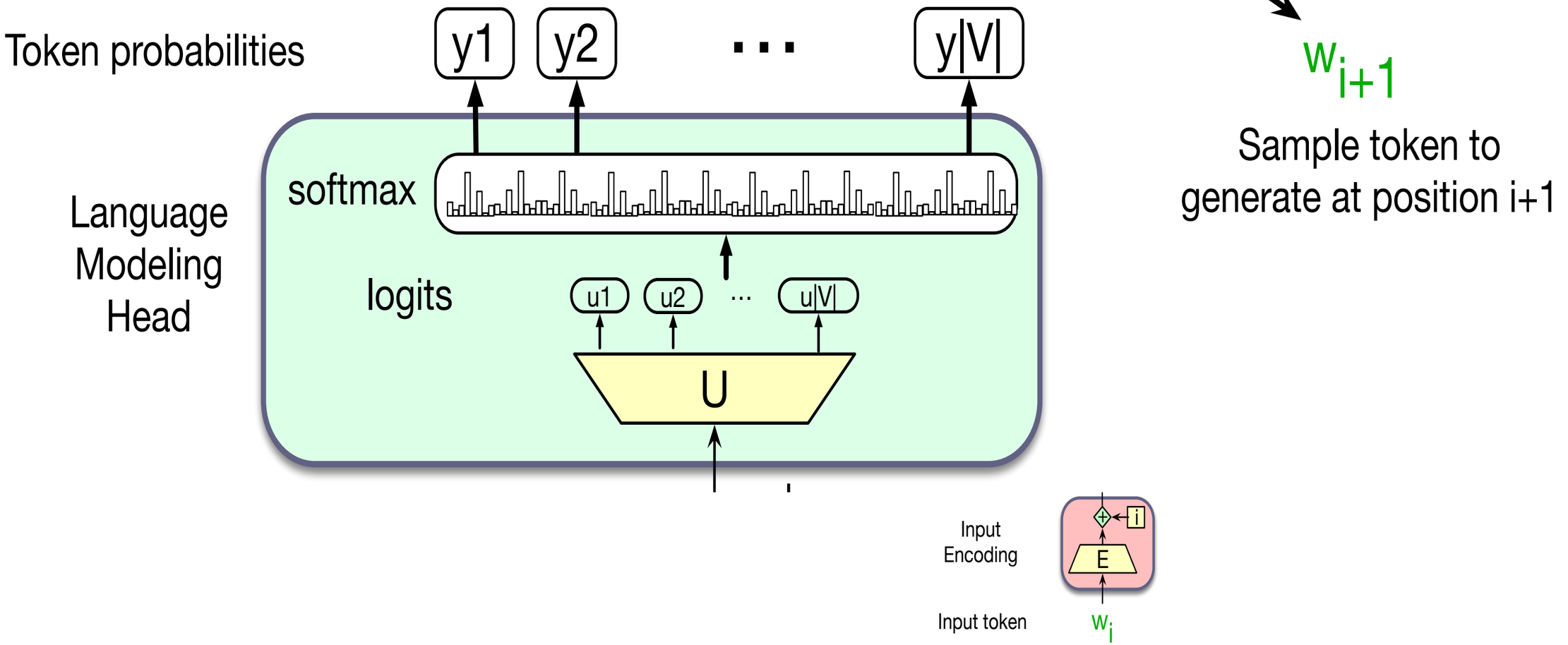
One score for each of the $|V|$ possible words in the vocabulary V. Shape $1 \times |V|$.

Softmax turns the logits into probabilities over vocabulary. Shape $1 \times |V|$.

$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^T$$

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

The final transformer model



Transformers

Input and output: Position embeddings and the Language Model Head

Large Language Models

Dealing with Scale

Scaling Laws

LLM performance depends on

- Model size: the number of parameters not counting embeddings
- Dataset size: the amount of training data
- Compute: Amount of compute (in FLOPS or etc

Can improve a model by adding parameters (more layers, wider contexts), more data, or training for more iterations

The performance of a large language model (the loss) scales as a power-law with each of these three

Scaling Laws

Loss *L* as a function of # parameters *N*, dataset size *D*, compute budget *C (*if other two are held constant)

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}$$

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

Scaling laws can be used early in training to predict what the loss would be if we were to add more data or increase model size.

Number of non-embedding parameters N

$$
\begin{aligned}
N &\approx 2\, d\, n_{\text{layer}}(2\, d_{\text{attn}} + d_{\text{ff}}) \\
&\approx 12\, n_{\text{layer}}\, d^2 \\
&\left(\text{assuming } \; d_{\text{attn}} = d_{\text{ff}}/4 = d\right)
\end{aligned}
$$

Thus GPT-3, with n = 96 layers and dimensionality d = 12288, has 12 × 96 × 122882 ≈ 175 billion parameters.

KV Cache

In training, we can compute attention very efficiently in parallel:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\mathbf{V}$$

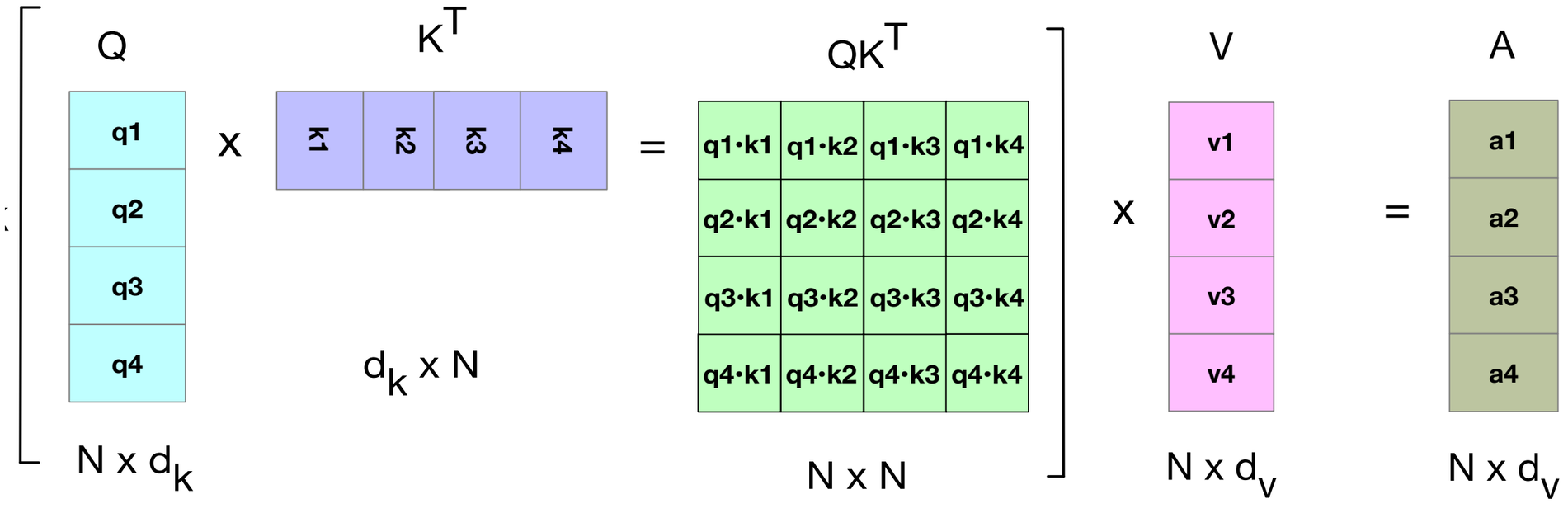
But not at inference! We generate the next tokens **one at a time!**

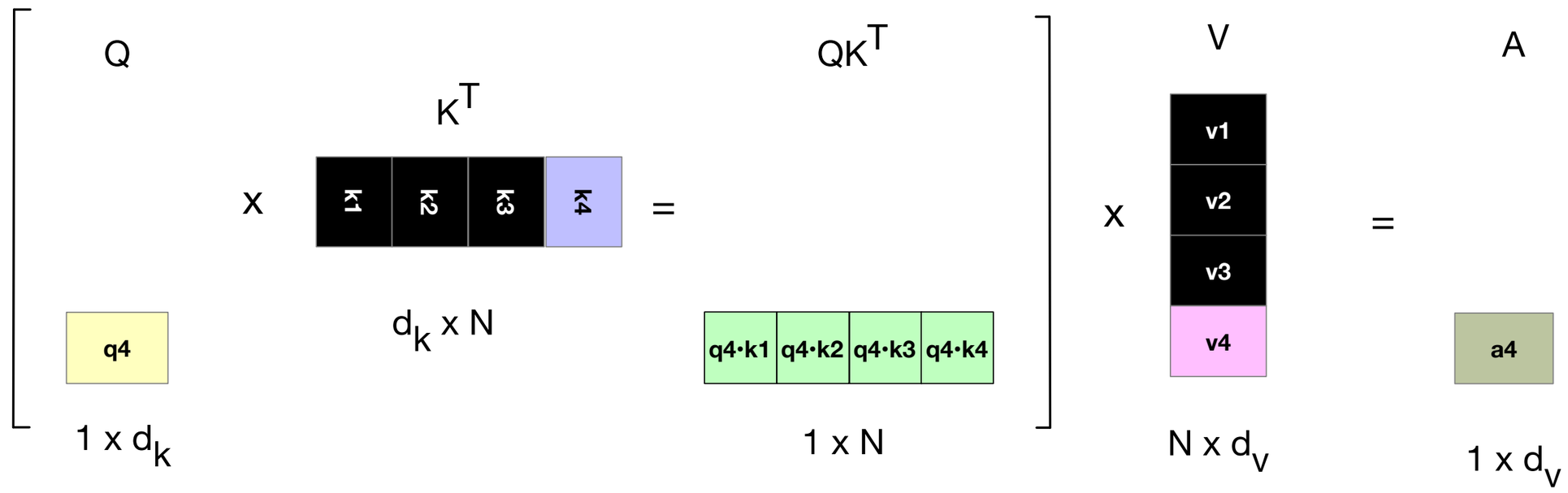
For a new token x, need to multiply by W^Q, W^K, and W^V to get query, key, values

But don't want to **recompute** the key and value vectors for all the prior tokens $x_{<i}$

Instead, store key and value vectors in memory in the KV cache, and then we can just grab them from the cache

KV Cache


Q
K^T
QK^T
V
A
q1
q2
q3
q4
x
k1
k2
k3
k4
=
q1·k1
q1·k2
q1·k3
q1·k4
q2·k1
q2·k2
q2·k3
q2·k4
q3·k1
q3·k2
q3·k3
q3·k4
q4·k1
q4·k2
q4·k3
q4·k4
v1
v2
v3
v4
a1
a2
a3
a4
N x d_k
d_k x N
N x N
N x d_v
N x d_v


Q
K^T
QK^T
V
A
q4
x
k1
k2
k3
k4
=
q4·k1
q4·k2
q4·k3
q4·k4
v1
v2
v3
v4
a4
1 x d_k
d_k x N
1 x N
N x d_v
1 x d_v

Parameter-Efficient Finetuning

Adapting to a new domain by continued pretraining (finetuning) is a problem with huge LLMs.

- Enormous numbers of parameters to train
- Each pass of batch gradient descent has to backpropagate through many many huge layers.
- Expensive in processing power, in memory, and in time.

Instead, **parameter-efficient fine tuning** (PEFT)

- Efficiently select a subset of parameters to update when finetuning.
- E.g., freeze some of the parameters (don’t change them),
- And only update some a few parameters.

LoRA (Low-Rank Adaptation)

- Trransformers have many dense matrix multiply layers
  - Like $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$, $\mathbf{W}^O$ layers in attention
- Instead of updating these layers during finetuning,
  - Freeze these layers
  - Update a low-rank approximation with fewer parameters.

LoRA

- Consider a **matrix W** (shape [$N \times d$]) that needs to be updated during finetuning via gradient descent.
  - Normally updates are ΔW (shape [$N \times d$])
- In LoRA, we freeze W and update instead a low-rank decomposition of W:
  - A of shape [$N \times r$],
  - B of shape [$r \times d$], r is very small (like 1 or 2)
  - That is, during finetuning we update A and B instead of W.
  - Replace $W + \Delta W$ with $W + BA$.

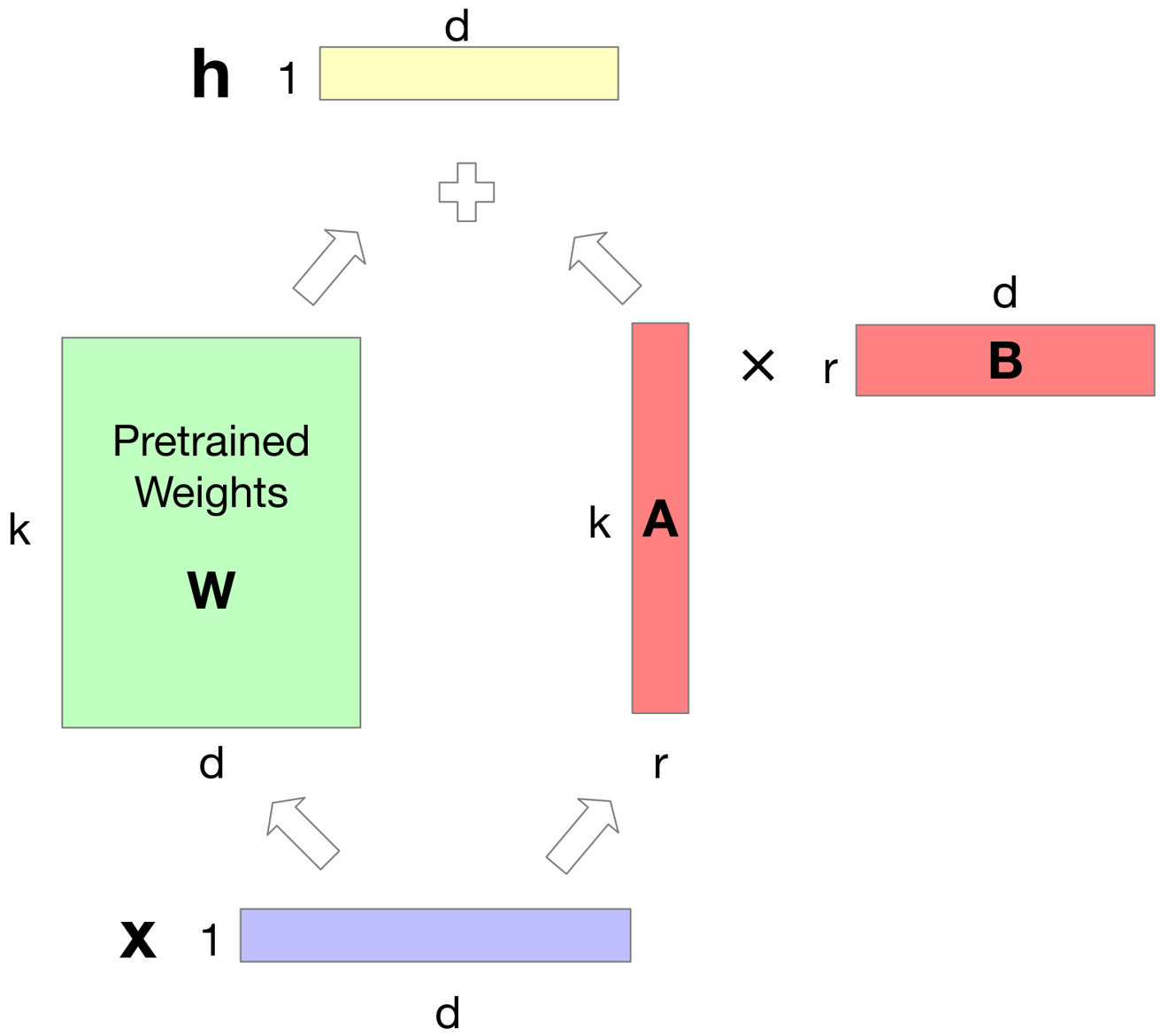
Forward pass: instead of

$$\mathbf{h} = \mathbf{xW}$$

We do

$$\mathbf{h} = \mathbf{xW} + \mathbf{xAB}$$

LoRA



Large Language Models

Dealing with Scale

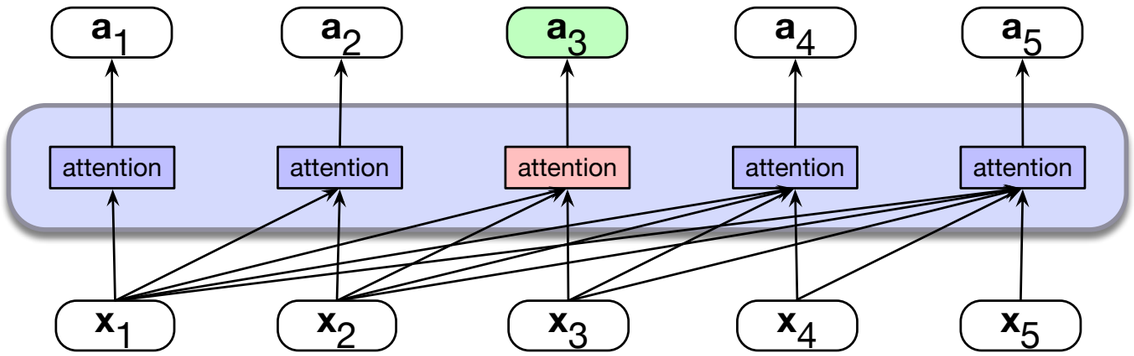
Masked Language Models

BERT

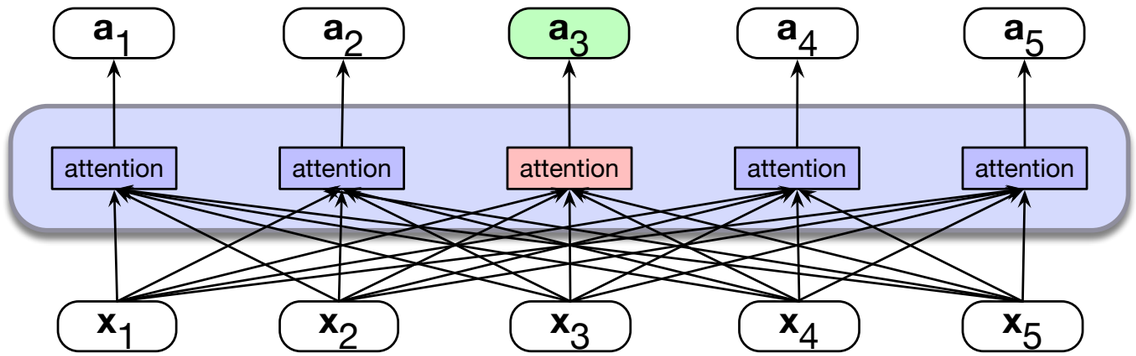
Masked Language Modeling

- We've seen autoregressive (causal, left-to-right) LMs.
- But what about tasks for which we want to peak at future tokens?
  - Especially true for tasks where we map each input token to an output token
- **Bidirectional** encoders use **masked self-attention** to
  - map sequences of input embeddings (x1,...,x*n*)
  - to sequences of output embeddings of the same length (h1,...,h*n*),
  - where the output vectors have been contextualized using information from the entire input sequence.

Bidirectional Self-Attention



a) A causal self-attention layer



b) A bidirectional self-attention layer

Easy! We just remove the mask

Casual self-attention

N

q1·k1	−∞	−∞	−∞
q2·k1	q2·k2	−∞	−∞
q3·k1	q3·k2	q3·k3	−∞
q4·k1	q4·k2	q4·k3	q4·k4

N

$$\mathbf{head} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

Bidirectional self-attention

N

q1·k1	q1·k2	q1·k3	q1·k4
q2·k1	q2·k2	q2·k3	q2·k4
q3·k1	q3·k2	q3·k3	q3·k4
q4·k1	q4·k2	q4·k3	q4·k4

N

$$\mathbf{head} = \text{softmax}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\mathbf{V}$$

BERT: Bidirectional Encoder Representations from Transformers

BERT (Devlin et al., 2019)

- 30,000 English-only tokens (WordPiece tokenizer)
- Input context window *N*=512 tokens, and model dimensionality *d*=768
- *L*=12 layers of transformer blocks, each with *A*=12 (bidirectional) multihead-attention layers.
- The resulting model has about 100M parameters.

XLM-RoBERTa (Conneau et al., 2020)

- 250,000 multilingual tokens (SentencePiece Unigram LM tokenizer)
- Input context window *N*=512 tokens,model dimensionality *d*=1024
- *L*=24 layers of transformer blocks, with *A*=16 multihead attention layers each
- The resulting model has about 550M parameters.

Masked Language Models

BERT

Masked Language Models

Masked LM training

Masked training intuition

- For **left-to-right LMs**, the model tries to predict the last word from prior words:

  The water of Walden Pond is so beautifully ____

  - And we train it to improve its predictions.

- For **bidirectional masked LMs**, the model tries to predict one or more words from all the rest of the words:

  The ____ of Walden Pond ____ so beautifully blue

  - The model generates a probability distribution over the vocabulary for each missing token
  - We use the cross-entropy loss from each of the model's predictions to drive the learning process.

MLM training in BERT

15% of the tokens are randomly chosen to be part of the masking

Example: "Lunch was **delicious**", if delicious was randomly chosen:

Three possibilities:

1. 80%: Token is replaced with special token [MASK]

   Lunch was **delicious ->** Lunch was **[MASK]**

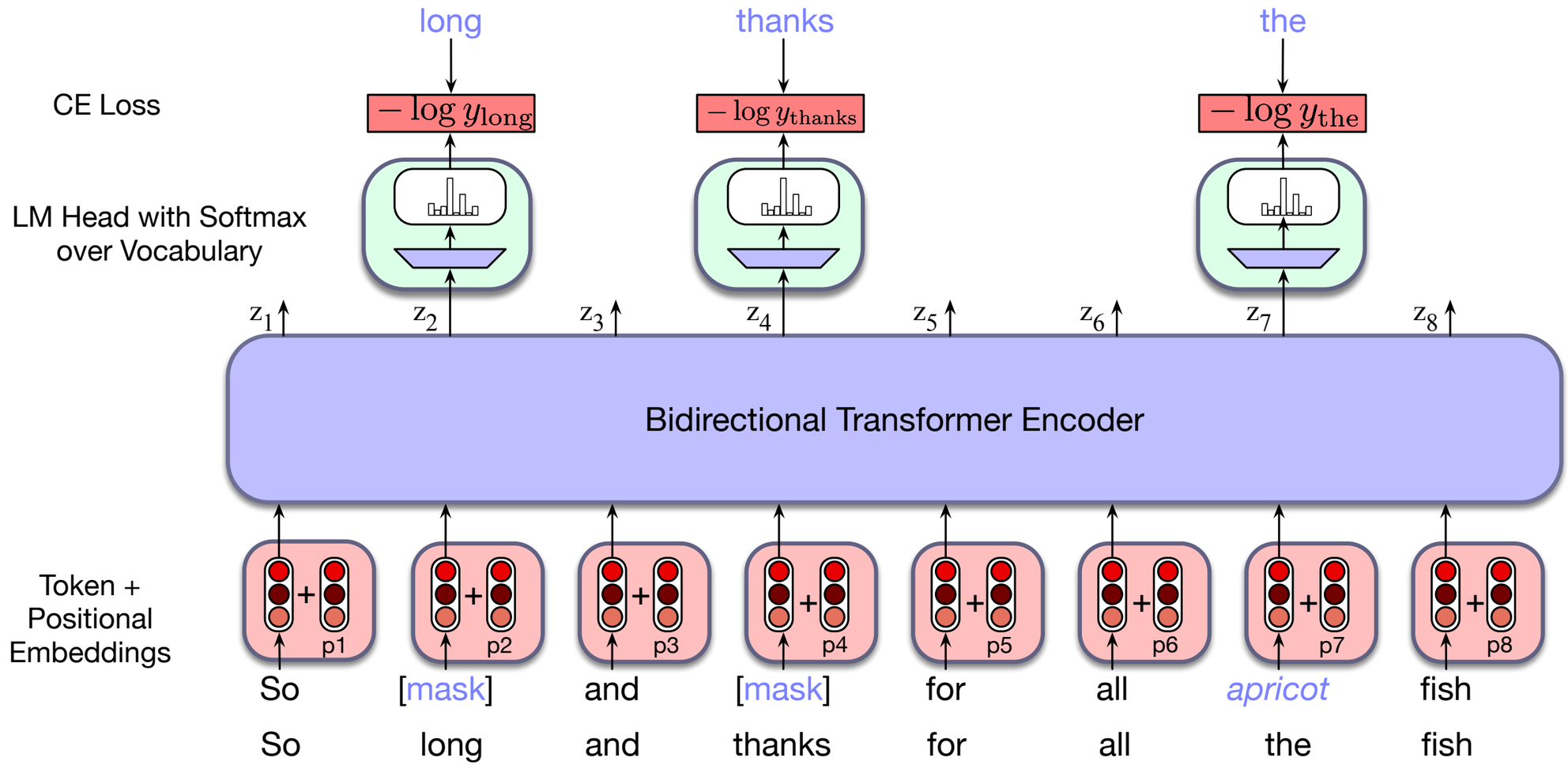
2. 10%: Token is replaced with a random token (sampled from unigram prob)

   Lunch was **delicious ->** Lunch was **gasp**

3. 10%: Token is unchanged

   Lunch was **delicious ->** Lunch was **delicious**

In detail


CE Loss
LM Head with Softmax over Vocabulary
Token + Positional Embeddings
long
thanks
the
$-\log y_{\text{long}}$
$-\log y_{\text{thanks}}$
$-\log y_{\text{the}}$
z_1 z_2 z_3 z_4 z_5 z_6 z_7 z_8
Bidirectional Transformer Encoder
p1 p2 p3 p4 p5 p6 p7 p8
So [mask] and [mask] for all apricot fish
So long and thanks for all the fish

MLM loss

The LM head takes output of final transformer layer L, multiplies it by unembedding layer and turns into probabilities:

$$\mathbf{u}_i = \mathbf{h}_\mathbf{i}^\mathbf{L}\ \mathbf{E}^\mathbf{T}$$

$$\mathbf{y}_i = \mathrm{softmax}(\mathbf{u_i})$$

E.g., for the x_i corresponding to "long", the loss is the probability of the correct word *long*, given output h^L_i):

$$L_{MLM}(x_i) = -\log P(x_i|\mathbf{h}_i^L)$$

We get the gradients by taking the average of this loss over the batch

$$L_{MLM} = -\frac{1}{|M|}\sum_{i\in M}\log P(x_i|\mathbf{h}_i^L)$$

Next Sentence Prediction

Given 2 sentences the model predicts if they are a real pair of adjacent sentences from the training corpus or a pair of unrelated sentences.

BERT introduces two special tokens

- [CLS] is prepended to the input sentence pair,
- [SEP] is placed between the sentences, and also after second sentence

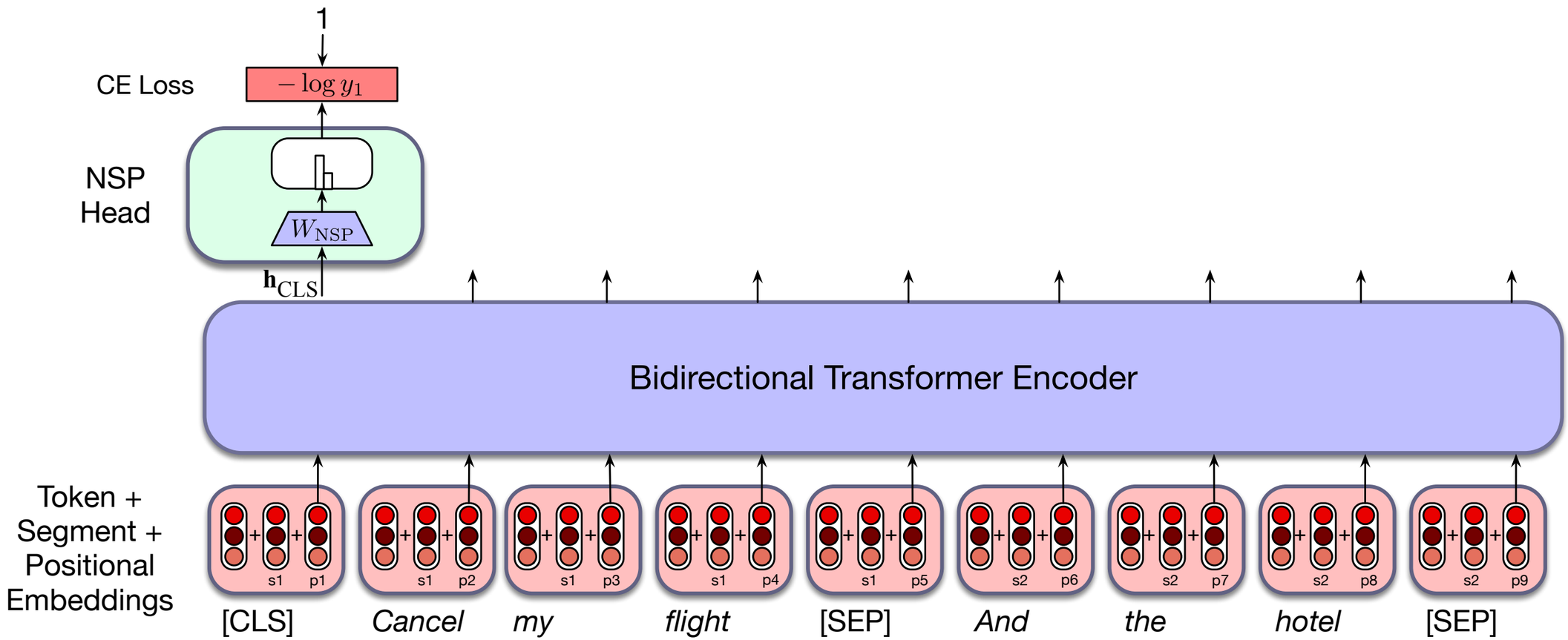
And two more special tokens

- [1st segment] and [2nd segment]
- These are added to the input embedding and positional embedding

h^L_{CLS} from the final layer [CLS] token is input to classifier head (weights W_{NSP}) that predicts two classes:.

$$\mathbf{y}_i = \text{softmax}(\mathbf{h}^L_{\text{CLS}}\mathbf{W}_{\mathbf{NSP}})$$

NSP Loss with classification head



More details

Original model was trained with 40 passes over training data

Some models (like RoBERTa) drop NSP loss

Tokenizer for multilingual models is trained from stratified sample of languages (some data from each language)

Multilingual models are better than monolingual models with small numbers of languages

- With large numbers of languages, monolingual models in that language can be better
- The "curse of multilinguality"

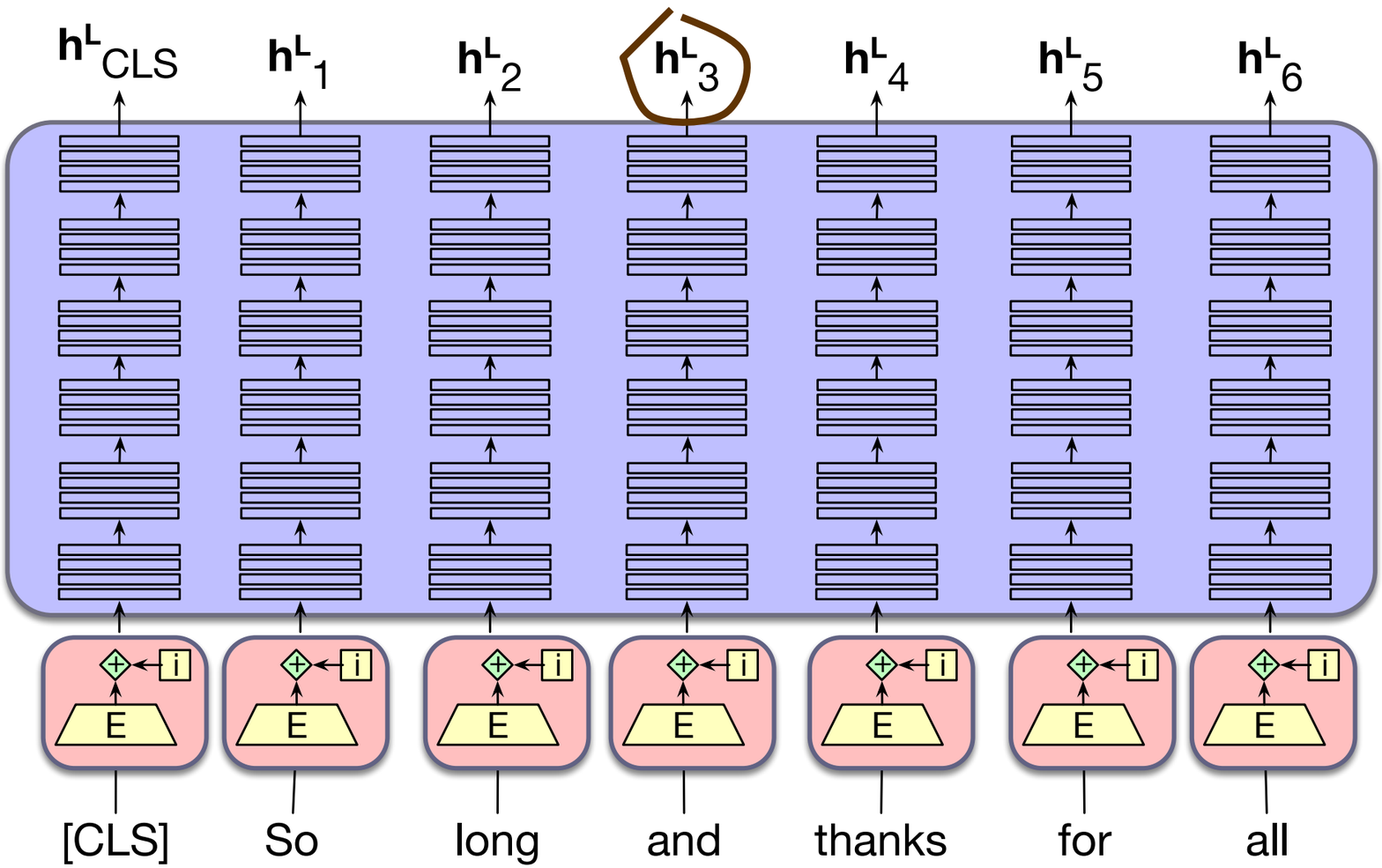
Masked Language Models

Masked LM training

Masked Language Models

Contextual Embeddings

Contextual Embeddings to represent words



Static vs Contextual Embeddings

Static embeddings represent **word types** (dictionary entries)

Contextual embeddings represent **word instances** (one for each time the word occurs in any context/sentence)



Word sense

Words are ambiguous

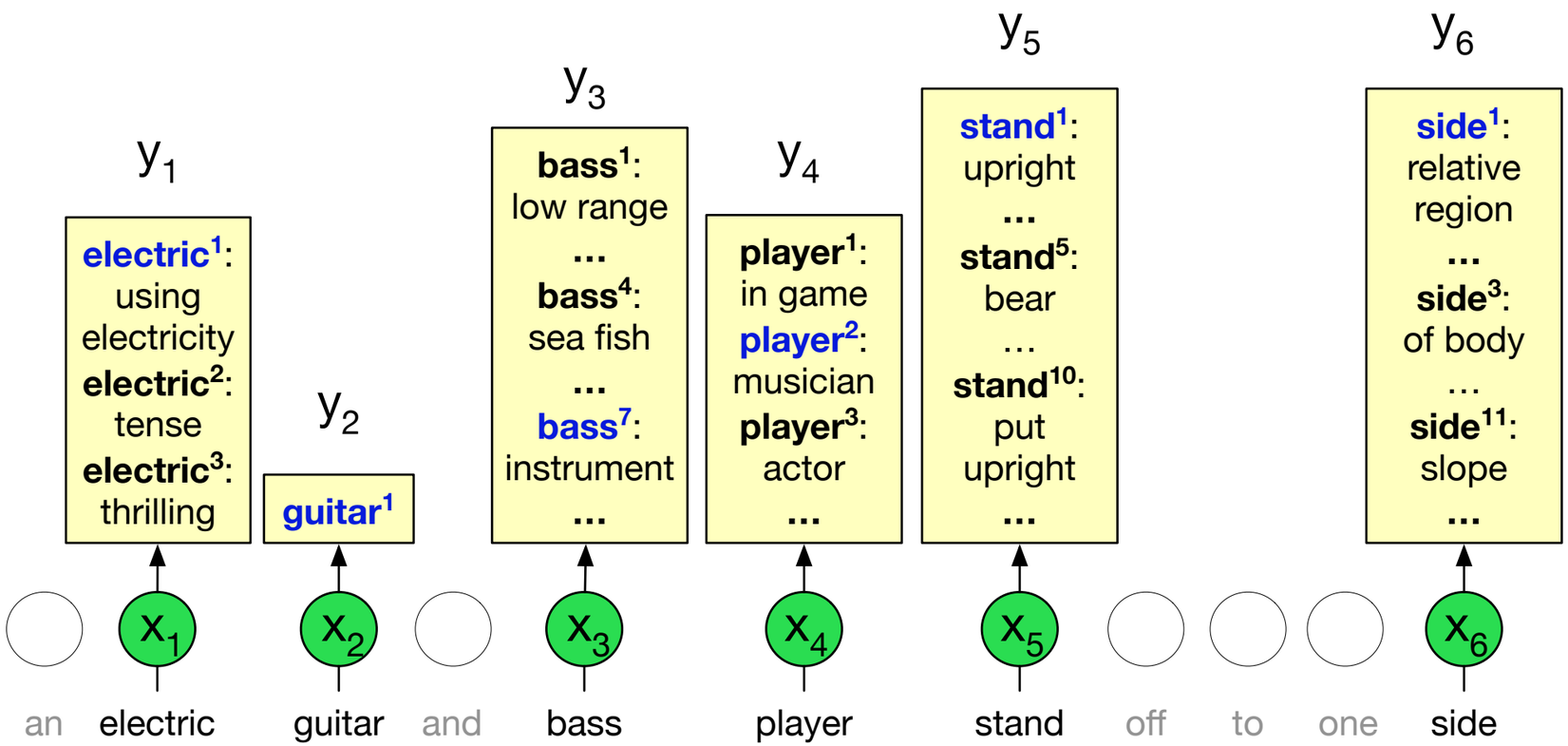
A **word sense** is a discrete representation of one aspect of meaning

mouse^{1} : .... a *mouse* controlling a computer system in 1968.
mouse^{2} : .... a quiet animal like a *mouse*
bank^{1} : ...a *bank* can hold the investments in a custodial account ...
bank^{2} : ...as agriculture burgeons on the east *bank*, the river ...

Contextual embeddings offer a continuous high-dimensional model of meaning that is more fine grained than discrete senses.

Word sense disambiguation (WSD)

The task of selecting the correct sense for a word.



1-nearest neighbor algorithm for WSD

Melamud et al (2016), Peters et al (2018)

At training time, take a sense-labeled corpus like SEMCOR

Run corpus through BERT to get contextual embedding for each token

- E.g., pooling representations from last 4 BERT transformer layer

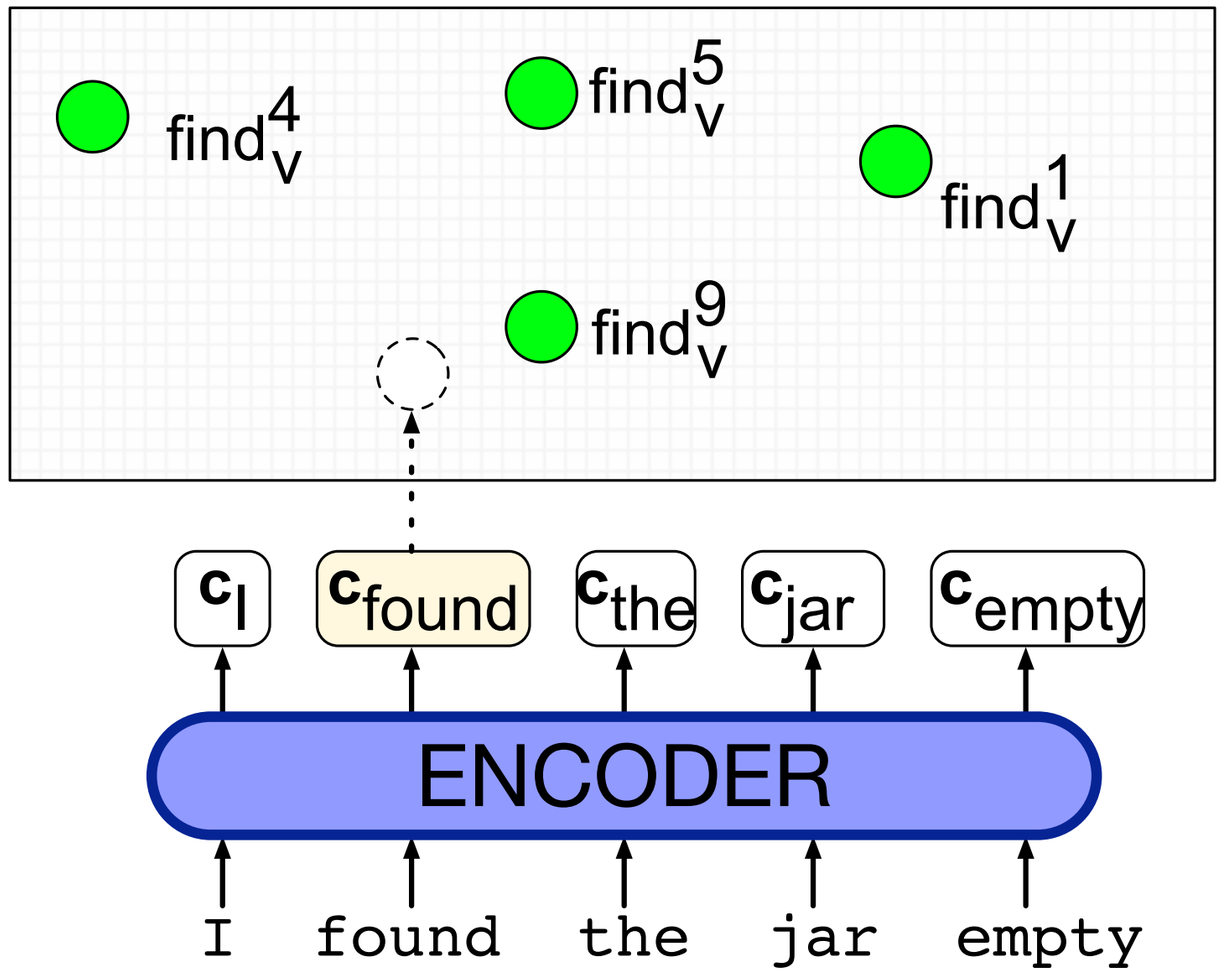
Then for each sense s of word w for n tokens of that sense, pool embeddings:

$$\mathbf{v}_s = \frac{1}{n}\sum_i \mathbf{v}_i \qquad \forall \mathbf{v}_i \in \text{tokens}(s)$$

At test time, given a token of a target word *t*, compute contextual embedding t and choose its nearest neighbor sense from training set

$$\text{sense}(t) = \underset{s \in \text{senses}(t)}{\text{argmax}}\ \text{cosine}(\mathbf{t}, \mathbf{v}_s)$$

1-nearest neighbor algorithm for WSD



Similarity and contextual embeddings

- We generally use cosine as for static embeddings
- But some issues:
  - Contextual embeddings tend to be **anisotropic:** all point in roughly the same direction so have high inherent cosines (Ethayarajh 2019)
  - Cosine measure are dominated by a small number of "rogue" dimensions with very high values (Timkey and van Schijndel 2021)
  - Cosine tends to underestimate human judgments on similarity of word meaning for very frequent words (Zhou et al., 2022)

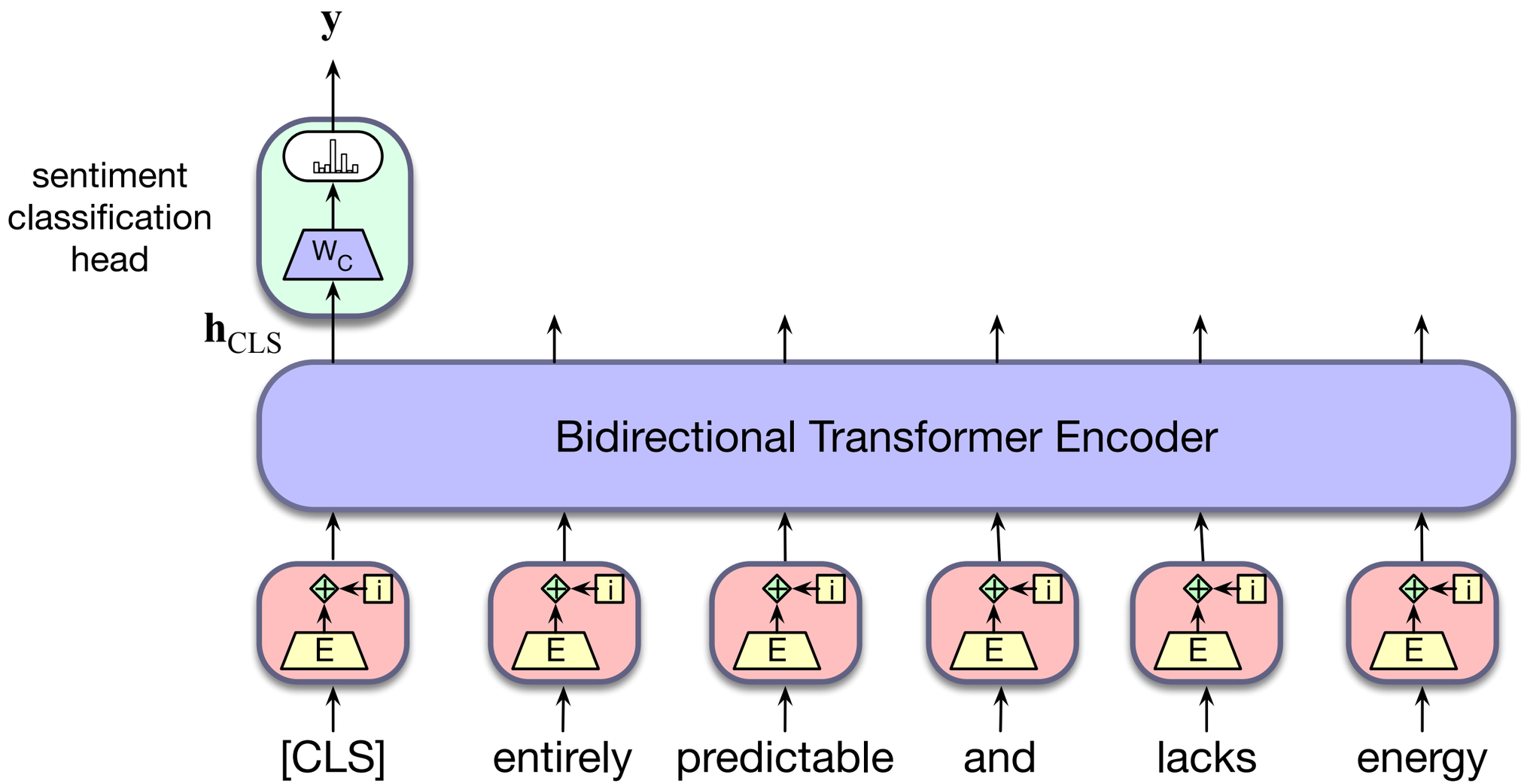
Masked Language Models

Contextual Embeddings

Masked Language Models

Fine-Tuning for Classification

Adding a sentiment classification head



Sequence-Pair classification

Assign a label to pairs of sentences:

- paraphrase detection (are the two sentences paraphrases of each other?)
- logical entailment (does sentence A logically entail sentence B?)
- discourse coherence (how coherent is sentence B as a follow-on to sentence A?)

Example: Natural Language Inference

Pairs of sentences are given one of 3 labels

- `Neutral`
  - a: Jon walked back to the town to the smithy.
  - b: Jon traveled back to his hometown.
- `Contradicts`
  - a: Tourist Information offices can be very helpful.
  - b: Tourist Information offices are never of any help.
- `Entails`
  - a: I'm confused.
  - b: Not all of it is very clear to me.

Algorithm: pass the premise/hypothesis pairs through a bidirectional encoder and use the output vector for the [CLS] token as the input to the classification head .

Fine-tuning for sequence labeling

Assign a label from a small fixed set of labels to each token in the sequence.

- Named entity recognition
- Part of speech tagging
-

Named Entity Recognition

A **named entity** is anything that can be referred to with a proper name: a person, a location, an organization

Named entity recognition (NER): find spans of text that constitute proper names and tag the type of the entity

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	**Turing** is a giant of computer science.
Organization	ORG	companies, sports teams	The **IPCC** warned about the cyclone.
Location	LOC	regions, mountains, seas	**Mt. Sanitas** is in **Sunshine Canyon**.
Geo-Political Entity	GPE	countries, states	**Palo Alto** is raising the fees for parking.

Named Entity Recognition

Citing high fuel prices, [$_{\text{ORG}}$ **United Airlines**] said [$_{\text{TIME}}$ **Friday**] it has increased fares by [$_{\text{MONEY}}$ **$6**] per round trip on flights to some cities also served by lower-cost carriers. [$_{\text{ORG}}$ **American Airlines**], a unit of [$_{\text{ORG}}$ **AMR Corp.**], immediately matched the move, spokesman [$_{\text{PER}}$ **Tim Wagner**] said. [$_{\text{ORG}}$ **United**], a unit of [$_{\text{ORG}}$ **UAL Corp.**], said the increase took effect [$_{\text{TIME}}$ **Thursday**] and applies to most routes where it competes against discount carriers, such as [$_{\text{LOC}}$ **Chicago**] to [$_{\text{LOC}}$ **Dallas**] and [$_{\text{LOC}}$ **Denver**] to [$_{\text{LOC}}$ **San Francisco**].

BIO Tagging

Ramshaw and Marcus (1995)

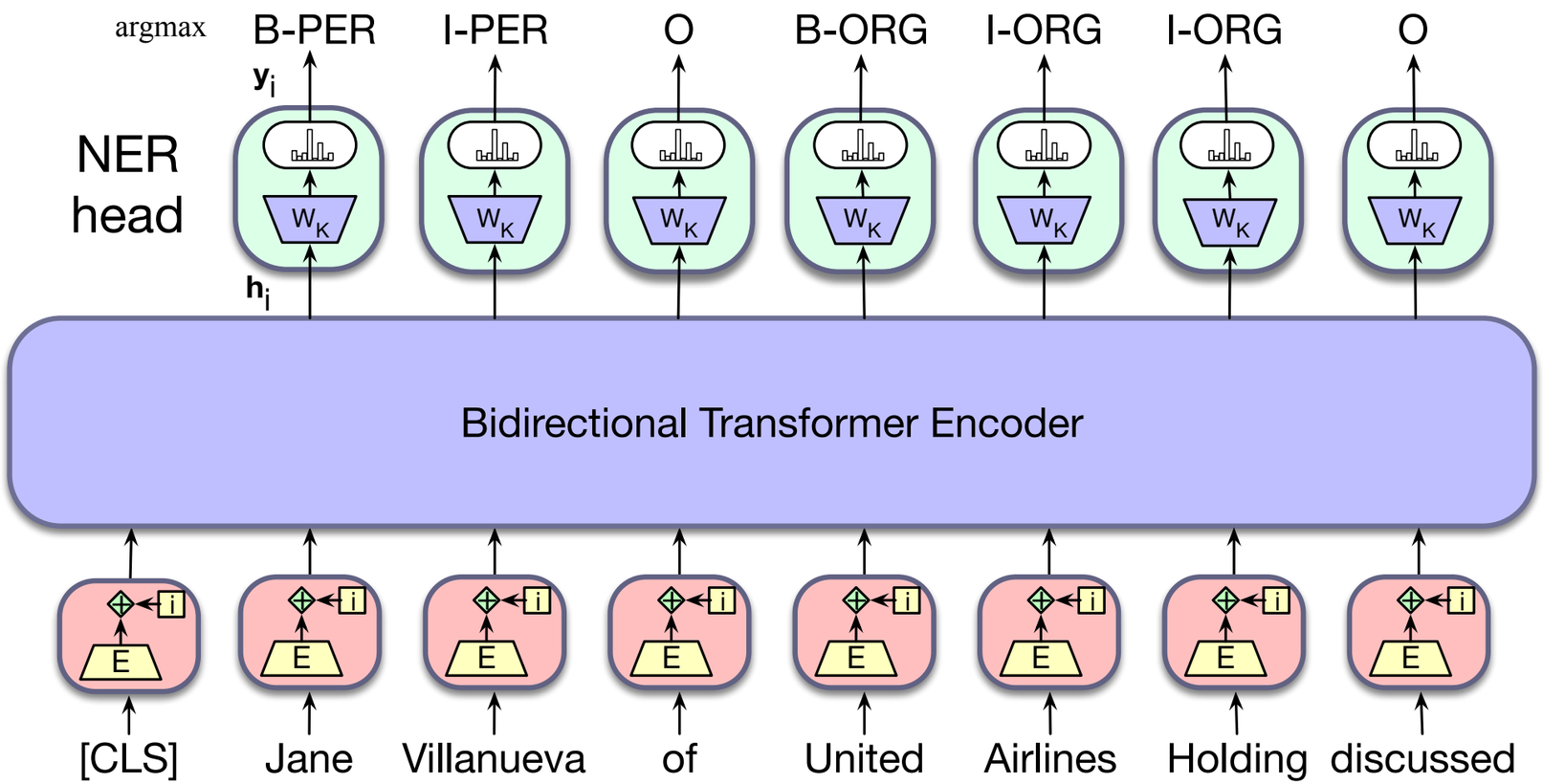
A method that lets us turn a segmentation task (finding boundaries of entities) into a classification task

[PER **Jane Villanueva**] of [ORG **United**] , a unit of [ORG **United Airlines Holding**] , said the fare applies to the [LOC **Chicago**] route.

Words	**IO Label**	**BIO Label**	**BIOES Label**
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Sequence labeling

$$\mathbf{y_i} = \text{softmax}(\mathbf{h_i^L W_K})$$
$$\mathbf{t_i} = \text{argmax}_k(\mathbf{y}_i)$$



More details

We need to map between tokens (used by LLM) and words (used in definition of name entities)

We evaluate NER with F1 (precision/recall)

Masked Language Models

Fine-Tuning for Classification

Information Retrieval and RAG

The Information Retrieval Task

Information retrieval

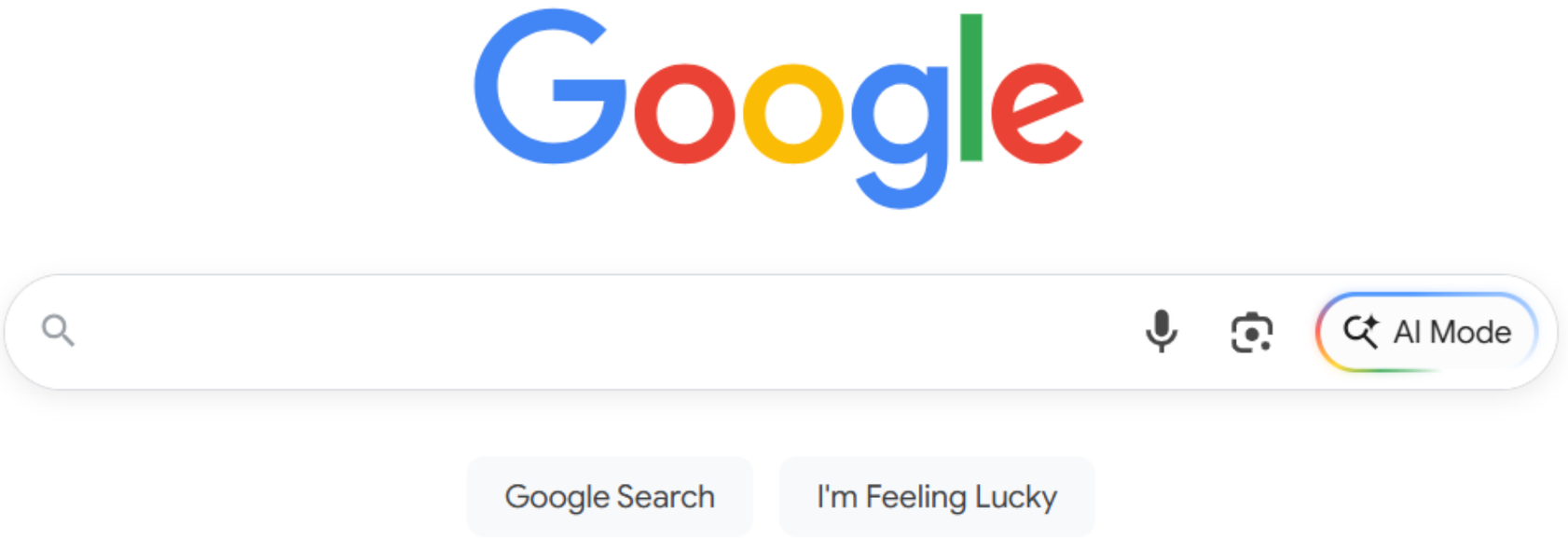
User has an information need

And has some collection of documents

User wants to find a **relevant** document

- a document (or documents)
- in the collection
- that satisfy their need

Web search



Not just the web

Searching our email

Searching corporate documents

Searching personal medical records

And also, part of LLMs

- retrieval-augmented generation (RAG)

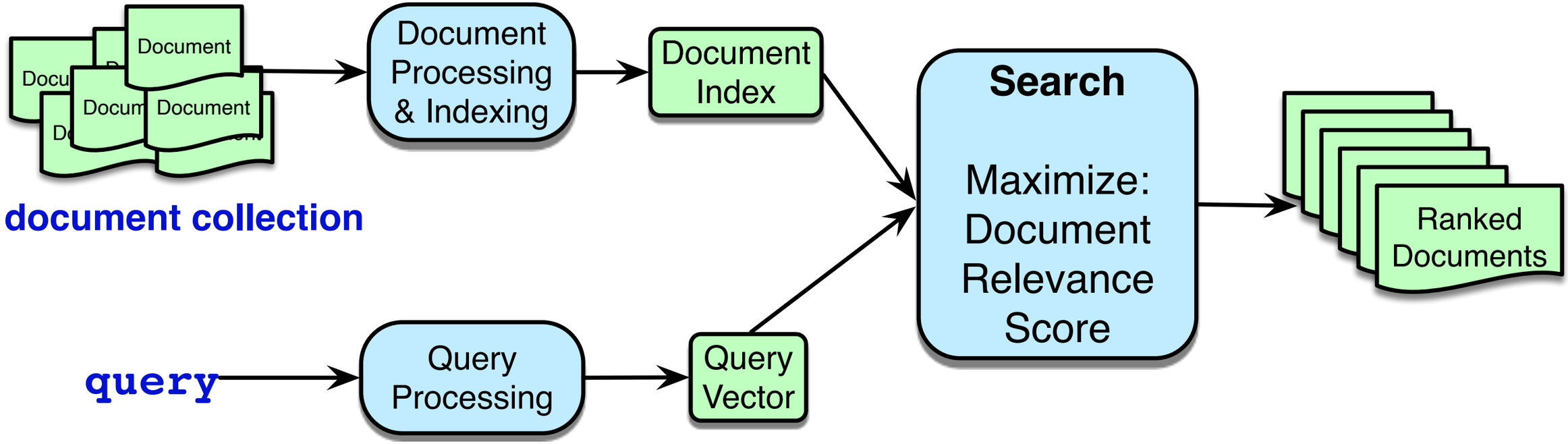
In most cases we do "ranked retrieval"

The retriever returns top-k documents

These are ranked

We can show the user these, or some subset.

Ad-hoc retrieval



Document Relevance Score

Goal is to assign a score to each document for whether it meets the user's information need

Instead, we just approximate this by the textual similarity between the query and the document.

Two architectures

Sparse retrieval

- represent query and doc as vectors of word counts
- weighted by tf-idf, BM25

Dense retrieval

- Use LLM to represent query and doc as embeddings

In both cases, similarity is dot product or cosine between query and document representations

Information Retrieval and RAG

The Information Retrieval Task

Information Retrieval and RAG

Sparse retrieval: the vector model of IR

The vector space model of IR

Gerard Salton, 1971

Represent a document as a vector of counts of the words it contains.

Bag-of-words model

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

Vector representation of that doc

[1 3 1 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1]

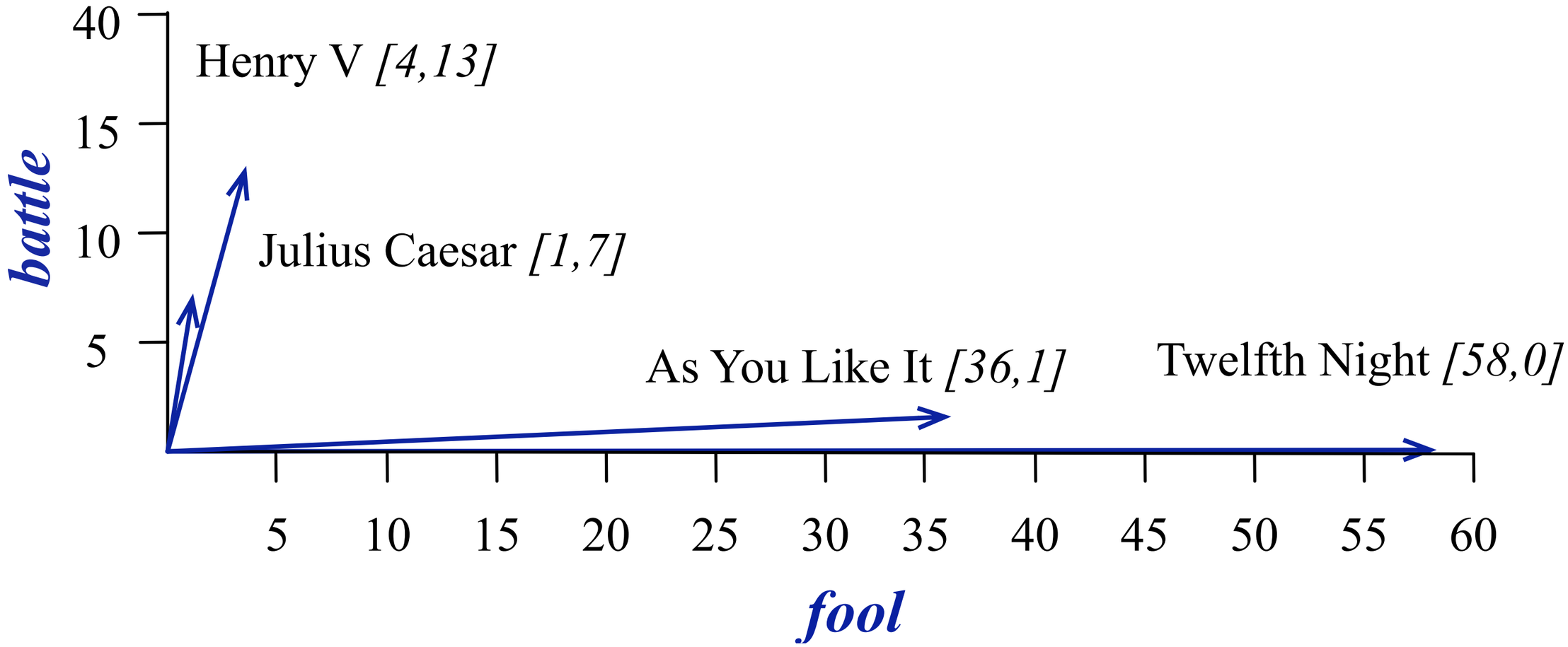
Term-document matrix

Each document is represented by a vector of words

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Visualizing document vectors

The two dimensional space [battle, fool]



Vectors are the basis of information retrieval

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Vectors are similar for the two comedies

But comedies are different than the other two

Comedies have more *fools* and *wit* and fewer *battles*.

Vector representations of queries and documents

Suppose we are looking for a witty fool play:

`Query = "fool wit"`

	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

Choose the document that is most similar to the query

Which of $\mathbf{d_1}$, $\mathbf{d_2}$, $\mathbf{d_3}$, $\mathbf{d_4}$ is most similar to **q**?

d_1 d_2 d_3 d_4 q

	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

Similarity methods are variants of dot product

The dot product is **q · d**

score (q,d_1) = q · d_1 =

d_1 d_2 d_3 d_4 q

	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

In fact we use cosine

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|}$$

In fact we use cosine

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|}$$

$$= \frac{\mathbf{q}}{|\mathbf{q}|} \cdot \frac{\mathbf{d}}{|\mathbf{d}|}$$

Information Retrieval and RAG

Sparse retrieval: the vector model of IR

Information Retrieval and RAG

TF-IDF

But raw frequency is a bad representation

- The co-occurrence matrices we have seen represent each cell by word frequencies.
- Frequency is clearly useful; if *sugar* appears a lot near *apricot*, that's useful information.
- But overly frequent words like *the*, *it*, or *they* are not very informative about the context
- It's a paradox! How can we balance these two conflicting constraints?

Two common solutions for word weighting

tf-idf: tf-idf value for word t in document d:

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

Words like "the" or "it" have very low idf

PMI: (Pointwise mutual information)

- $\mathrm{PMI}(w_1, w_2) = log \frac{p(w_1,w_2)}{p(w_1)p(w_2)}$

See if words like "good" appear more often with "great" than we would expect by chance

Term frequency (tf) in the tf-idf algorithm

We could imagine using raw count:

$$\mathrm{tf}_{t,d} = \mathrm{count}(t,d)$$

But instead of using raw count, we usually squash a bit:

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t,d) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Document frequency (df)

df_t *is* the number of documents *t* occurs in.

(note this is not collection frequency: total count across all documents)

"*Romeo*" is very distinctive for one Shakespeare play:

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

Inverse document frequency (idf)

$$\text{idf}_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

N is the total number of documents in the collection

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

What is a document?

Could be a play or a Wikipedia article

But for the purposes of tf-idf, documents can be **anything**; we often call each paragraph a document!

Final tf-idf weighted value for a word

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

Raw counts:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

tf-idf:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

Information Retrieval and RAG

TF-IDF

Information Retrieval and RAG

TF-IDF: a worked example

Cosine

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|}$$

$$= \frac{\mathbf{q}}{|\mathbf{q}|} \cdot \frac{\mathbf{d}}{|\mathbf{d}|}$$

TF-IDF weighted cosine

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}}{|\mathbf{q}|} \cdot \frac{\mathbf{d}}{|\mathbf{d}|}$$

$$= \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

TF-IDF weighted cosine

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t,d) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\mathrm{idf}_t = \log_{10}\left(\frac{N}{\mathrm{df}_t}\right)$$

$$\mathrm{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

TF-IDF nano-example

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

TF-IDF nano-example

Query: sweet love

$|q| = \sqrt{}$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized
sweet	**1**					
nurse	0					
love	**1**					
how	0					
sorrow	0					
is	0					

TF-IDF nano-example

Query: sweet love

$$|q| = \sqrt{(.125^2 + .301^2)} = .325$$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized
sweet	**1**	1	3	0.125	0.125	0.383
nurse	0					
love	**1**	1	2	0.301	0.301	0.924
how	0					
sorrow	0					
is	0					

TF-IDF nano-example

Doc 2: Sweet sorrow

$$|q| = \sqrt{\quad}$$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	× q
sweet	**1**						
nurse	0						
love	0						
how	0						
sorrow	**1**						
is	0						

TF-IDF nano-example

Doc 2: Sweet sorrow

$$|q| = \sqrt{(.125^2 + .602^2)} = .615$$

Word	Cnt	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	× q
sweet	**1**	1	3	0.125	.125	.203	0.0779
nurse	0						
love	0						
how	0						
sorrow	**1**	1	1	.602	0.602	.979	0
is	0						

TF-IDF nano-example

Query: sweet love . **Doc 2**: Sweet sorrow = 0.0779

Word	Cnt	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	× q
sweet	**1**	1	3	0.125	.125	.203	**0.0779**
nurse	0						
love	0						
how	0						
sorrow	**1**	1	1	.602	0.602	.979	0
is	0						

Final cosine

(details in chapter)

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

score(q,d1) = 0.747

score(q,d2) = 0.0779

- d1 has both terms, including 2 instances of *sweet*
- d2 is missing one of the terms

Information Retrieval and RAG

TF-IDF: a worked example

Information Retrieval and RAG

Efficiency: The Inverted Index

Goal: rank documents in **D** by their TF-IDF-weighted cosines with query **q**

Do we have to consider all documents in D?

No! **We can ignore all documents that don't have any query words!**

They will have a cosine of 0!

How do we efficiently find all documents that contain a query term $\mathbf{q_i}$?

An index!

Which for historical reasons we call an inverted index!

Inverted Index

Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

Two parts

Dictionary:	**Postings:**
how	→ 3
is	→ 3
love	→ 1→ 3
nurse	→ 1 → 4
sorrow	→ 2
sweet	→ 1→ 2 → 3

Inverted Index Creation

Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

→

Term	Doc#
sweet	1
sweet	1
nurse	1
love	1
sweet	2
sorrow	2
how	3
sweet	3
is	3
love	3
nurse	4

1. Sort by term and document

→

Term	Doc#
how	3
is	3
love	1
love	3
nurse	1
nurse	4
sorrow	2
sweet	2
sweet	1
sweet	1
sweet	3

2. Create linked postings list

→

Dict: Postings:

how	→ 3
is	→ 3
love	→ 1→ 3
nurse	→ 1 → 4
sorrow	→ 2
sweet	→ 1→ 2 → 3

Inverted Index

Dict	Postings
how	→ 3
is	→ 3
love	→ 1 → 3
nurse	→ 1 → 4
sorrow	→ 2
sweet	→ 1 → 2 → 3

So far this just tells us which documents to grab

Next we need enough information to compute tf-idf:

For each term t in vocabulary:

- The df_t (document frequency) of word t

For each term t in each document d:

- The term frequency or count(t,d) of word i in doc d

Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

Doc#

tf

df

how {1} → 3 [1]
is {1} → 3 [1]
love {2} → 1 [1] → 3 [1]
nurse {2} → 1 [1] → 4 [1]
sorry {1} → 2 [1]
sweet {3} → 1 [2] → 2 [1] → 3 [1]

Information Retrieval and RAG

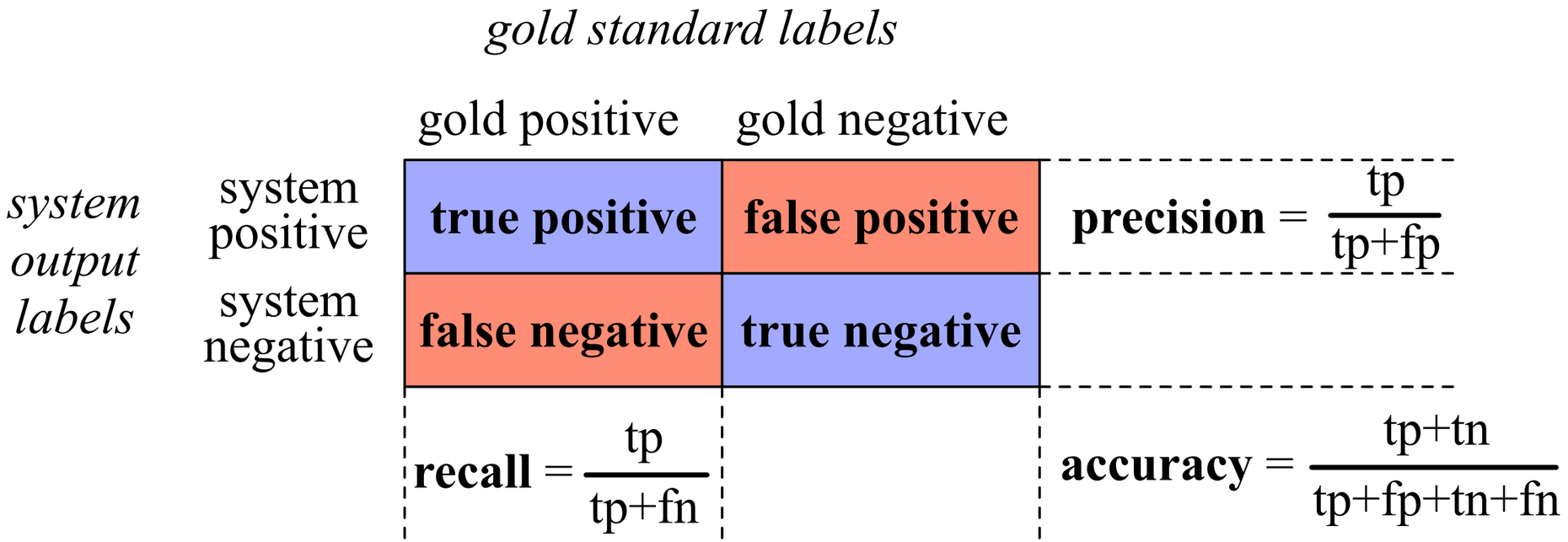
Efficiency: The Inverted Index

Information Retrieval and RAG

Evaluation of IR

Precision and Recall

We saw these already for classification



Precision: % of selected items that are correct

Recall: % of correct items that are selected

Precision and Recall for IR

User makes an information request

Every document in collection is either:

- **Relevant** to the user
- **Not relevant** to the user

The system retrieves a ranked set of documents

Precision for IR

Precision = % of **retrieved** documents that are **relevant**

System retrieves two kinds of documents

- **relevant** documents
- **irrelevant** documents

$$\text{Precision} = \frac{|\textbf{relevant retrieved } docs|}{|\textbf{relevant retrieved } docs| + |\textbf{irrelevant retrieved } docs|}$$

Recall for IR

Recall = % of **relevant** documents that are **retrieved**

Recall = |retrieved **relevant** documents|

|all **relevant** document|

Precision and Recall aren't enough

This is **ranked** retrieval

- Given two ranked retrieval systems
- We want a metric that prefers the one that **ranks relevant documents higher**

We need to adapt precision and recall!

- to be sensitive to **where in the ranking** the relevant document occur

Rank-specific precision and recall

Rank	Judgment	Precision$_{Rank}$	Recall$_{Rank}$
1	R	1.0	.11

Example:

- 25 documents
- 9 are relevant
- If we return all 25
  P=.36, R= 1.0
- Suppose we just return one (and it is relevant)?

Rank-specific precision and recall

Recall is non-decreasing

- Relevant docs increase recall
- Non-relevant docs don't

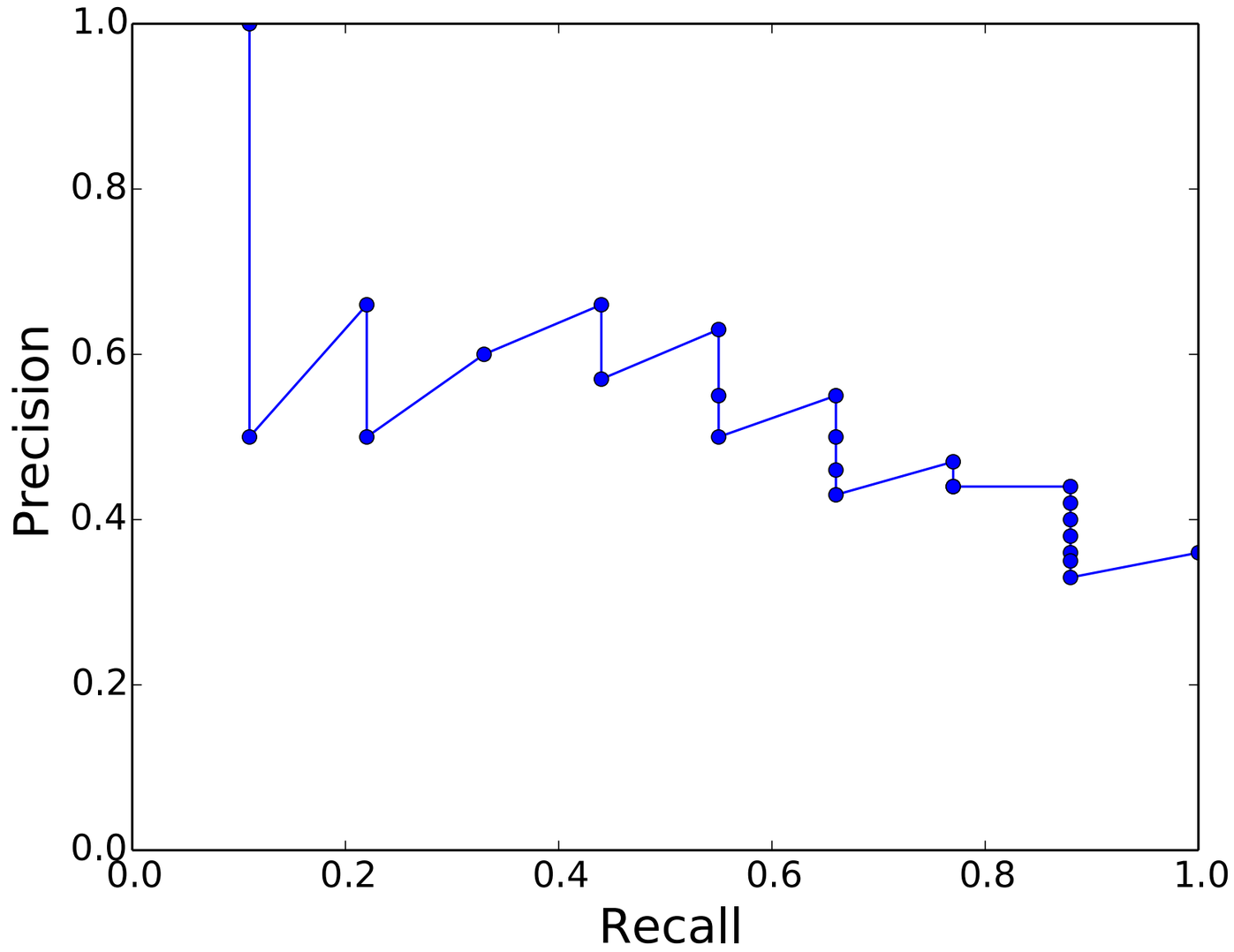
Precision jumps up and down

- increasing for relevant docs
- decreasing otherwise.

Rank	Judgment	Precision_{Rank}	Recall_{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55
11	R	.55	.66
12	N	.50	.66
13	N	.46	.66
14	N	.43	.66
15	R	.47	.77
16	N	.44	.77
17	N	.44	.77
18	R	.44	.88
19	N	.42	.88
20	N	.40	.88
21	N	.38	.88
22	N	.36	.88
23	N	.35	.88
24	N	.33	.88
25	R	.36	1.0

The precision-recall curve (for one query)

Rank	Judgment	Precision$_{Rank}$	Recall$_{Rank}$
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55
11	R	.55	.66
12	N	.50	.66
13	N	.46	.66
14	N	.43	.66
15	R	.47	.77
16	N	.44	.77
17	N	.44	.77
18	R	.44	.88
19	N	.42	.88



Need a metric that aggregates over many queries

Two common approaches

- Mean Average Precision
- Interpolated Precision

Mean Average Precision

Descend through ranked items

Note precision only if item is **relevant**

- e.g. ranks 1, 3, 5, 6 but not 2 or 4:

Precision$_r$(d) "ranked precision"

precision at the rank doc *d* was found.

Rank	Judgment	Precision$_{Rank}$	Recall$_{Rank}$
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44

Average Precision

Descend through ranked items

Note precision only if item is **relevant**

Take the average of the ranked-precisions

$$\mathrm{AP} = \frac{1}{|R_r|} \sum_{d \in R_r} \mathrm{Precision}_r(d)$$

- R_r is the set of relevant documents at or above r
- $\mathrm{Precision}_r(d)$ is precision measured at the rank at which document *d* was found.

Mean Average Precision

For a set of Q queries

Mean Average Precision (MAP):

$$\mathrm{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{AP}(q)$$

Information Retrieval and RAG

Evaluation of IR

Information Retrieval and RAG

Dense Retrieval

Problem with classic IR

The **vocabulary mismatch problem**

- tf-idf or BM25 cosine similarities only work if there is **exact word overlap** between query and doc!
- But query-writer can't know the exact words the doc might include!

Dense retrieval

Instead of representing query and documents with count vectors

Represent both with embeddings!

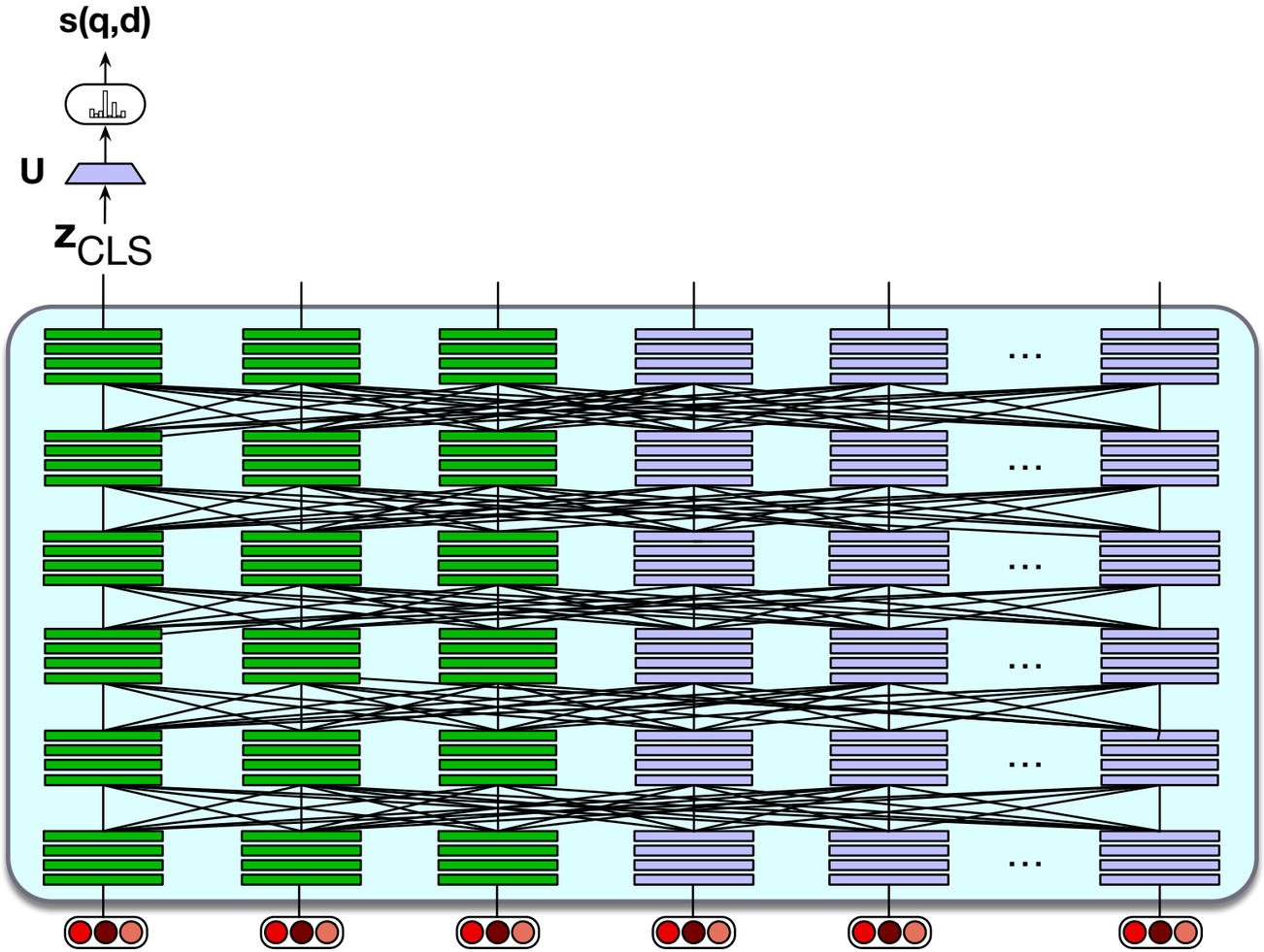
Hypothetical version of dense retrieval: static embeddings

Replace tf-idf vectors with, e.g., word2vec

- Query: the mean of the embeddings of each query word
- Doc: the mean of the doc word embeddings
- Now just compute query-doc cosine as normal.

We don't do this because **contextual embeddings** work much better!

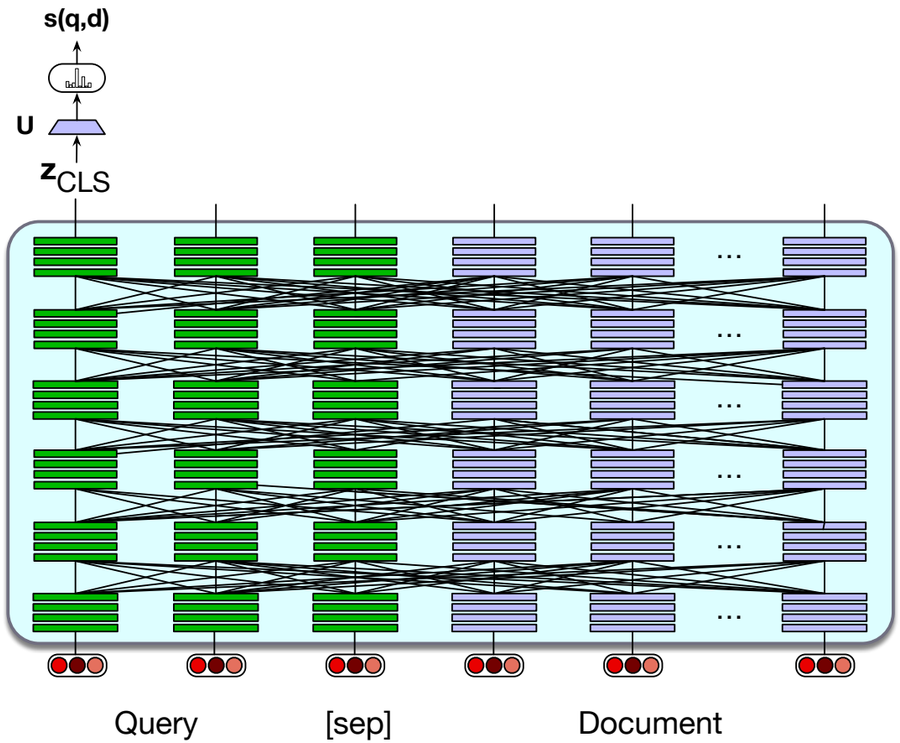
Dense retrieval #1: Single encoder



$$\mathbf{z} = \text{BERT}(\text{q}; \texttt{[SEP]}; \text{d})\texttt{[CLS]}$$

$$\text{score}(q, d) = \text{softmax}(\mathbf{U}(\mathbf{z}))$$

Dense retrieval #1: Single encoder

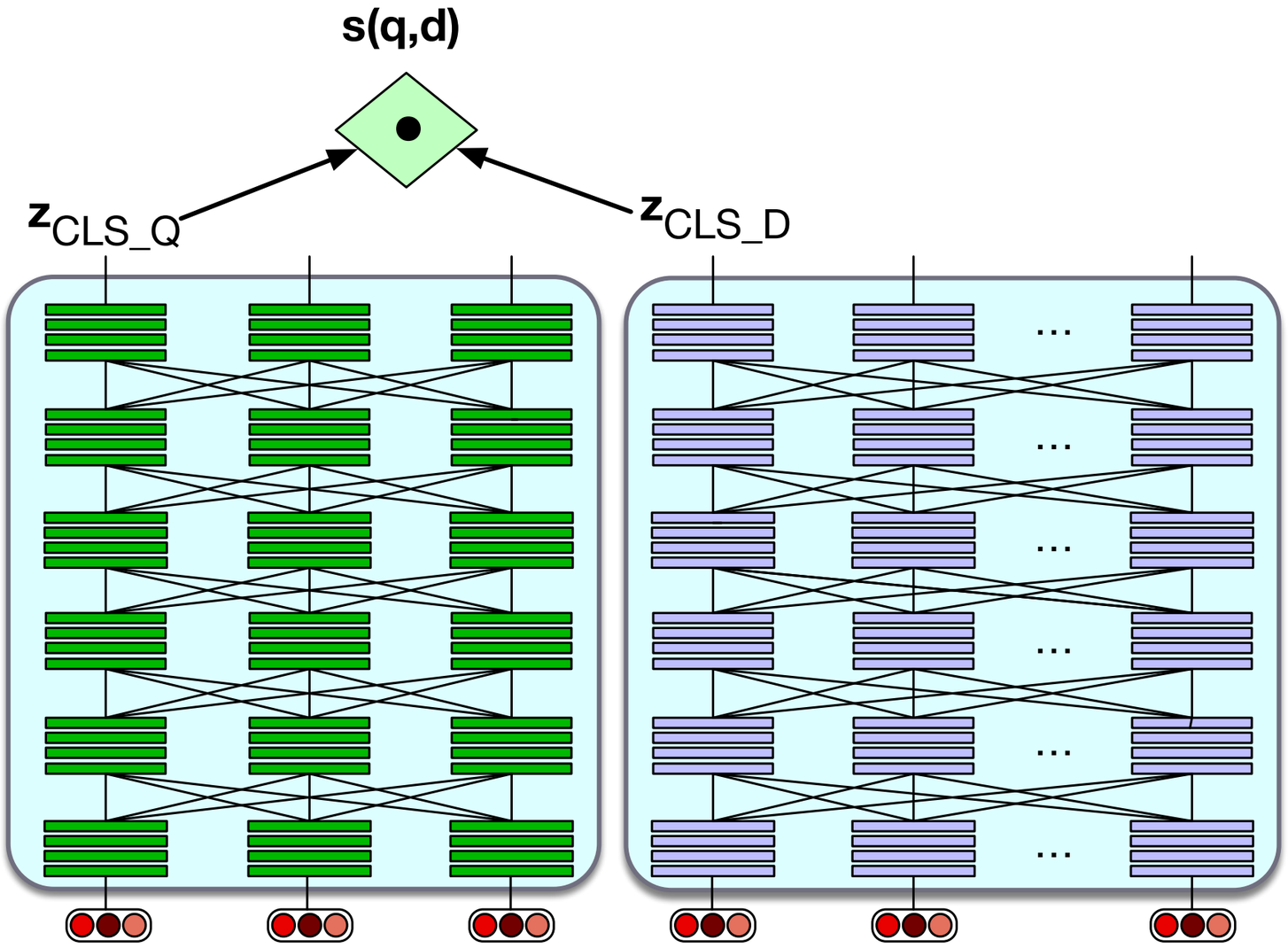


System is run on passages (say 100 tokens) instead of whole documents

Training:

- BERT and linear layer U can then fine-tuned for relevance
- Creating a tuning dataset of relevant and non-relevant passages.

Dense retrieval #2 (**biencoder**)

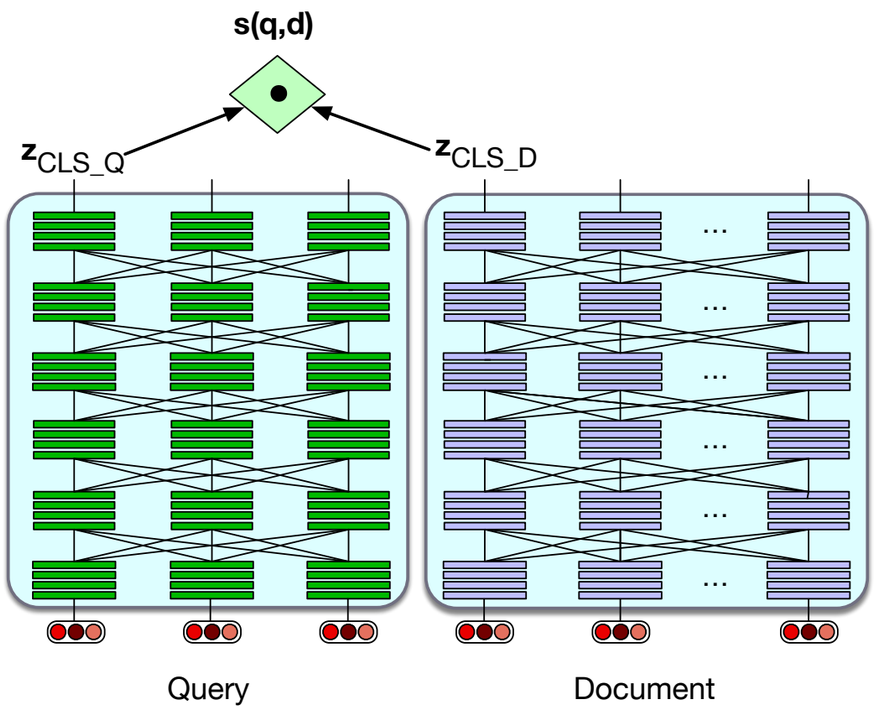


$$\mathbf{z}_q = \mathrm{BERT}_Q(\mathrm{q})[\mathrm{CLS}]$$

$$\mathbf{z}_d = \mathrm{BERT}_D(\mathrm{d})[\mathrm{CLS}]$$

$$\mathrm{score}(q,d) = \mathbf{z}_q \cdot \mathbf{z}_d$$

Dense retrieval #2 (**biencoder**)



Encode doc vectors in advance.

Encode query when it arrives

- Score is dot product between query vector and precomputed doc vector
- Cheaper but less accurate

In-between dense retrieval methods

Use cheap methods (like BM25) as first pass relevance ranking for each document,

Then just rerank the top N ranked docs,

- Using expensive methods like the full BERT scoring

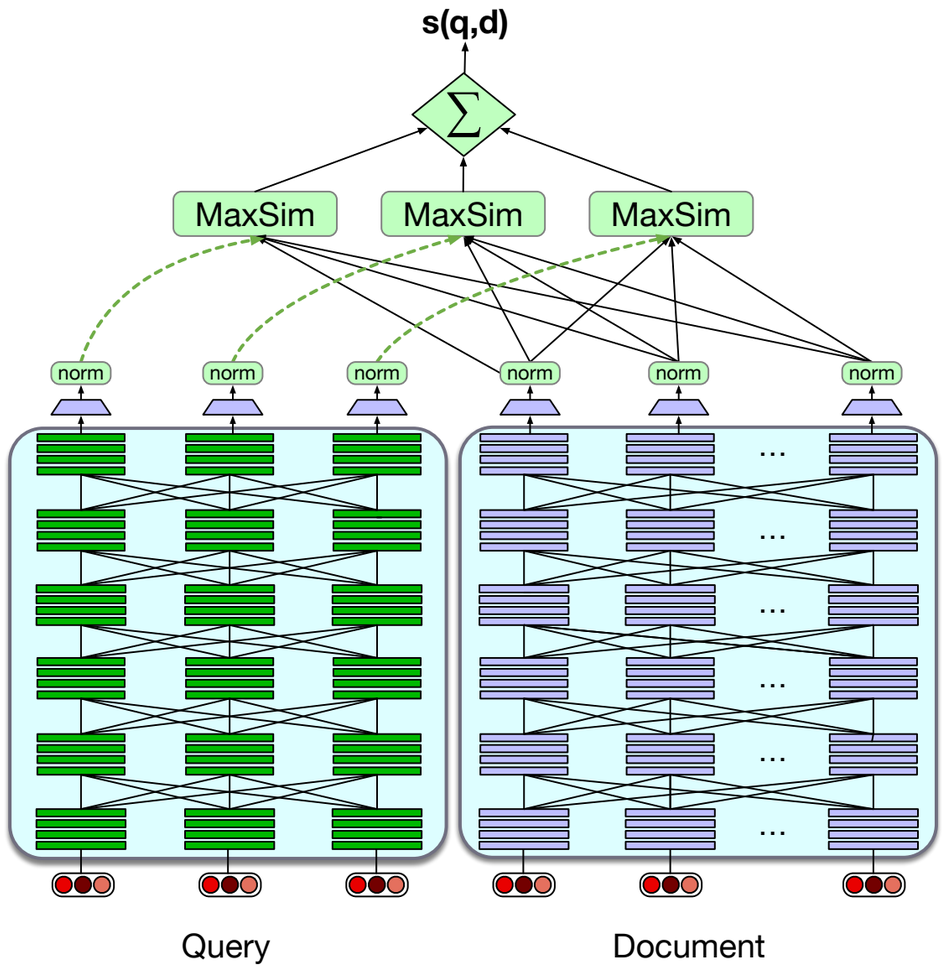
ColBERT

Precompute document but store each word vector

Then do maxsim between document and query words

Khattab and Zaharia (2020), Khattab et al (2021)

ColBERT



$$\text{score}(q,d) = \sum_{i=1}^{N} \max_{j=1}^{m} \mathbf{E}_{q_i} \cdot \mathbf{E}_{d_j}$$

Training for dense retrieval

ColBERT and other models need to be trained

- To fine-tune the BERT encoders and train the linear layers (and the special [Q] and [D] embeddings)
- On datasets of triples $\langle q, d^+, d^- \rangle$
- Some datasets like MS MARCO Ranking have positive examples

Efficiency in dense retrieval

We must rank **every document** for its similarity to the query!

Efficiency for sparse word-count vectors: inverted index

Efficiency for dense retrieval:

- **nearest neighbor search**: finding the set of dense document vectors that have the highest dot product with a dense query vector.
- Approximate nearest neighbor algorithm **Faiss** (Johnson et al., 2017).
  - Approximates the doc vector by a smaller quantized vector

Information Retrieval and RAG

Dense Retrieval

Information Retrieval and RAG

Retrieval-Augmented Generation

IR plays a central role in modern LLMs

How to answer factual questions like

- Where is the Louvre Museum located?
- How to get a script l in latex?
- Where does the energy in a nuclear explosion come from?

Just prompt an LLM!



AI Overview

The main Louvre Museum is located in **Paris, France, at the Musée du Louvre, 75001 Paris, France**. It is situated on the Right Bank of the Seine River in the city's 1st arrondissement, housed within the historic Louvre Palace.

- **Address:** Rue de Rivoli, 75001 Paris, France.

LLMs seem to store facts in the connections in their feedforward layers!

But there are issues!

LLMs Hallucinate

Hallucination: a response that is not faithful to the facts of the world.

In the legal domain LLMs were shown to hallucinate up to 88% of the time!

Dahl et al. (2024)

Can't use Proprietary Data

People need to ask questions about:

- personal email.
- healthcare applications to medical records.
- internal corporate documents
- legal documents discovery

Can't Handle Dynamic Data

LLMs can't answer questions about rapidly changing information

Things that happened last week

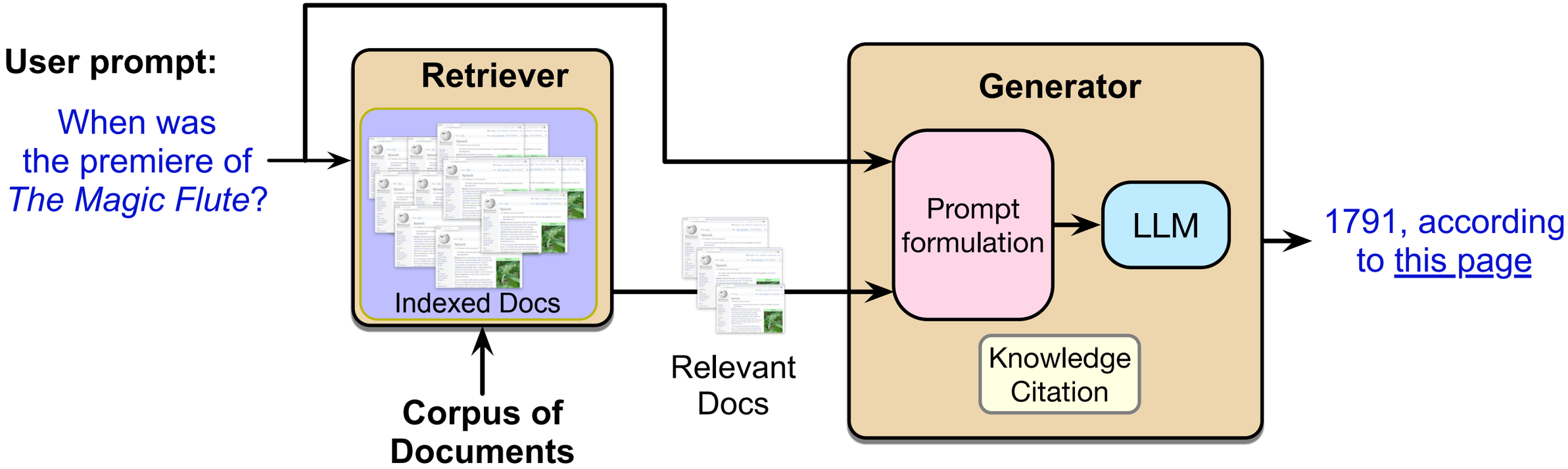
In general, data shifts over time

Solution: RAG

Retrieval-Augmented Generation

1. Use IR to **retrieve** documents from some collection
2. Then use LLM to **generate** an answer conditioned on the documents

Retrieval Augmented Generation (RAG)



Basic RAG

Given a document collection D and a user query q

- Call a retriever to return top *k* passages
- Create a prompt that includes *q* and the passages
- Call an LLM with the prompt

Schematic of a RAG Prompt

```
retrieved passage 1

retrieved passage 2

...

retrieved passage k

Based on these texts, answer this question:  What year
was the premiere of The Magic Flute?
```

$$p(x_1, \ldots, x_n) \;=\; \prod_{i=1}^{n} p(x_i | \mathrm{R}(q)\ ;\ \text{Answer the following question...}\ ;\ q\ ; x_{<i})$$

Extensions: Agent-based RAG

Instead of running RAG automatically on every user turn

Have a retrieval agents

System decides when to call it and for which document collection

Extensions: Training

Instruction-tune an LLM on a dataset of questions with retrieved passages and correct answers

Test-time compute: prompt an LLM to answer the question and simultaneously to generate reflections on which passages were useful

Extensions: Knowledge Citations

Q: Which films have Gong Li as a member of their cast?
A: The Story of Qiu Ju [1], Farewell My Concubine [2], The Monkey King 2 [3], Mulan [3], Saturday Fiction [3] ...

‘‘Write an answer for the given question using only the provided search results (some of which might be irrelevant) and cite them properly... Always cite for any factual claim".

Information Retrieval and RAG

Retrieval-Augmented Generation

Information Retrieval and RAG

Question Answering datasets and evals

Two kinds of question answering datasets

Natural information-seeking questions

- Someone actually wanted to know the answer to this

Probing (testing) questions

- Exam-type questions for LLM evaluation

Natural Questions
(Kwiatkowski et al., 2019),

anonymized English **queries** to the Google search engine and short and long **answers** (hand-created from Wikipedia)

"When are hops added to the brewing process?"

short answer: *the boiling process*

long answer: paragraph from the Wikipedia page on *Brewing*

MS MARCO (Microsoft Machine Reading Comprehension) collection of datasets,

1 million real anonymized English questions from Microsoft Bing query logs

human generated answer

9 million passages (Bajaj et al., 2016)

Probing dataset: MMLU

15908 knowledge and reasoning questions in 57 areas including medicine, mathematics, computer science, law, etc..

Sourced from exams for humans like GRE, AP

College Computer Science

Any set of Boolean operators that is sufficient to represent all Boolean expressions is said to be complete. Which of the following is NOT complete?

(A) AND, NOT

(B) NOT, OR

(C) AND, OR

(D) NAND

Information Retrieval and RAG

Question Answering datasets and evals

Speech and Language Processing

Chapter 8: Speech Synthesis

Outline

1) TTS Architectures
2) TTS Components
   - Text Analysis
     - Text Normalization
     - Homonym Disambiguation
     - Grapheme-to-Phoneme (Letter-to-Sound)
     - Intonation
   - Waveform Generation
     - Unit Selection
     - Diphones

Dave Barry on TTS

"And computers are getting smarter all the time; scientists tell us that soon they will be able to talk with us.

(By "they", I mean computers; I doubt scientists will ever be able to talk to us.)

Example: Siri, Google Assistant, etc.

Modern TTS systems

- 1960's first full TTS: Umeda et al (1968)
- 1970's
  - Joe Olive 1977 concatenation of linear-prediction diphones
  - Speak and Spell
- 1980's
  - 1979 MIT MITalk (Allen, Hunnicut, Klatt)
- 1990's-present
  - Diphone synthesis
  - Unit selection synthesis



TTS Architecture

- The three types of TTS
  - Formant
  - Articulatory
  - Concatenative

- A full system needs to go all the way from random text to sound.

Overview of TTS: Architectures of Modern Synthesis

- Articulatory Synthesis:
  - Model movements of articulators and acoustics of vocal tract
- Formant Synthesis:
  - Start with acoustics, create rules/filters to create each formant
- Concatenative Synthesis:
  - Use databases of stored speech to assemble new utterances.

Concatenative Synthesis

- Concatenative synthesis is a technique for synthesising sounds by concatenating short samples of recorded sound (called units).
- The duration of the units is not strictly defined and may vary according to the implementation, roughly in the range of 10 milliseconds up to 1 second.
- All current commercial systems.

Speech Synthesis

- The task of speech synthesis is to map a text like the following:

  "Natural Language Processing"

  To a waveform like the following:



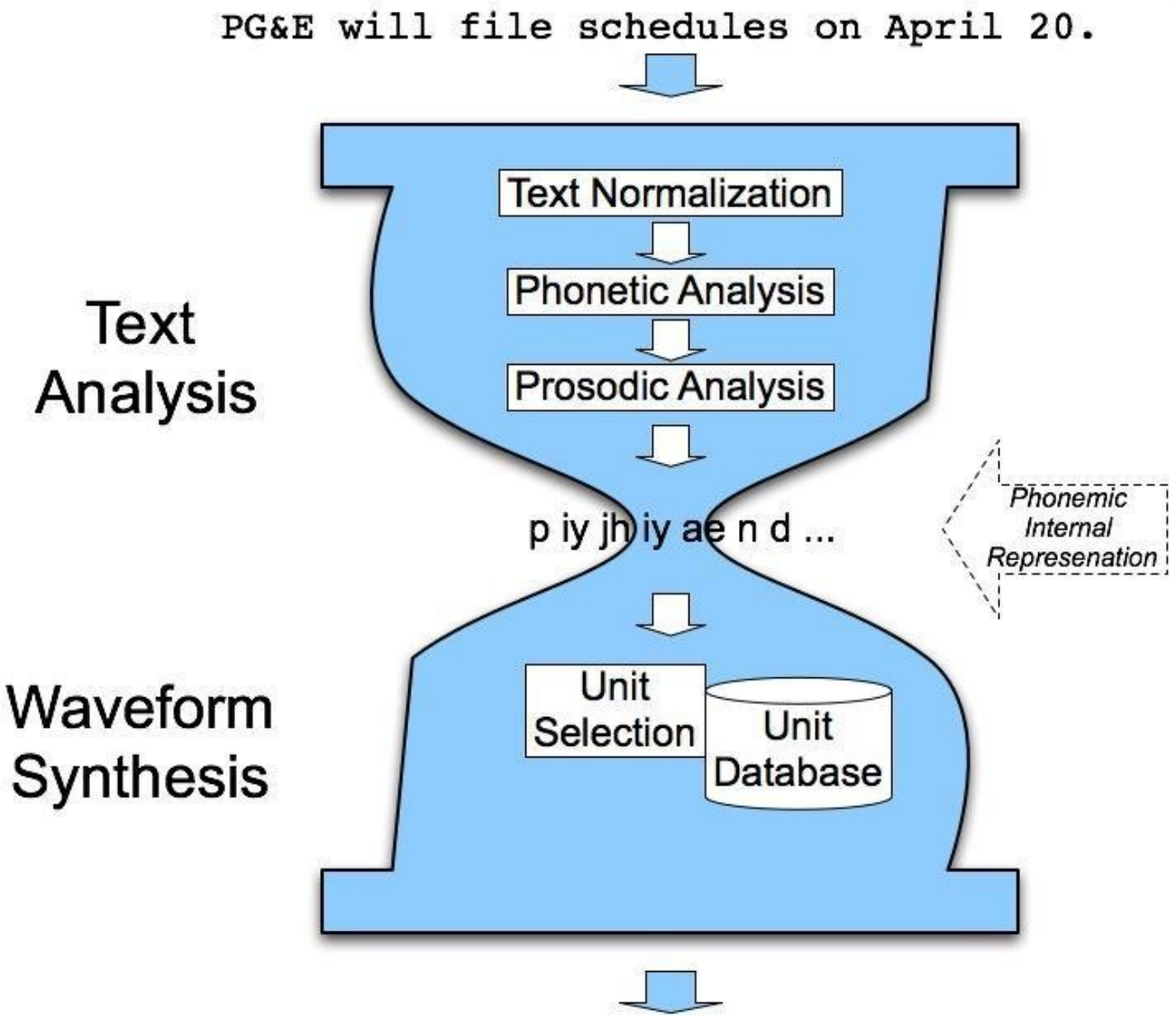
- Speech synthesis perform this mapping in two steps.

Two steps

- Step 1 (text analysis): convert the input text into a phonemic internal representation.

| | | |

The Hourglass Metaphor



1. Text Normalization

He said the increase in credit limits helped B.C. Hydro achieve record net income of about $1 billion during the year ending March 31. This figure does not include any write-downs that may occur if Powerex determines that any of its customer accounts are not collectible. Cousins, however, was insistent that all debts will be collected: "We continue to pursue monies owing and we expect to be paid for electricity we have sold."

- 1st task is Sentence Tokenization
- Text Normalization
  - Identify tokens in text
  - Chunk tokens into reasonably sized sections
  - Map tokens to words
  - Identify types for words

Identify Types of Tokens, and Convert Tokens to Words

- Pronunciation of numbers often depends on type:
  - 1776 **date**:
    - seventeen seventy six.
  - 1776 **phone number**:
    - one seven seven six
  - 1776 **quantifier**:
    - one thousand seven hundred (and) seventy six
  - 25 **day**:
    - twenty-fifth

Classify token into 1 of 20 types

- EXPN: abbrev, contractions (adv, N.Y., mph, gov't)
- LSEQ: letter sequence (CIA, D.C., CDs)
- ASWD: read as word, e.g. CAT, proper names
- MSPL: misspelling
- NUM: number (cardinal) (12,45,1/2, 0.6)
- NORD: number (ordinal) e.g. May 7, 3rd, Bill Gates II
- NTEL: telephone (or part) e.g. 212-555-4523
- NDIG: number as digits e.g. Room 101
- NIDE: identifier, e.g. 747, 386, I5, PC110
- NADDR: number as stresst address, e.g. 5000 Pennsylvania
- NZIP, NTIME, NDATE, NYER, MONEY, BMONY, PRCT,URL,etc
- SLNT: not spoken (KENT*REALTY)

Homograph disambiguation

Do you live (/l ih v/) near a zoo with live (/l ay v/) animals?

I prefer bass (/b ae s/) fishing to playing the bass (/b ey s/) guitar.

19 most frequent homographs, from Liberman and Church

use	319	survey	91
increase	230	project	90
close	215	separate	87
record	195	present	80
house	150	read	72
contract	143	subject	68
lead	131	rebel	48
live	130	finance	46
lives	105	estimate	46
protest	94		

Not a huge problem, but still important

Letter-to-Sound: Getting from words to phones

- Two methods:
  - Dictionary-based
  - Rule-based (Letter-to-sound=LTS)
- Early systems, all LTS
- MITalk was radical in having huge 10K word dictionary
- Now systems use a combination

Pronunciation Dictionaries: CMU

- CMU dictionary: 127K words
  - http://www.speech.cs.cmu.edu/cgi-bin/cmudict
- Some problems:
  - Has errors
  - Only American pronunciations
  - No syllable boundaries
  - Doesn't tell us which pronunciation to use for which homophones
    - (no POS tags)
  - Doesn't distinguish case
    - The word US has 2 pronunciations
      - [AH1 S] and [Y UW1 EH1 S]

Pronunciation Dictionaries: UNISYN

- UNISYN dictionary: 110K words (Fitt 2002)
  - http://www.cstr.ed.ac.uk/projects/unisyn/

- Benefits:
  - Has syllabification, stress, some morphological boundaries
  - Pronunciations can be read off in
    - General American
    - RP British
    - Australia
    - Etc

- (Other dictionaries like CELEX not used because too small, British-only)

Dictionaries aren't sufficient

- Unknown words (= OOV = "out of vocabulary")
  - Increase with the (sqrt of) number of words in unseen text
  - Black et al (1998) OALD on 1st section of Penn Treebank:
  - Out of 39923 word tokens,
    - 1775 tokens were OOV: 4.6% (943 unique types):

names	unknown	Typos and other
1360	351	64
76.6%	19.8%	3.6%

- So commercial systems have 4-part system:
  - Big **dictionary**
  - **Names** handled by special routines
  - **Acronyms** handled by special routines
  - Machine learned **g2p** (grapheme-to-phoneme) algorithm for other unknown words

Names

- Big problem area is names
- Names are common
  - 20% of tokens in typical newswire text will be names
  - 1987 Donnelly list (72 million households) contains about 1.5 million names
  - Personal names: McArthur, D'Angelo, Jiminez, Rajan, Raghavan, Sondhi, Xu, Hsu, Zhang, Chang, Nguyen
  - Company/Brand names: Infinit, Kmart, Cytyc, Medamicus, Inforte, Aaon, Idexx Labs, Bebe

Names

- Methods:
  - Can do morphology (Walters -> Walter)
  - Can write stress-shifting rules (Jordan -> Jordanian)
  - Rhyme analogy: Plotsky by analogy with Trostsky (replace tr with pl)
  - Liberman and Church: for 250K most common names, got 212K (85%) from these modified-dictionary methods, used LTS for rest.
  - Can do automatic country detection (from letter trigrams) and then do country-specific rules
  - Can train **g2p** system specifically on names
    - Or specifically on types of names (brand names, Russian names, etc)

Acronyms

- Use machine learning to detect acronyms
  - EXPN
  - ASWORD
  - LETTERS
- Use acronym dictionary, hand-written rules to augment

Letter-to-Sound Rules

- Earliest algorithms: handwritten Chomsky+Halle-style rules:

$$c \rightarrow [\text{k}] \;/\; \underline{\quad} \;\{\text{a,o}\}\text{V} \qquad \text{; context dependent}$$
$$c \rightarrow [\text{s}] \qquad \text{; context independent}$$

- Festival version of such LTS rules:
- (LEFTCONTEXT [ITEMS] RIGHTCONTEXT = NEWITEMS)
- Example:
  - (# [c h] C = k)
  - (# [c h] = ch)
- # denotes beginning of word
- C means all consonants
- Rules apply in order
  - “christmas” pronounced with [k]
  - But word with ch followed by non-consonant pronounced [ch]
    - E.g., “choice”

*

Stress rules in hand-written LTS

- English famously evil: one from Allen et al 1987

$$V \rightarrow [+\text{stress}] \,/\, X __ \text{C}^* \{\text{V}_{\text{short}} \text{ C C?}|\text{V}\} \{\text{V}_{\text{short}} \text{ C}^*|\text{V}\}$$

- This rule represents the following two situations:
  1. Assign 1-stress to the vowel in a syllable preceding a weak syllable followed by a morpheme-final syllable containing a short vowel and 0 or more consonants (e.g.difficult)
  2. Assign 1-stress to the vowel in a syllable preceding a weak syllable followed by a morpheme-final vowel (e.g.oregano)

*

Modern method: Learning LTS rules automatically

- Induce LTS from a dictionary of the language
- Black et al. 1998
- Applied to English, German, French
- Two steps:
  - **alignment**
  - (CART-based) **rule-induction**

*

Alignment

- Letters: c h e c k e d
- Phones: ch _ eh _ k _ t
- Black et al Method 1:



  - First scatter epsilons in all possible ways to cause letters and phones to align
  - Then collect stats for P(phone|letter) and select best to generate new stats

$$P(p_i|l_j) = \frac{\text{count}(p_i, l_j)}{\text{count}(l_j)}$$

  - This iterated a number of times until settles (5-6)
  - This is EM (expectation maximization) alg

Alignment: Black et al method 2

- Hand specify which letters can be rendered as which phones
  - C goes to k/ch/s/sh
  - W goes to w/v/f, etc
- An actual list:

```
c: k ch s sh t-s ϵ
e: ih iy er ax ah eh ey uw ay ow y-uw oy aa ϵ
```

- Once mapping table is created, find all valid alignments, find p(letter|phone), score all alignments, take best

Alignment

- Some alignments will turn out to be really bad.
- These are just the cases where pronunciation doesn't match letters:
  - Dept d ih p aa r t m ah n t
  - CMU s iy eh m y uw
  - Lieutenant l eh f t eh n ax n t (British)
- Also foreign words
- These can just be removed from alignment training

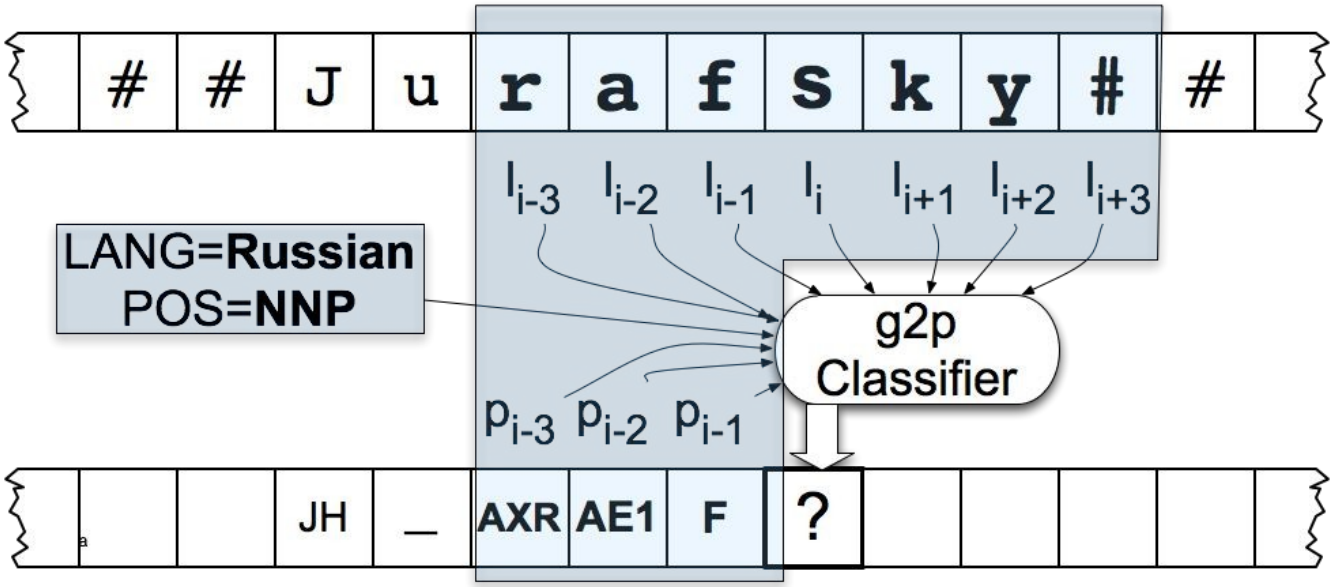
*

Building CART trees

- Build a CART tree for each letter in alphabet (26 plus accented) using context of +-3 letters
- # # # c h e c -> ch
- c h e c k e d -> _

*

Add more features



- Even more: for French liaison, we need to know what the next word is, and whether it starts with a vowel
- French 'six'
  - [s iy s] in j'en veux six
  - [s iy z] in six enfants
  - [s iy] in six filles

Prosody:

from words+phones to boundaries, accent, F0, duration

- Prosodic phrasing
  - Need to break utterances into phrases
  - Punctuation is useful, not sufficient
- Accents:
  - Predictions of accents: which syllables should be accented
  - Realization of F0 contour: given accents/tones, generate F0 contour
- Duration:
  - Predicting duration of each phone

Defining Intonation

- Ladd (1996) “Intonational phonology”
- “The use of suprasegmental phonetic features

  Suprasegmental = above and beyond the segment/phone
  - F0
  - Intensity (energy)
  - Duration
- to convey sentence-level pragmatic meanings”
  - i.e. meanings that apply to phrases or utterances as a whole, not lexical stress, not lexical tone.

Three aspects of prosody

- Prominence: some syllables/words are more prominent than others
- Structure/boundaries: sentences have prosodic structure
  - Some words group naturally together
  - Others have a noticeable break or disjuncture between them
- Tune: the intonational melody of an utterance.

*

Prosodic Prominence: Pitch Accents

A: What types of foods are a good source of vitamins?

B1: Legumes are a good source of VITAMINS.

B2: LEGUMES are a good source of vitamins.

- Prominent syllables are:
  - Louder
  - Longer
  - Have higher F0 and/or sharper changes in F0 (higher F0 velocity)

*

Stress vs. accent (2)

- The speaker decides to make the word vitamin more prominent by accenting it.
- Lexical stress tell us that this prominence will appear on the first syllable, hence VItamin.

Which word receives an accent?

- **It depends on the context. For example, the 'new' information in the answer to a question is often accented, while the 'old' information usually is not.**
  - Q1: What types of foods are a good source of vitamins?
  - A1: LEGUMES are a good source of vitamins. 
  - Q2: Are legumes a source of vitamins?
  - A2: Legumes are a GOOD source of vitamins. 
  - Q3: I've heard that legumes are healthy, but what are they a good source of ?
  - A3: Legumes are a good source of VITAMINS. 

Factors in accent prediction

- Part of speech:
  - Content words are usually accented
  - Function words are rarely accented
    - Of, for, in on, that, the, a, an, no, to, and but or will may would can her is their its our there is am are was were, etc

Complex Noun Phrase Structure

- **Sproat, R. 1994. English noun-phrase accent prediction for text-to-speech. Computer Speech and Language 8:79-94.**
- Proper Names, stress on right-most word
  - New York CITY; Paris, FRANCE
- Adjective-Noun combinations, stress on noun
  - Large HOUSE, red PEN, new NOTEBOOK
- Noun-Noun compounds: stress left noun
  - HOTdog (food) versus HOT DOG (overheated animal)
  - WHITE house (place) versus WHITE HOUSE (made of stucco)
- examples:
  - MEDICAL Building, APPLE cake, cherry PIE.
  - What about: Madison avenue, Park street ???
- Some Rules:
  - Furniture+Room -> RIGHT (e.g., kitchen TABLE)
  - Proper-name + Street -> LEFT (e.g. PARK street)

State of the art

- Hand-label large training sets
- Use CART, SVM, CRF, etc to predict accent
- Lots of rich features from context (parts of speech, syntactic structure, information structure, contrast, etc.)
- Classic lit:
  - Hirschberg, Julia. 1993. Pitch Accent in context: predicting intonational prominence from text. Artificial Intelligence 63, 305-340

Levels of prominence

- Most phrases have more than one accent
- The last accent in a phrase is perceived as more prominent
  - Called the Nuclear Accent
- **Emphatic** accents like nuclear accent often used for semantic purposes, such as indicating that a word is contrastive, or the semantic focus.
  - The kind of thing you represent via ***s in IM, or capitalized letters
  - ‘ I know **SOMETHING** interesting is sure to happen,’ she said to herself.
- Can also have words that are **less** prominent than usual
  - Reduced words, especially function words.
- Often use 4 classes of prominence:
  1. **emphatic accent,**
  2. **pitch accent,**
  3. **unaccented,**
  4. **reduced**

Yes-No question

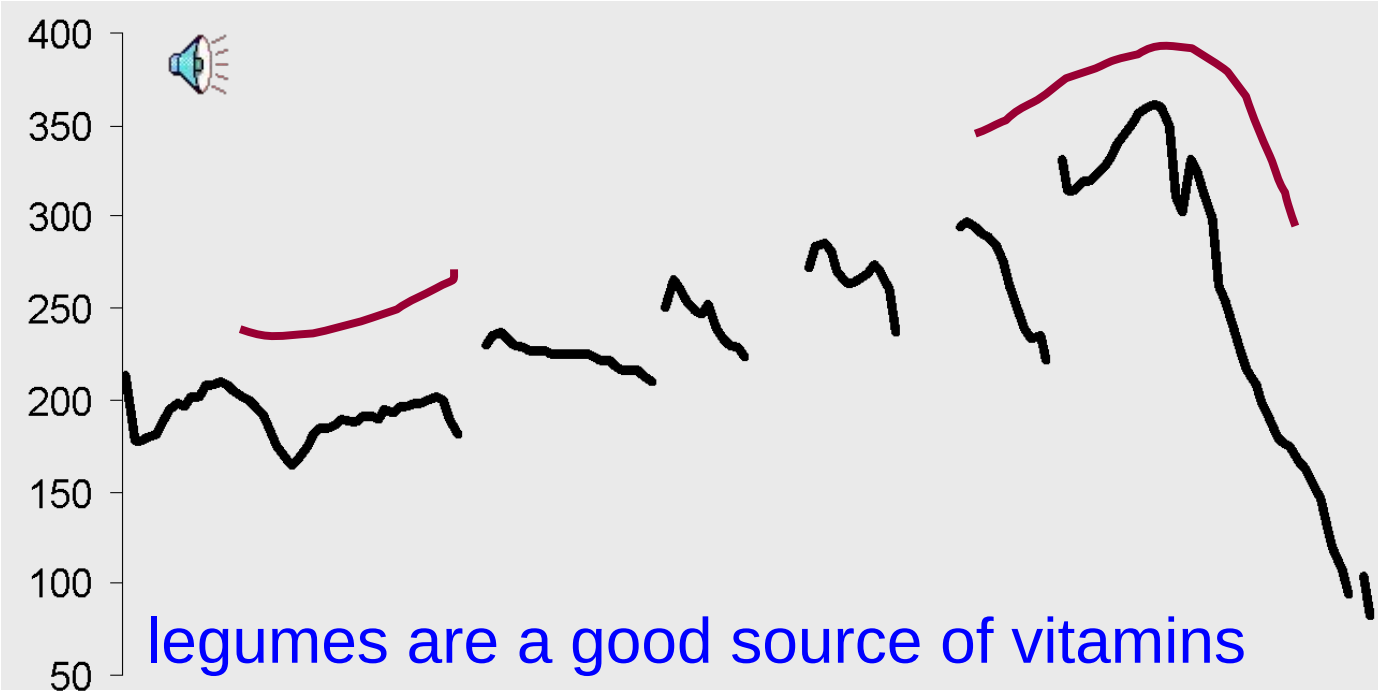


Rise from the main accent to the end of the sentence.

*

‘Surprise-redundancy’ tune

[How many times do I have to tell you ...]



Low beginning followed by a gradual rise to a high at the end.

*

‘Contradiction’ tune

“I’ve heard that linguini is a good source of vitamins.”



[... how could you think that?]

Sharp fall at the beginning, flat and low, then rising at the end.

*

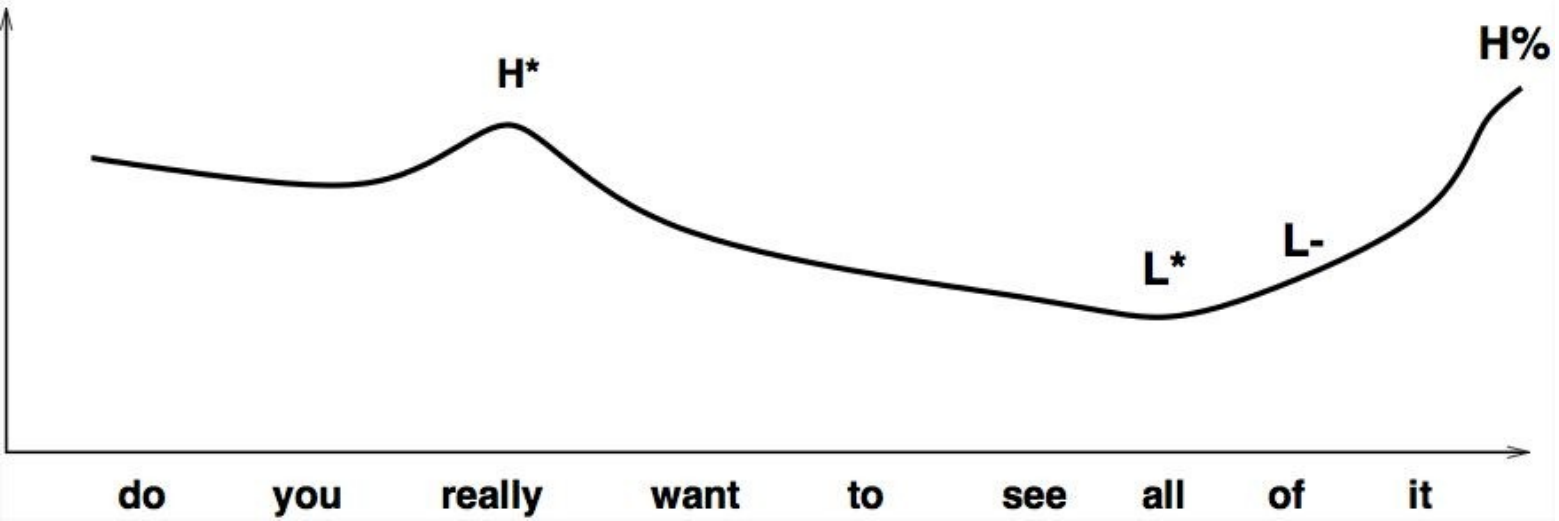
Duration

- **Simplest**:
  - fixed size for all phones (100 ms)
- **Next simplest**:
  - average duration for that phone (from training data). Samples from SWBD in ms:
  - aa 118 b 68
  - ax 59 d 68
  - ay 138 dh 44
  - eh 87 f 90
  - ih 77 g 66
- Next Next Simplest:
  - add in phrase-final and initial lengthening plus stress:

Intermediate representation: using Festival

- Do you really want to see all of it?

				H*										L*		L- H%			
do		you		really				want				to		see		all		of	it
d	uw	y	uw	r	ih	l	iy	w	aa	n	t	t	ax	s	iy	ao	l	ah	v	ih	t
110	110	50	50	75	64	57	82	57	50	72	41	43	47	54	130	76	90	44	62	46	220



*

Waveform Synthesis

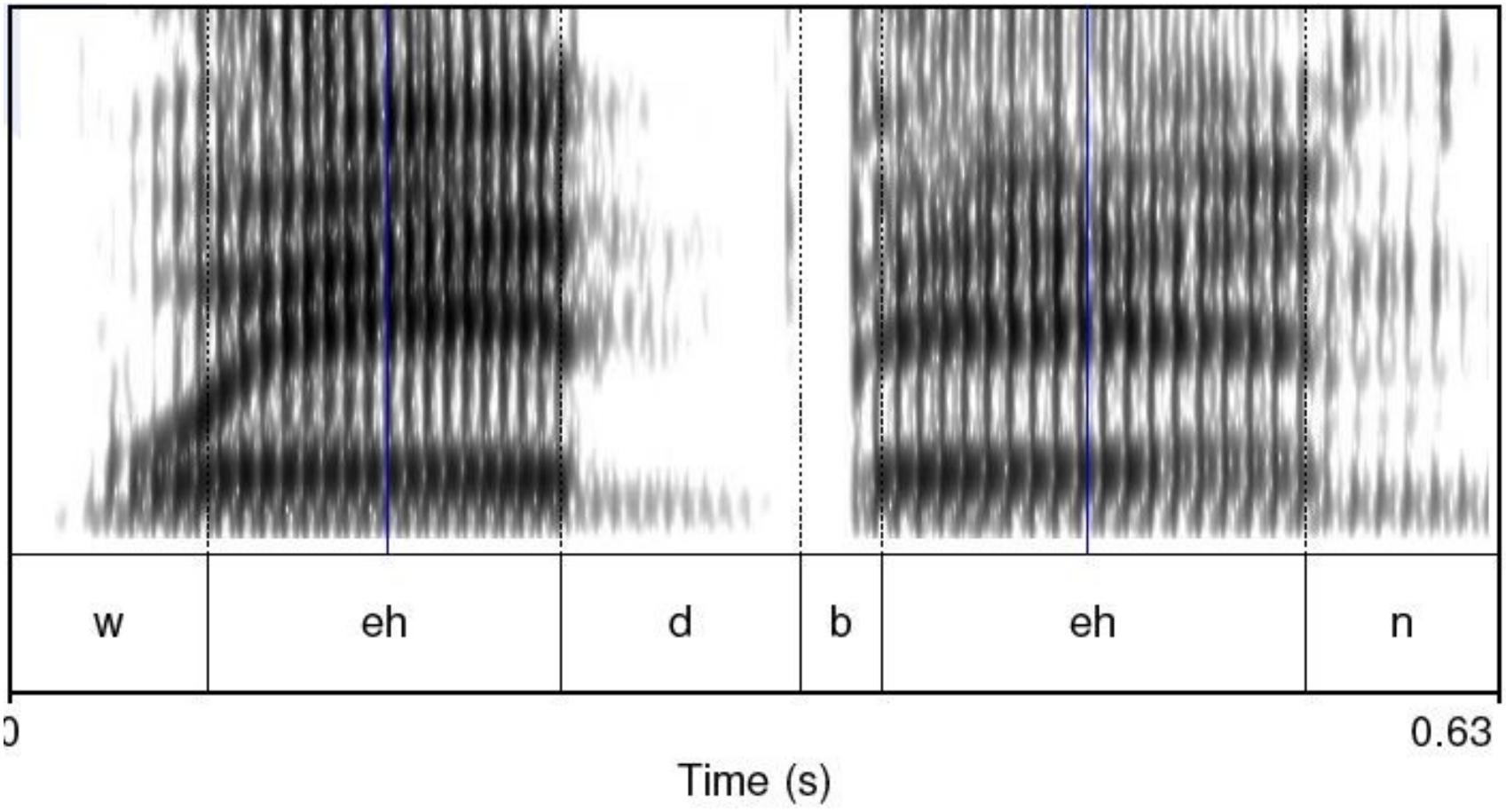
- Given:
  - String of phones
  - Prosody
    - Desired F0 for entire utterance
    - Duration for each phone
    - Stress value for each phone, possibly accent value
- Generate:
  - Waveforms

Diphone TTS architecture

- Training:
  - Choose units (kinds of diphones)
  - Record 1 speaker saying 1 example of each diphone
  - Mark the boundaries of each diphones,
    - cut each diphone out and create a diphone database
- Synthesizing an utterance,
  - grab relevant sequence of diphones from database
  - Concatenate the diphones, doing slight signal processing at boundaries
  - use signal processing to change the prosody (F0, energy, duration) of selected sequence of diphones

Diphones

- Mid-phone is more stable than edge:



*

Diphones

- mid-phone is more stable than edge
- Need O(phone^2) number of units
  - Some combinations don't exist (hopefully)
  - ATT (Olive et al. 1998) system had 43 phones
    - 1849 possible diphones
    - Phonotactics ([h] only occurs before vowels), don't need to keep diphones across silence
    - Only 1172 actual diphones
  - May include stress, consonant clusters
    - So could have more
  - Lots of phonetic knowledge in design
- Database relatively small (by today's standards)
  - Around 8 megabytes for English (16 KHz 16 bit)

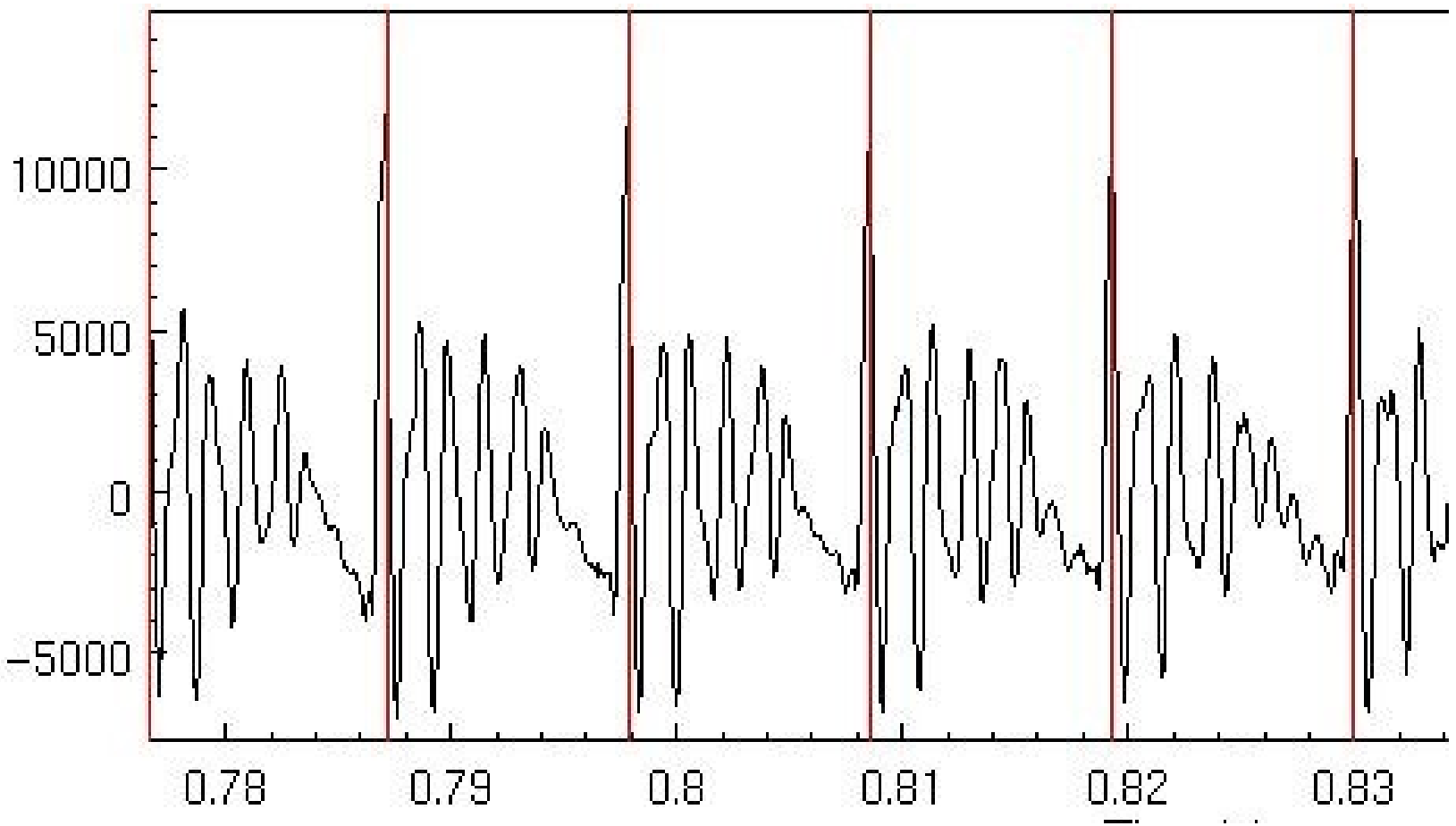
Voice

- Speaker
  - Called a **voice talent**
- Diphone database
  - Called a **voice**

Prosodic Modification

- Modifying pitch and duration independently
- Changing sample rate modifies both:
  - Chipmunk speech
- Duration: duplicate/remove parts of the signal
- Pitch: resample to change pitch

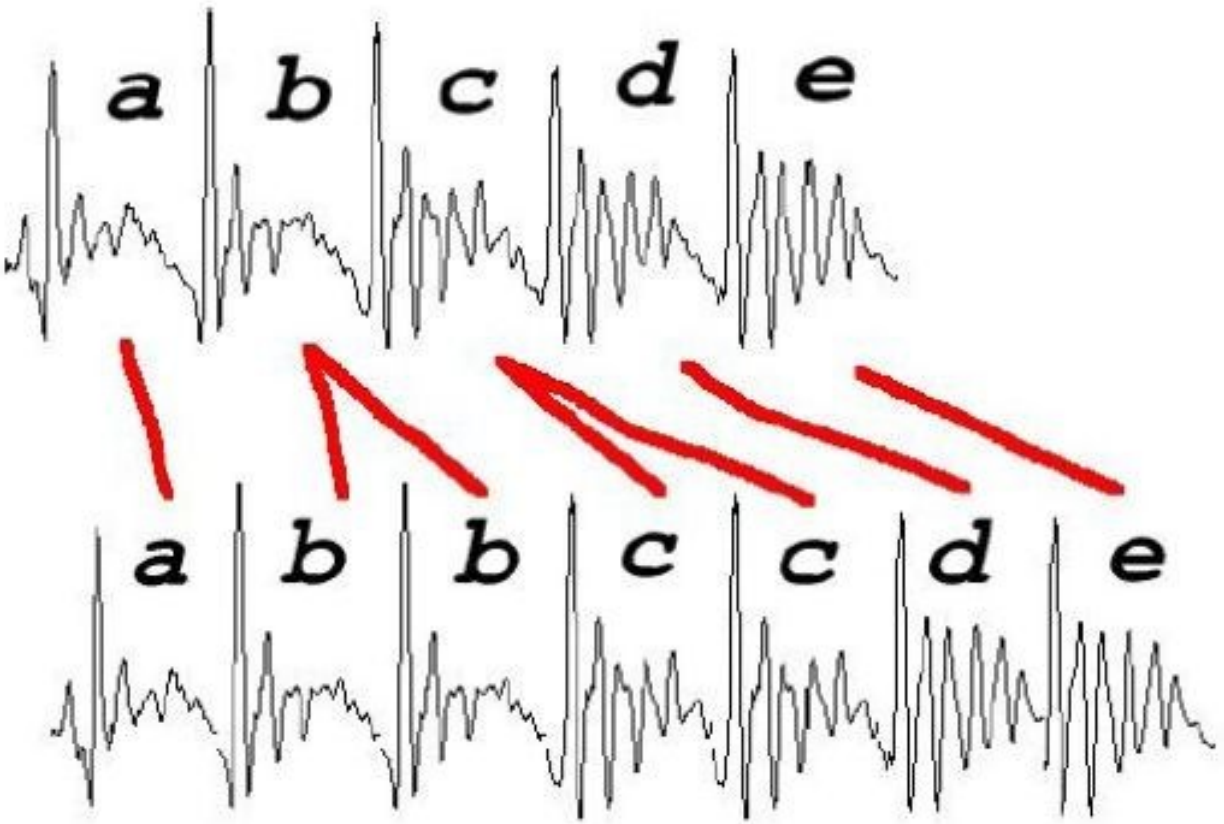
Speech as Short Term signals



*

Duration modification

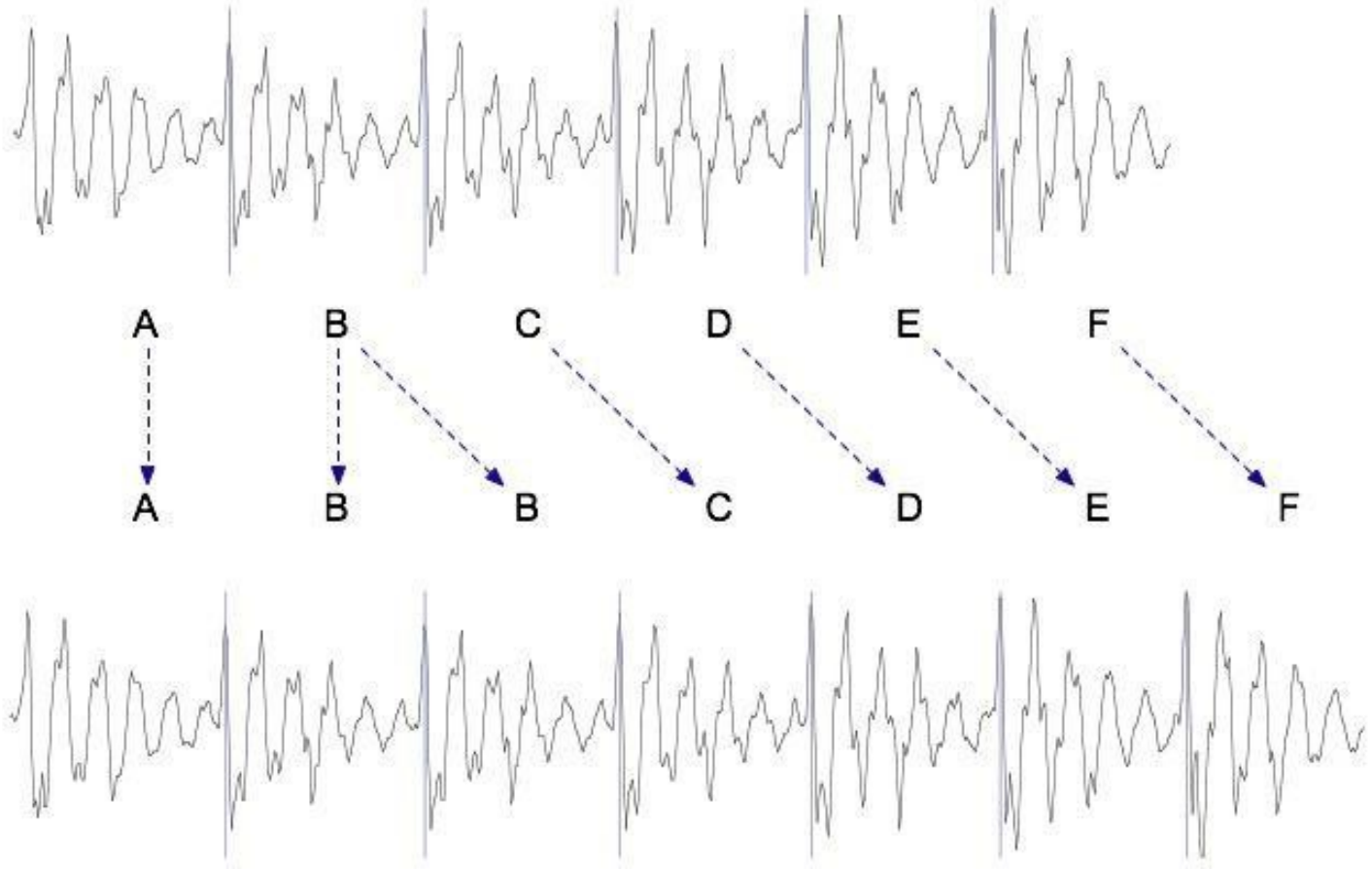
- Duplicate/remove short term signals



*

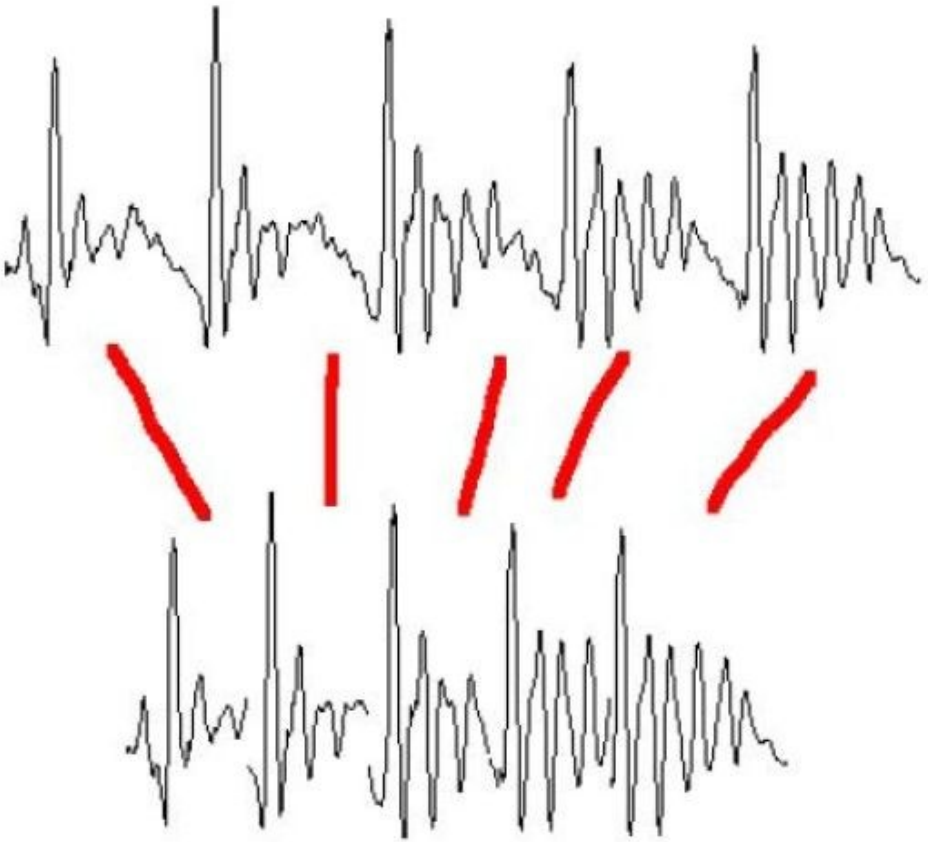
Duration modification

- Duplicate/remove short term signals



Pitch Modification

- Move short-term signals closer together/further apart



*

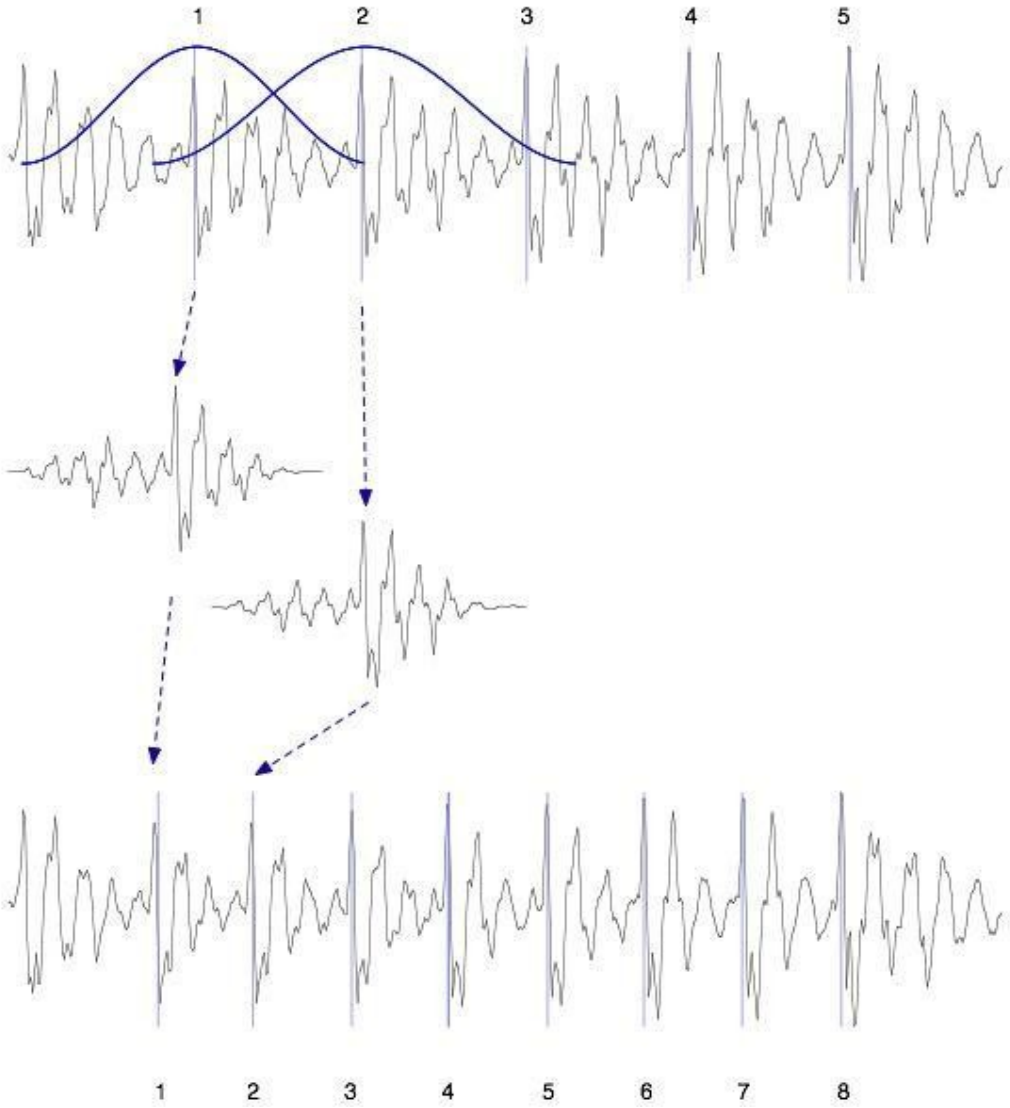
TD-PSOLA ™

- Time-Domain Pitch Synchronous Overlap and Add
- Patented by France Telecom (CNET)
- Very efficient
  - No FFT (or inverse FFT) required
- Can modify Hz up to two times or by half

TD-PSOLA ™

- Time-Domain Pitch Synchronous Overlap and Add
- Patented by France Telecom (CNET)

- Windowed
- Pitch-synchronous
- Overlap-and-add
- Very efficient
- Can modify Hz up to two times or by half



*

Unit Selection Synthesis

- Generalization of the diphone intuition
  - Larger units
    - From diphones to sentences
  - Many many copies of each unit
    - 10 hours of speech instead of 1500 diphones (a few minutes of speech)

Unit Selection Intuition

- Given a big database
- Find the unit in the database that is the best to synthesize some target segment
- What does “best” mean?
  - “Target cost”: Closest match to the target description, in terms of
    - Phonetic context
    - F0, stress, phrase position
  - “Join cost”: Best join with neighboring units
    - Matching formants + other spectral characteristics
    - Matching energy
    - Matching F0

Targets and Target Costs

- Target cost $T(u_t,s_t)$: How well the target specification s_t matches the potential unit in the database u_t
- Features, costs, and weights
- Examples:
  - /ih-t/ +stress, phrase internal, high F0, content word
  - /n-t/ -stress, phrase final, high F0, function word
  - /dh-ax/ -stress, phrase initial, low F0, word “the”

Target Costs

- Comprised of k subcosts
  - Stress
  - Phrase position
  - F0
  - Phone duration
  - Lexical identity
- Target cost for a unit:

$$C^t(t_i, u_i) = \sum_{k=1}^{p} w_k^t C_k^t(t_i, u_i)$$

Join (Concatenation) Cost

- Measure of smoothness of join
- Measured between two database units (target is irrelevant)
- Features, costs, and weights
- Comprised of k subcosts:
  - Spectral features
  - F0
  - Energy
- Join cost:

$$C^j(u_{i-1}, u_i) = \sum_{k=1}^{p} w_k^j C_k^j(u_{i-1}, u_i)$$

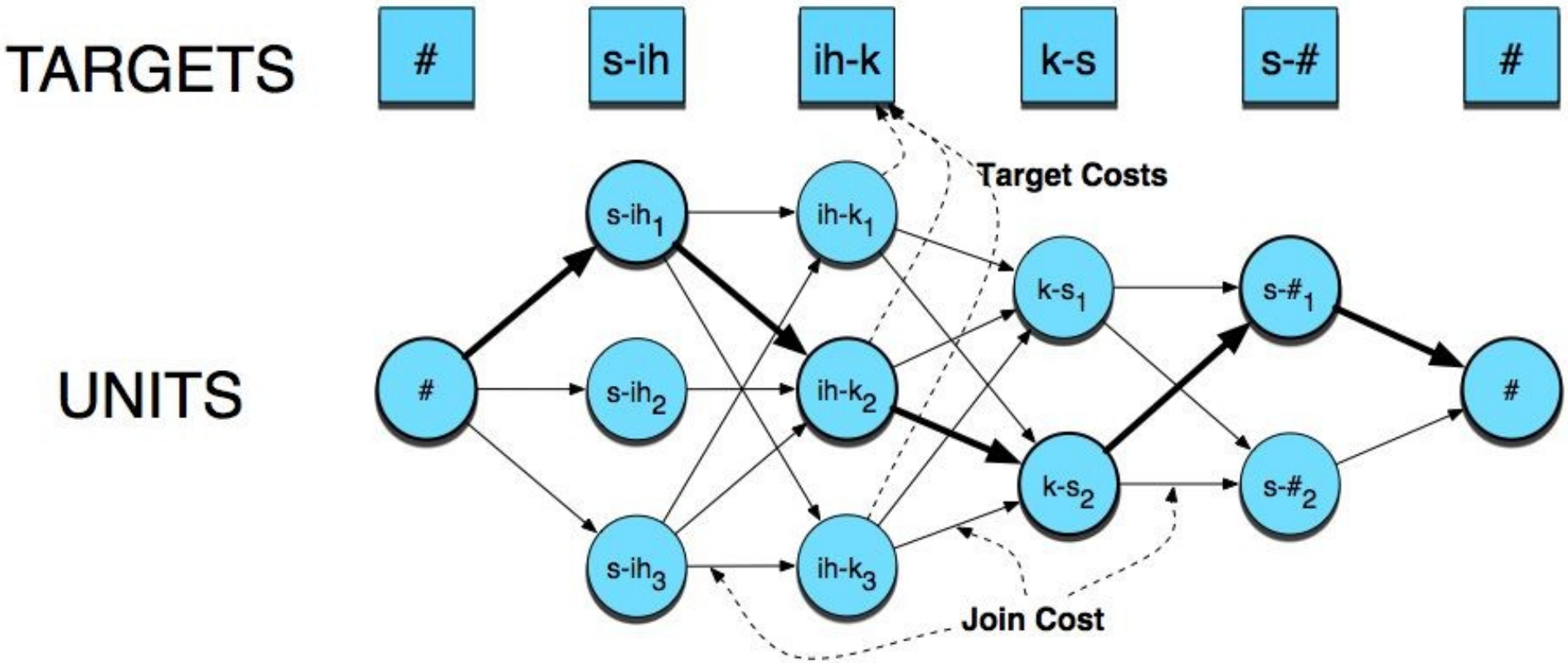
Total Costs

- Hunt and Black 1996
- We now have weights (per phone type) for features set between target and database units
- Find best path of units through database that minimize:

$$C(t_1^n, u_1^n) = \sum_{i=1}^{n} C^{target}(t_i, u_i) + \sum_{i=2}^{n} C^{join}(u_{i-1}, u_i)$$

$$\hat{u}_1^n = \underset{u_1,\ldots,u_n}{\operatorname{argmin}} C(t_1^n, u_1^n)$$

- Standard problem solvable with Viterbi search with beam width constraint for pruning


TARGETS
#
s-ih
ih-k
k-s
s-#
#
Target Costs
UNITS
s-ih1
ih-k1
k-s1
s-#1
#
s-ih2
ih-k2
k-s2
s-#2
#
s-ih3
ih-k3
Join Cost

*

Unit Selection Summary

- Advantages
  - Quality is far superior to diphones
  - Natural prosody selection sounds better
- Disadvantages:
  - Quality can be very bad in places
    - HCI problem: mix of very good and very bad is quite annoying
  - Synthesis is computationally expensive
  - Can't synthesize everything you want:
    - Diphone technique can move emphasis
    - Unit selection gives good (but possibly incorrect) result

Evaluation of TTS

- Intelligibility Tests
  - Diagnostic Rhyme Test (**DRT**) and Modified Rhyme Test (**MRT**)
    - Humans do listening identification choice between two words differing by a single phonetic feature
      - Voicing, nasality, sustenation, sibilation
    - DRT: 96 rhyming pairs
      - Dense/tense, bond/pond, etc
        - Subject hears “dense”, chooses either “dense” or “tense”
      - % of right answers is intelligibility score.
    - MRT: 300 words, 50 sets of 6 words (went, sent, bent, tent, dent, rent)
    - Embedded in **carrier phrases**:
      - Now we will say “dense” again
- **Mean Opinion Score**
  - Have listeners rate space on a scale from 1 (bad) to 5 (excellent)
- More natural:
  - Reading addresses out loud, reading news text, using two different systems.
  - Do a **preference test** (prefer A, prefer B)

Recent stuff

- Problems with Unit Selection Synthesis
  - Can't modify signal
  - (mixing modified and unmodified sounds bad)
  - But database often doesn't have exactly what you want
- Solution: HMM (Hidden Markov Model) Synthesis
  - Won recent TTS bakeoff.
  - Sounds less natural to researchers
  - But naïve subjects preferred it
  - Has the potential to improve on both diphone and unit selection.